RĪGAS METĀLAPSTRĀDES PROFESIONĀLĀ VIDUSSKOLA
Izglītības programma: Programmēšanas tehniķis

KVALIFIKĀCIJAS DARBS
“Sporta zāles grafiks”

Paskaidrojošais raksts 40 lpp.

Audzēknis: Raivis Raivo Ločmelis

Prakses vadītājs:

Nodaļas vadītājs:

Rīga 2026

ANOTĀCIJA

Kvalifikācijas darbā izstrādāta sporta zāles grafika un rezervācijas tīmekļa sistēma “Gym_Tracker2”. Darba galvenais mērķis ir izveidot ērtu un pārskatāmu sistēmu, kas palīdz organizēt nodarbību grafiku un atvieglo rezervāciju procesu gan klientiem, gan treneriem un administratoriem.

Darba sākumā tika izpētīti esošie risinājumi un analizētas problēmas, ar kurām sporta zāles saskaras ikdienā – pārpildītas nodarbības, rezervāciju kļūdas un sarežģīta grafiku pārvaldība. Balstoties uz iegūto informāciju, tika izstrādātas sistēmas funkcionālās un nefunkcionālās prasības.

Izstrādātā sistēma ļauj lietotājiem reģistrēties ar e-pasta verifikāciju, apskatīt nodarbību grafiku, iegādāties abonementu, veikt un atcelt rezervācijas. Treneriem un administratoriem ir pieejamas papildu funkcijas nodarbību izveidei, lietotāju pārvaldībai, statistikai un datu eksportam. Darbā aprakstīts sistēmas izstrādes process, izmantotās tehnoloģijas (PHP, MySQL/MariaDB, HTML, CSS, JavaScript), datubāzes struktūra un veiktā testēšana.

Rezultātā izveidota sistēma, kas padara sporta zāles darba organizēšanu vienkāršāku un pārskatāmāku.

ANNOTATION

This qualification paper describes the development of “Gym_Tracker2”, a web-based gym schedule and reservation system. The main goal of the project is to create a simple and user-friendly system that helps manage class schedules and simplifies the reservation process for clients, trainers and administrators.

At the beginning of the project, existing solutions were analysed and common problems in gym management were identified, such as overcrowded classes, booking errors and complicated schedule management. Based on this research, functional and non-functional requirements were defined.

The developed system allows users to register with e-mail verification, view the class schedule, purchase a membership, and make and cancel reservations. Trainers and administrators have additional functions for managing classes and users, viewing statistics and exporting data. The paper also describes the development process, the technologies used (PHP, MySQL/MariaDB, HTML, CSS, JavaScript), the database structure and the testing performed.

The final result is a system that makes the organisation of gym activities simpler and more transparent.

SATURS

Lpp.
IEVADS....5
1. UZDEVUMA NOSTĀDNE....7
2. PRASĪBU SPECIFIKĀCIJA....8
2.1. Funkcionālās prasības....8
2.2. Nefunkcionālās prasības....9
3. UZDEVUMA RISINĀŠANAS LĪDZEKĻU APRAKSTS UN IZVĒLES PAMATOJUMS 11
4. SISTĒMAS MODELĒŠANA UN PROJEKTĒŠANA....13
4.1. Sistēmas struktūras modelis....13
4.2. Funkcionālais un dinamiskais sistēmas modelis....15
4.3. Datu vārdnīca / datu struktūras....17
5. LIETOTĀJA CEĻVEDIS....20
5.1. Sistēmas prasības aparatūrai un programmatūrai....20
5.2. Sistēmas instalācija un palaišana....20
5.3. Darbs ar sistēmu....21
6. TESTA PIEMĒRS....26
6.1. Reģistrācijas un autentifikācijas testēšana....26
6.2. Rezervēšanas testēšana....29
6.3. Trenera un administratora funkciju testēšana....31
6.4. Abonementu testēšana....33
6.5. Testēšanas rezultātu kopsavilkums....34
7. IEGULDĪJUMS DARBĀ....36
SECINĀJUMI....37
SAĪSINĀJUMU SARAKSTS....39
LITERATŪRAS SARAKSTS....40
PIELIKUMI....41
1. pielikums. Failu struktūra....42
2. pielikums. Programmas pirmkods....43

IEVADS

Mūsdienās veselīgs dzīvesveids, fiziskās aktivitātes un regulāri treniņi kļūst par nozīmīgu sabiedrības ikdienas sastāvdaļu. Pieaugot iedzīvotāju interesei par sporta zālēm, grupu nodarbībām un individuālajiem treniņiem, sporta klubiem kļūst arvien svarīgāk nodrošināt efektīvu un pārskatāmu nodarbību plānošanas un rezervācijas sistēmu. Tradicionālās metodes – drukāti grafiki un pieraksti kladēs – vairs neatbilst mūsdienu prasībām, jo tās rada kļūdu risku, informācijas nesakritības un apgrūtina operatīvu komunikāciju ar klientiem.

Pašreizējā situācijā daudzas sporta zāles saskaras ar problēmām, kas saistītas ar nodarbību pārpildījumu vai nepietiekamu aizpildījumu, neprecīzu rezervāciju uzskaiti, kā arī sarežģītu treneru darba laika plānošanu. Klientiem bieži nav iespējas vienuviet ērti apskatīt aktuālo grafiku, pārbaudīt pieejamo vietu skaitu vai pārvaldīt savas rezervācijas. Rezultātā tiek zaudēts laiks, samazinās klientu apmierinātība un palielinās administratīvais slogs darbiniekiem.

Veicot tirgus izpēti, tika analizēti vairāki Latvijā populāri sporta klubu risinājumi, piemēram, “MyFitness” [9] un “Lemon Gym” [5], kā arī mazāku sporta studiju izmantotās rezervācijas sistēmas. “Lemon Gym” piedāvā QR koda piekļuvi klubam un savienojamību ar viedierīču lietotnēm (piemēram, Strava, Fitbit, Garmin), taču nodarbību rezervēšanas iespējas ir ierobežotas – nodarbības iespējams rezervēt tikai noteiktā laika periodā un pastāv stingri atcelšanas noteikumi [5]. Savukārt “MyFitness” nodrošina plašu klubu tīklu un atlaižu sistēmu, taču jaunajiem lietotājiem sistēma var šķist sarežģīta, kā arī netiek nodrošināta integrācija ar viedierīcēm [9]. Daudzi risinājumi ir pielāgoti lieliem sporta centriem un ir finansiāli dārgi mazākām sporta studijām.

Digitāla sporta zāles grafika sistēma ir nepieciešama, lai uzlabotu darba efektivitāti, samazinātu administratīvo slodzi, nodrošinātu precīzu datu uzskaiti un uzlabotu klientu pieredzi. Centralizēta sistēma ļauj automatizēt rezervāciju procesu, kontrolēt nodarbību noslodzi, analizēt apmeklējuma statistiku un nodrošināt drošu lietotāju datu apstrādi, ievērojot personas datu aizsardzības prasības [1; 2].

Kvalifikācijas darba mērķis ir izstrādāt sporta zāles grafika un rezervācijas sistēmu “Gym_Tracker2”, kas nodrošina efektīvu grupu nodarbību plānošanu, rezervāciju pārvaldību, treneru nodarbību administrēšanu un abonementu kontroli, piedāvājot ērtu, drošu un lietotājam draudzīgu digitālo vidi klientiem, treneriem un administratoriem.

Mērķa sasniegšanai tika izvirzīti šādi uzdevumi:

- izpētīt esošos risinājumus un formulēt sistēmas funkcionālās un nefunkcionālās prasības;
- izvēlēties un pamatot programmēšanas valodas, datubāzi un izstrādes rīkus;
- izstrādāt sistēmas arhitektūru, datubāzes struktūru un galveno procesu modeļus;
- realizēt reģistrāciju, autentifikāciju, nodarbību kalendāru, rezervācijas, abonementus un administrācijas paneli;
- sagatavot lietotāja ceļvedi un veikt programmatūras funkcionālo testēšanu.

Paskaidrojošais raksts sastāv no uzdevuma nostādnes, prasību specifikācijas, līdzekļu izvēles pamatojuma, sistēmas modelēšanas un projektēšanas nodaļas (arhitektūra, ER modelis, funkcionālais modelis un datu vārdnīca), lietotāja ceļveža, testa piemēra, ieguldījuma darbā un secinājumiem. Pielikumos pievienota failu struktūra un sistēmas pirmkods.

1. UZDEVUMA NOSTĀDNE

Kvalifikācijas darba uzdevums ir izstrādāt tīmekļa bāzētu sporta zāles grafika un rezervācijas sistēmu, kas nodrošina klientiem iespēju pārskatīt nodarbību grafiku, veikt rezervācijas un pārvaldīt savu abonementu, savukārt treneriem un administratoriem – efektīvi plānot nodarbības, kontrolēt noslodzi un apskatīt lietošanas statistiku.

Darba ietvaros sistēmai jānodrošina šādas galvenās funkcionalitātes:

- lietotāju reģistrācija ar e-pasta verifikāciju un autentifikācija (klients, treneris, administrators);
- nodarbību grafika pārskatīšana kalendāra skatā;
- nodarbību rezervēšana ar pieejamo vietu skaita kontroli;
- rezervāciju atcelšana;
- nodarbību izveide, rediģēšana un dzēšana (trenera un administratora līmenī);
- apmeklējuma statistikas apskate un datu eksports (administratora līmenī);
- abonementu sistēma un abonementa derīguma kontrole.

Sistēmas lietotāju lomas ir klients (sistēmā – “user”), treneris (“trainer”) un administrators (“admin”). Lietotāju mijiedarbība ar sistēmu attēlota lietojumgadījumu diagrammā (skat. 4.3. att., 4.2.1. apakšnodaļā).

2. PRASĪBU SPECIFIKĀCIJA

2.1. Funkcionālās prasības

Funkcionālās prasības apraksta, ko sistēmai jādara. Katrai prasībai ir identifikators (FPn), uz kuru atsaucas testēšanas nodaļa.

FP-1 Lietotāja reģistrācija

Sistēmai jānodrošina jauna lietotāja reģistrācija, ievadot vārdu, e-pasta adresi un paroli.

- Sistēmai jāpārbauda, vai e-pasta adrese nav jau reģistrēta.
- Uz norādīto e-pasta adresi tiek nosūtīta verifikācijas saite; konts tiek aktivizēts pēc saites atvēršanas.
- Ja dati ir korekti, sistēma izveido lietotāja kontu un saglabā to datubāzē (parole – hash formātā).
- Ja dati nav korekti, sistēma attēlo kļūdas paziņojumu.

FP-2 Lietotāja autentifikācija un paroles atjaunošana

Sistēmai jānodrošina lietotāja pieslēgšanās, izmantojot e-pastu un paroli, un aizmirstas paroles atjaunošana.

- Ja ievadītie dati ir pareizi un e-pasts ir verificēts, lietotājs tiek novirzīts uz sākumlapu.
- Ja dati ir nepareizi, sistēma attēlo kļūdas paziņojumu; pēc 5 neveiksmīgiem mēģinājumiem konts tiek uz laiku bloķēts.
- Paroles atjaunošanai uz e-pastu tiek nosūtīta saite ar ierobežotu derīguma termiņu.

FP-3 Nodarbību grafika apskate

Sistēmai jānodrošina iespēja apskatīt nodarbību grafiku kalendāra skatā.

- Jāattēlo nodarbības nosaukums, datums, laiks, treneris un pieejamo vietu skaits.
- Grafikam jāatjaunojas bez lapas pārlādes pēc nodarbību izmaiņām (izmantojot servera sūtītos notikumus – Server-Sent Events).

FP-4 Nodarbības rezervēšana

Klientam jābūt iespējai rezervēt nodarbību.

- Rezervācija iespējama tikai tad, ja nodarbība vēl nav sākusies, ir brīvas vietas un klientam ir aktīvs Advanced vai Pro abonements.
- Sistēma neļauj izveidot divas rezervācijas vienai nodarbībai vienam lietotājam.
- Pēc rezervācijas pieejamo vietu skaits samazinās par vienu, un lietotājs saņem apstiprinājuma paziņojumu.

FP-5 Rezervācijas atcelšana

Klientam jābūt iespējai atcelt savu rezervāciju līdz nodarbības sākumam.

- Pēc atcelšanas vietu skaits tiek automātiski atjaunots.
- Rezervāciju nav iespējams atcelt nodarbībai, kas jau ir sākusies vai beigusies.

FP-6 Nodarbību pārvaldība (treneris)

Trenerim jābūt iespējai izveidot, rediģēt un dzēst savas nodarbības, norādot nosaukumu, aprakstu, datumu, laiku un vietu skaitu.

- Trenerim jāspēj apskatīt rezervēto lietotāju sarakstu un eksportēt to failā.

FP-7 Statistika un datu eksports (administrators)

Administratoram jābūt iespējai apskatīt vietnes apmeklējuma un pieslēgšanās statistiku un eksportēt datus.

- Administrācijas panelī tiek attēlots apmeklējumu un pieslēgšanās reižu skaits un populārākās lapas (statistika tiek uzkrāta tikai pēc lietotāja piekrišanas sīkdatņu izmantošanai).
- Datus iespējams eksportēt Excel un PDF formātā.

FP-8 Lietotāju un nodarbību administrēšana (administrators)

Administratoram jābūt iespējai pārvaldīt lietotājus un jebkuras nodarbības.

- Lietotāju meklēšana, vārda un lomas maiņa, lietotāja dzēšana.
- Jebkuras nodarbības izveide, rediģēšana un dzēšana.

FP-9 Abonementu pārvaldība

Sistēmai jānodrošina abonementa iegāde, derīguma termiņa kontrole un abonementa piešķiršana administratora līmenī.

- Pieejami trīs abonementu plāni: Basic, Advanced un Pro.
- Abonements tiek aktivizēts uz 31 dienu; pēc termiņa beigām rezervēšana tiek liegta līdz abonementa atjaunošanai.
- Maksājumu dati tiek glabāti tabulā “payments”.

2.2. Nefunkcionālās prasības

Nefunkcionālās prasības apraksta sistēmas kvalitātes rādītājus. Veiktspējas un pieejamības rādītāji ir mērķa lielumi; to izpildes pārbaude (slodzes testi, darbības laika monitorings) šajā darbā nav veikta, tāpēc tie nav uzskatāmi par pierādītiem rezultātiem.

NFP-1 Veiktspēja

- Mērķis: lapas ielāde ne ilgāk kā 3 sekundēs, ja sistēmu vienlaicīgi izmanto līdz 100 lietotājiem.

NFP-2 Drošība

- Lietotāju paroles tiek glabātas šifrētā veidā (PHP funkcija password_hash) [13; 12].
- Visām datu mainīgajām darbībām tiek izmantoti CSRF žetoni [11], datubāzes vaicājumi tiek veidoti kā sagatavoti izteikumi (prepared statements).
- Pēc 5 neveiksmīgiem pieslēgšanās mēģinājumiem konts tiek bloķēts uz 15 minūtēm; sesijas sīkdatne tiek atzīmēta kā httponly un (HTTPS vidē) kā secure.
- Ražošanas vidē jānodrošina HTTPS savienojums servera līmenī.

NFP-3 Lietojamība

- Lietotāja interfeisam jābūt responsīvam un pielāgotam ierīcēm ar ekrāna platumu no 320 px līdz 1920 px.

NFP-4 Pieejamība

- Mērķis: sistēmai jābūt pieejamai 99 % laika mēneša griezumā, izņemot plānotos apkopes darbus.

NFP-5 Daudzvalodība

- Saskarne pieejama latviešu, angļu un krievu valodā.

NFP-6 Privātums

- Lietotāju darbību statistika tiek uzkrāta tikai pēc sīkdatņu piekrišanas (cookie popup), ievērojot personas datu aizsardzības prasības [1; 2].

3. UZDEVUMA RISINĀŠANAS LĪDZEKĻU APRAKSTS UN IZVĒLES PAMATOJUMS

Izstrādātā platforma ir tīmekļa bāzēta sporta zāles grafika un rezervācijas sistēma, kas paredzēta nodarbību plānošanai, rezervāciju veikšanai un sporta zāles administrēšanai. Sistēma sastāv no divām galvenajām daļām – priekšgala (lietotāja saskarne) un aizmugursistēmas (servera loģika un datubāze). Priekšgala un aizmugursistēmas izstrādē tika izmantotas šādas tehnoloģijas:

- **HTML (HyperText Markup Language)** – iezīmēšanas valoda, ar kuras palīdzību tika veidotas lapu struktūras, formas, tabulas un ievades lauki [7];
- **CSS (Cascading Style Sheets)** – stilu valoda vienota dizaina un responsīva izkārtojuma izveidei, lai sistēmu varētu ērti lietot datorā, planšetē un telefonā [7];
- **JavaScript** – programmēšanas valoda, ar kuru tika realizēta formu datu pārbaude, kalendāra attēlošana (fails `assets/js/calendar.js`), paziņojumi un grafika atjaunināšana bez lapas pārlādes [7];
- **Server-Sent Events (SSE)** – tehnoloģija, ar kuru serveris nosūta pārlūkam nodarbību izmaiņas, nodrošinot kalendāra atjaunošanu bez manuālas pārlādes (fails `admin/classes_sse.php`) [8];
- **PHP 8.2** – servera puses valoda, ar kuru tika izstrādāta autentifikācija, rezervāciju apstrāde, nodarbību pievienošana un rediģēšana, eksports un darbs ar datubāzi [13];
- **MySQL / MariaDB** – relāciju datubāzes pārvaldības sistēma lietotāju, nodarbību, rezervāciju un maksājumu datu glabāšanai; sistēma ir saderīga ar MySQL 8.0 [10];
- **PHPMailer** – PHP bibliotēka e-pasta sūtīšanai (verifikācijas un paroles atjaunošanas saites) [14];
- **Composer** – PHP atkarību pārvaldnieks, ar ko tiek instalētas projekta bibliotēkas [3];
- **Stripe** – maksājumu platforma, kuras konfigurācijas parametri (atslēgas, webhook noslēpums) ir paredzēti projekta konfigurācijā maksājumu integrācijai [15].

Tehnoloģiju izvēle tika veikta, jo tās ir uzticamas, plaši izmantotas un savstarpēji labi saderīgas. Tās ļauj izveidot funkcionālu, drošu un pārskatāmu sistēmu bez sarežģītu ietvaru (framework) izmantošanas, kas atvieglo koda uzturēšanu un turpmāku pilnveidošanu. Tā kā projekts paredzēts nelielai vai vidējai sporta zālei, izvēlētais risinājums nodrošina pietiekamu veiktspēju un vienkāršu administrēšanu; PHP un MySQL ir pieejami lielākajā daļā tīmekļa hostingu, kas samazina ieviešanas izmaksas.

3.1. tabula

Izmantotās tehnoloģijas un to uzdevums

Tehnoloģija	*Uzdevums sistēmā*	*Galvenie faili*
HTML / CSS	Lapu struktūra un vizuālais noformējums	`views/`, `assets/css/`
JavaScript	Kalendārs, formu validācija, paziņojumi	`assets/js/calendar.js`
PHP 8.2	Biznesa loģika, sesijas, eksports	`api/`, `admin/`, `export/`
MySQL / MariaDB	Datu glabāšana	`api/db.php`
PHPMailer	E-pasta verifikācija, paroles atjaunošana	`api/Mailer.php`
SSE	Kalendāra atjaunošana reāllaikā	`admin/classes_sse.php`

Sistēmas dizains tika izstrādāts, koncentrējoties uz lietošanas ērtumu un saprotamu navigāciju. Saskarne ir minimālistiska un intuitīva, lai lietotāji varētu ātri atrast nepieciešamo informāciju, apskatīt grafiku un veikt rezervācijas bez sarežģītām darbībām. Īpaša uzmanība tika pievērsta responsivitātei un daudzvalodībai (latviešu, angļu, krievu).

Sistēmas izstrādes procesā tika izmantotas šādas izstrādes vides un rīki:

– **Git** – versiju kontroles sistēma pirmkoda pārvaldībai un izmaiņu vēstures saglabāšanai [4];
– **Visual Studio Code** – koda redaktors ar plašu paplašinājumu klāstu, kas atbalsta visas projektā izmantotās tehnoloģijas [6].

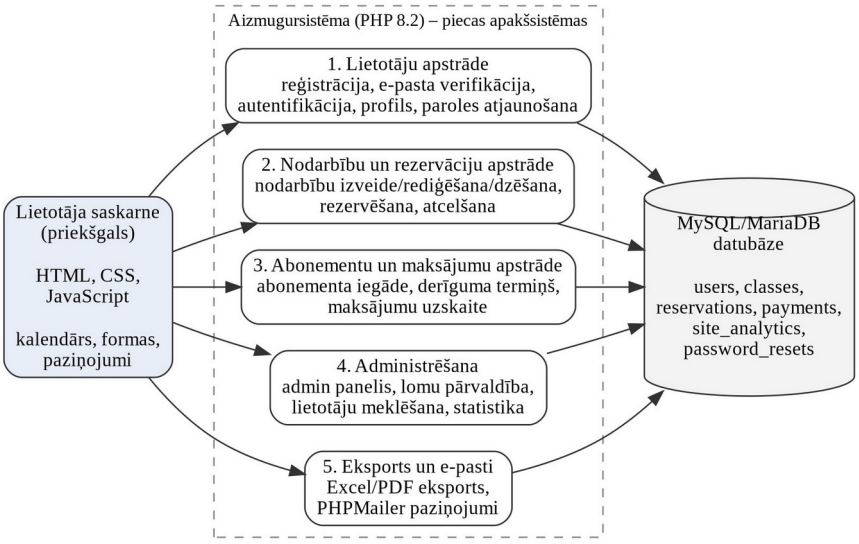
Izvēlētais tehnoloģiskais risinājums nodrošina kvalifikācijas darbā izvirzītā mērķa sasniegšanu – drošas, efektīvas un lietotājam draudzīgas sporta zāles grafika un rezervācijas sistēmas izveidi.

4. SISTĒMAS MODELĒŠANA UN PROJEKTĒŠANA

4.1. Sistēmas struktūras modelis

4.1.1. Sistēmas arhitektūra

Sistēma ir veidota kā trīs līmeņu tīmekļa lietojumprogramma: lietotāja saskarne (HTML, CSS, JavaScript), servera loģika (PHP) un datu glabāšanas līmenis (MySQL/MariaDB). Servera loģika sadalīta piecās apakšsistēmās (skat. 4.1. att.).



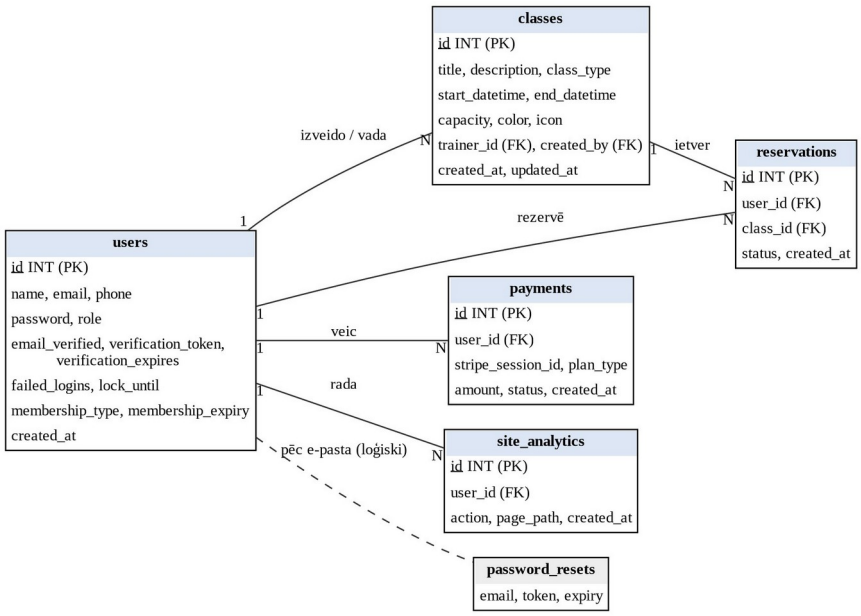
4.1. att. Sistēmas “Gym_Tracker2” arhitektūra

Apakšsistēmu uzdevumi ir šādi:

- **Lietotāju apstrāde** – jauna lietotāja konta izveide, e-pasta verifikācija, pieslēgšanās un izslēgšanās, profila pārvaldība, paroles atjaunošana (faili `register.php`, `login.php`, `verify_email.php`, `forgot_password.php`, `reset_password.php`, `profile.php`);
- **Nodarbību un rezervāciju apstrāde** – nodarbību izveide, rediģēšana un dzēšana, rezervēšana un atcelšana, dalībnieku saraksts (`api/save_class.php`, `api/reserve_class.php`, `api/cancel_reservation.php`, `api/get_classes.php`, `api/get_participants.php`);
- **Abonementu un maksājumu apstrāde** – abonementa plāna izvēle, derīguma termiņa uzstādīšana, maksājumu uzskaite (`membership_offer.php`, `api/buy_membership.php`, tabula payments);
- **Administrēšana** – administrācijas panelis ar statistiku, lietotāju meklēšana, lomu maiņa, abonementu piešķiršana (`admin/` mape, `api/admin_assign_membership.php`);
- **Eksports un e-pasti** – dalībnieku, lietotāju un rezervāciju eksports Excel un PDF formātā (`export/` mape), e-pastu sūtīšana ar PHPMailer (`api/Mailer.php`).

4.1.2. Sistēmas ER modelis

ER modelis (skat. 4.2. att.) veidots atbilstoši datubāzei, ko sistēma izmanto reāli. Datubāzē ir seši galvenie datu apjomi: lietotāji (users), nodarbības (classes), rezervācijas (reservations), maksājumi (payments), vietnes statistika (site_analytics) un paroles atjaunošanas pieprasījumi (password_resets). Entītijas “Sportists”, “Treneris” un “Klients” sistēmā nav atsevišķas tabulas – tās ir lietotāji ar dažādām lomām (lauks users.role).



4.2. att. Datubāzes ER diagramma

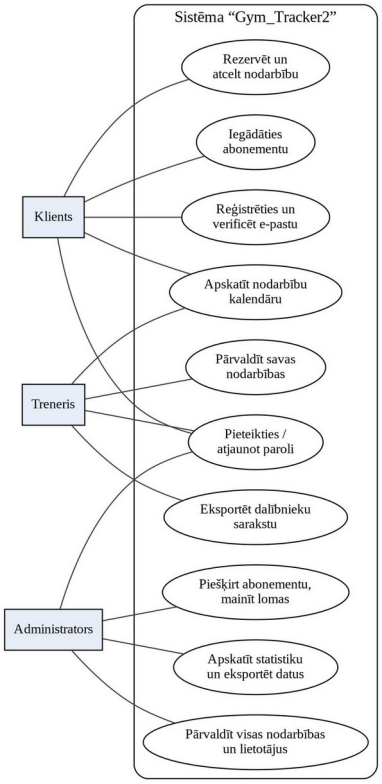
Attiecības starp entītijām:

- **users – classes (1 : N):** viens treneris vai administrators var izveidot un vadīt vairākas nodarbības (classes.created_by, classes.trainer_id), bet katrai nodarbībai ir viens izveidotājs;
- **users – reservations (1 : N):** viens lietotājs var veikt vairākas rezervācijas, katra rezervācija pieder vienam lietotājam (reservations.user_id);
- **classes – reservations (1 : N):** uz vienu nodarbību var pieteikties vairāki lietotāji, katra rezervācija attiecas uz vienu nodarbību (reservations.class_id); starp users un classes tādējādi veidojas attiecība “daudzi pret daudziem”, kas realizēta ar saistošo tabulu reservations;
- **users – payments (1 : N):** viens lietotājs var veikt vairākus maksājumus (payments.user_id);
- **users – site_analytics (1 : N):** katram lietotājam tiek reģistrētas apmeklējuma un pieslēgšanās darbības (site_analytics.user_id);
- **password_resets:** neatkarīga tabula, kas ar lietotāju saistīta loģiski – pēc e-pasta adreses.

4.2. Funkcionālais un dinamiskais sistēmas modelis

4.2.1. Lietojumgadījumu diagramma

Lietojumgadījumu diagramma (skat. 4.3. att.) parāda trīs lietotāju lomas un funkcijas, kuras katrai lomai ir pieejamas. Trenera un administratora loma ietver arī klienta pamatfunkcijas – pieslēgšanos un kalendāra apskati.



4.3. att. Lietojumgadījumu diagramma

4.1. tabula

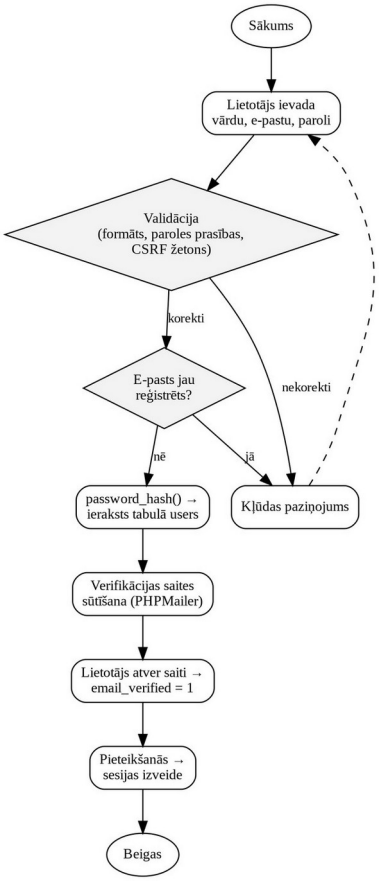
Lomu piekļuves tiesības

Funkcija	***Klients***	***Treneris***	***Administrators***
Reģistrācija, pieslēgšanās, paroles atjaunošana	jā	jā	jā
Kalendāra apskate	jā	jā	jā
Nodarbības rezervēšana un atcelšana	jā (Advanced/Pro)	–	–
Abonementa iegāde	jā	–	–
Savu nodarbību izveide, rediģēšana, dzēšana	–	jā	jā
Jebkuras nodarbības pārvaldība	–	–	jā
Dalībnieku saraksta eksports	–	savām nodarbībām	jā
Lietotāju pārvaldība, abonementu piešķiršana, statistika	–	–	jā

4.2.2. Reģistrācijas un autentifikācijas process

Jauns lietotājs izveido kontu, aizpildot reģistrācijas formu (vārds, e-pasts, parole). Sistēma veic datu validāciju – pārbauda e-pasta formātu, tā unikālitāti un paroles prasību izpildi. Ja dati ir korekti, parole tiek šifrēta ar funkciju password_hash, lietotāja ieraksts tiek saglabāts

datubāzē, un uz e-pastu tiek nosūtīta verifikācijas saite (skat. 4.4. att.). Verifikācijas žetons ir derīgs ierobežotu laiku (lauks verification_expires); līdz e-pasta verifikācijai pieslēgšanās nav iespējama.

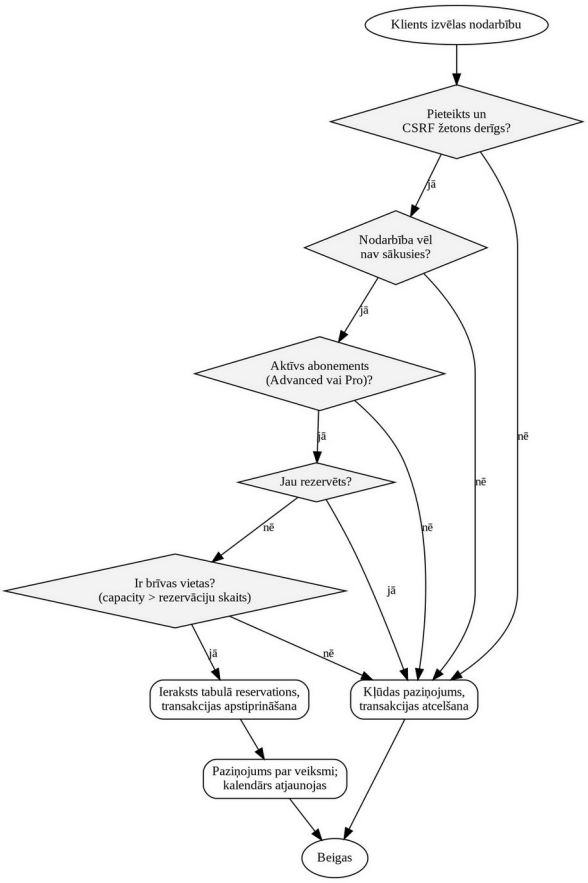


4.4. att. Reģistrācijas procesa algoritma shēma

Pieslēdzoties sistēma pārbauda e-pasta un paroles atbilstību. Katrs neveiksmīgs mēģinājums palielina skaitītāju failed_logins; pēc 5 neveiksmīgiem mēģinājumiem laukā lock_until tiek ierakstīts laiks 15 minūtes uz priekšu, un konts šajā laikā ir bloķēts. Veiksmīgas pieslēgšanās gadījumā skaitītājs tiek nullēts un tiek izveidota sesija. Paroles atjaunošanai tiek izveidots žetons tabulā password_resets ar 1 stundas derīguma termiņu un nosūtīta atjaunošanas saite.

4.2.3. Nodarbības rezervēšanas process

Rezervēšana tiek veikta vienas datubāzes transakcijas ietvaros. Nodarbības ieraksts tiek bloķēts ar vaicājumu SELECT ... FOR UPDATE, tādējādi divi lietotāji vienlaicīgi nevar aizņemt pēdējo brīvo vietu. Secīgi tiek pārbaudīts: vai lietotājs ir pieteicies un CSRF žetons ir derīgs, vai nodarbība vēl nav sākusies, vai abonements ir derīgs un tā plāns ir Advanced vai Pro, vai lietotājs jau nav rezervējis šo nodarbību un vai ir brīvas vietas (skat. 4.5. att.). Atceļot rezervāciju, ieraksts tiek dzēsts, un vieta kļūst pieejama citiem; atcelšana nav iespējama pēc nodarbības sākuma.



4.5. att. Nodarbības rezervēšanas algoritma shēma

4.2.4. Abonementu process

Sistēmā ir trīs abonementu plāni (skat. 4.2. tabulu). Izvēloties plānu, lietotājam tiek uzstādīts abonementa veids (users.membership_type) un derīguma termiņš 31 dienu uz priekšu (users.membership_expiry). Administrators abonementu var piešķirt vai atcelt arī manuāli. Rezervēšana ir pieejama tikai ar derīgu Advanced vai Pro abonementu.

4.2. tabula

Abonementu plāni

Plāns	*Cena*	*Iekļauts*
Basic	20 EUR	Piekļuve sporta zālei
Advanced	50 EUR	Piekļuve zālei un grupu nodarbību rezervēšana
Pro	70 EUR	Zāle, grupu nodarbības un treneris

4.3. Datu vārdnīca / datu struktūras

Sistēmas datubāze ir izveidota, izmantojot MySQL/MariaDB relāciju datubāzu pārvaldības sistēmu [10]. Datubāze sastāv no sešām tabulām: users, classes, reservations, payments, site_analytics un password_resets. Tabulu apraksti sniegti 4.3.–4.8. tabulā.

4.3. tabula

Tabula “users” – sistēmas lietotāji

Nr.	*Lauks*	*Datu tips*	*Apraksts*
1.	id	INT	Lietotāja unikālais identifikators (PK)

Nr.	*Lauks*	*Datu tips*	*Apraksts*
2.	name	VARCHAR(100)	Lietotāja vārds
3.	email	VARCHAR(150)	E-pasta adrese
4.	pending_email	VARCHAR(150)	Jauna e-pasta adrese, kas gaida apstiprinājumu
5.	phone	VARCHAR(20)	Telefona numurs
6.	password	VARCHAR(255)	Šifrēta parole (hash)
7.	role	ENUM('user','trainer','admin')	Lietotāja loma sistēmā
8.	email_verified	TINYINT(1)	E-pasta verifikācijas statuss
9.	verification_token	VARCHAR(255)	Verifikācijas žetons
10.	verification_expires	DATETIME	Verifikācijas žetona derīguma termiņš
11.	failed_logins	INT	Neveiksmīgu pieslēgšanās mēģinājumu skaits
12.	lock_until	DATETIME	Konta bloķēšanas beigu laiks
13.	created_at	TIMESTAMP	Konta izveides datums
14.	membership_type	VARCHAR(50)	Abonementa veids (basic, advanced, pro)
15.	membership_expiry	DATETIME	Abonementa derīguma termiņš

4.4. tabula

Tabula “classes” – nodarbības

Nr.	*Lauks*	*Datu tips*	*Apraksts*
1.	id	INT	Nodarbības identifikators (PK)
2.	title	VARCHAR(150)	Nodarbības nosaukums
3.	description	TEXT	Nodarbības apraksts
4.	trainer_id	INT	Trenera identifikators (FK → users.id)
5.	start_datetime	DATETIME	Nodarbības sākuma laiks
6.	end_datetime	DATETIME	Nodarbības beigu laiks
7.	capacity	INT	Maksimālais dalībnieku skaits
8.	color	VARCHAR(20)	Krāsa kalendāra attēlošanai
9.	class_type	VARCHAR(50)	Nodarbības tips
10.	created_by	INT	Izveidotāja identifikators (FK → users.id)
11.	created_at	TIMESTAMP	Ieraksta izveides datums
12.	updated_at	TIMESTAMP	Pēdējo izmaiņu datums
13.	icon	VARCHAR(10)	Nodarbības ikona

4.5. tabula

Tabula “reservations” – rezervācijas

Nr.	*Lauks*	*Datu tips*	*Apraksts*
1.	id	INT	Rezervācijas identifikators (PK)
2.	user_id	INT	Lietotāja identifikators (FK → users.id)
3.	class_id	INT	Nodarbības identifikators (FK → classes.id)
4.	status	ENUM('reserved','cancelled','attended')	Rezervācijas statuss
5.	created_at	TIMESTAMP	Rezervācijas izveides datums

4.6. tabula

Tabula “payments” – maksājumi

Nr.	*Lauks*	*Datu tips*	*Apraksts*
1.	id	INT	Maksājuma identifikators (PK)
2.	user_id	INT	Lietotāja identifikators (FK → users.id)
3.	stripe_session_id	VARCHAR(255)	Stripe maksājuma sesijas identifikators
4.	plan_type	VARCHAR(50)	Abonementa plāna veids
5.	amount	INT	Maksājuma summa
6.	status	VARCHAR(50)	Maksājuma statuss
7.	created_at	DATETIME	Maksājuma datums

4.7. tabula

Tabula “site_analytics” – vietnes statistika

Nr.	*Lauks*	*Datu tips*	*Apraksts*
1.	id	INT	Ieraksta identifikators (PK)
2.	user_id	INT	Lietotāja identifikators (FK → users.id)
3.	action	ENUM('visit','login')	Veiktā darbība
4.	page_path	VARCHAR(255)	Apmeklētās lapas adrese
5.	created_at	DATETIME	Darbības laiks

4.8. tabula

Tabula “password_resets” – paroles atjaunošana

Nr.	*Lauks*	*Datu tips*	*Apraksts*
1.	email	VARCHAR(255)	Lietotāja e-pasta adrese
2.	token	VARCHAR(255)	Paroles atjaunošanas žetons
3.	expiry	DATETIME	Žetona derīguma termiņš

4.3.1. Tabulu savstarpējās attiecības

Tabulas ir saistītas ar primārajām (PK) un ārējām (FK) atslēgām, kas nodrošina datu integritāti (skat. 4.9. tabulu un 4.2. att.).

4.9. tabula

Tabulu attiecības

Primārā atslēga	*Ārējā atslēga*	*Tips*	*Nozīme*
users.id	classes.created_by	1 : N	Lietotājs izveido nodarbības
users.id	classes.trainer_id	1 : N	Treneris vada nodarbības
users.id	reservations.user_id	1 : N	Lietotājs veic rezervācijas
classes.id	reservations.class_id	1 : N	Nodarbībai ir vairākas rezervācijas
users.id	payments.user_id	1 : N	Lietotājs veic maksājumus
users.id	site_analytics.user_id	1 : N	Lietotāja darbību vēsture

5. LIETOTĀJA CEĻVEDIS

5.1. Sistēmas prasības aparatūrai un programmatūrai

Tā kā sporta zāles nodarbību plānošanas sistēma ir tīmekļa lietotne, to var izmantot lielākajā daļā ierīču, ja tās atbilst šādām prasībām:

- **Operētājsistēma:** Microsoft Windows 10 vai jaunāka; Apple macOS 10.15 vai jaunāka; jebkura moderna Linux distribūcija; Android 10 vai jaunāka; iOS 14 vai jaunāka.
- **Interneta pārlūks:** Google Chrome 89+, Mozilla Firefox 86+, Apple Safari 14+, Microsoft Edge 89+ vai jebkurš cits moderns Chromium bāzēts pārlūks.

Pilnvērtīgai darbībai pārlūkam jāatbalsta JavaScript un sīkdatnes (cookies), kā arī nepieciešams stabils interneta savienojums.

Servera pusē nepieciešams: Git; PHP 8.2 vai jaunāka versija; MySQL 8.0 vai jaunāka (vai saderīga MariaDB) versija; Apache vai Nginx tīmekļa serveris; Composer pakotņu pārvaldnieks [3].

5.2. Sistēmas instalācija un palaišana

1. solis – projekta koda iegūšana. Terminālī klonējiet projekta repozitoriju un pārejiet uz tā mapi:

```
git clone https://github.com/rayWeee/SportaZ-les-Grafiks.git
cd SportaZ-les-Grafiks
```

2. solis – vides konfigurēšana. Nokopējiet konfigurācijas parauga failu un norādiet datubāzes savienojuma parametrus (servera adrese, datubāzes nosaukums gym_tracker2, lietotājvārds, parole), e-pasta (SMTP) piekļuves datus verifikācijas vēstulēm un, ja nepieciešams, Stripe atslēgas:

```
cp .env.example .env
```

3. solis – datubāzes izveide. MySQL pārvaldības rīkā (phpMyAdmin vai MySQL Workbench) izveidojiet datubāzi un importējiet sistēmas SQL failu ar tabulām un sākotnējiem datiem:

```
CREATE DATABASE gym_tracker2;
```

4. solis – atkarību instalēšana. Projekta mapē izpildiet komandu:

```
composer install
```

5. solis – palaišana. Izstrādes vidē projektu var palaist ar PHP iebūvēto serveri (maršrutēšanu nodrošina fails router.php):

```
php -S localhost:8000 router.php
```

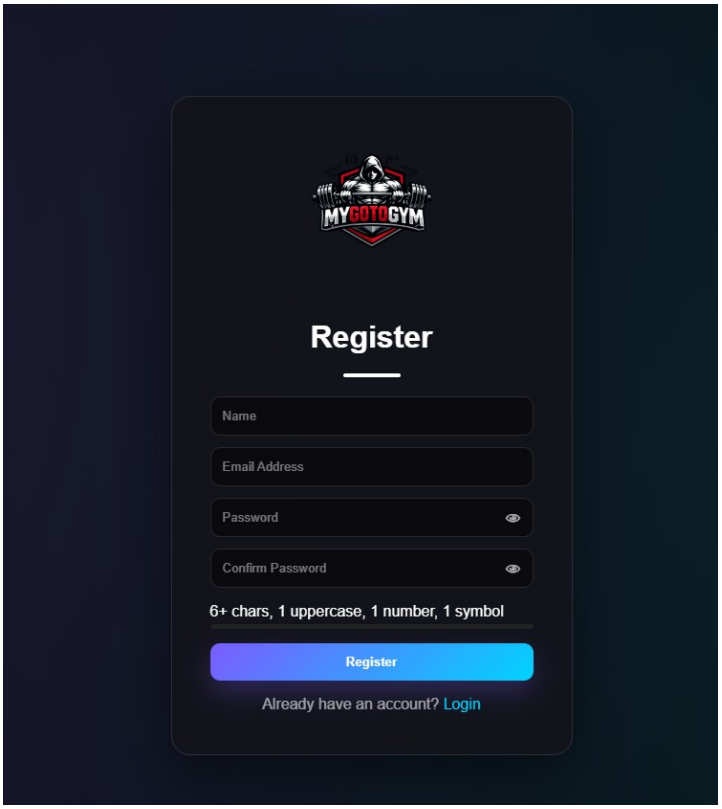
Alternatīvi projektu var izvietot Apache vai Nginx serverī, norādot projekta mapi kā vietnes saknes direktoriju. Pēc palaišanas sistēma ir pieejama adresē http://localhost:8000 vai servera konfigurācijā norādītajā adresē. Ja rodas problēmas, jāpārbauda datubāzes konfigurācija, tīmekļa servera iestatījumi un servera kļūdu žurnālfaili.

5.3. Darbs ar sistēmu

5.3.1. Klienta darbības

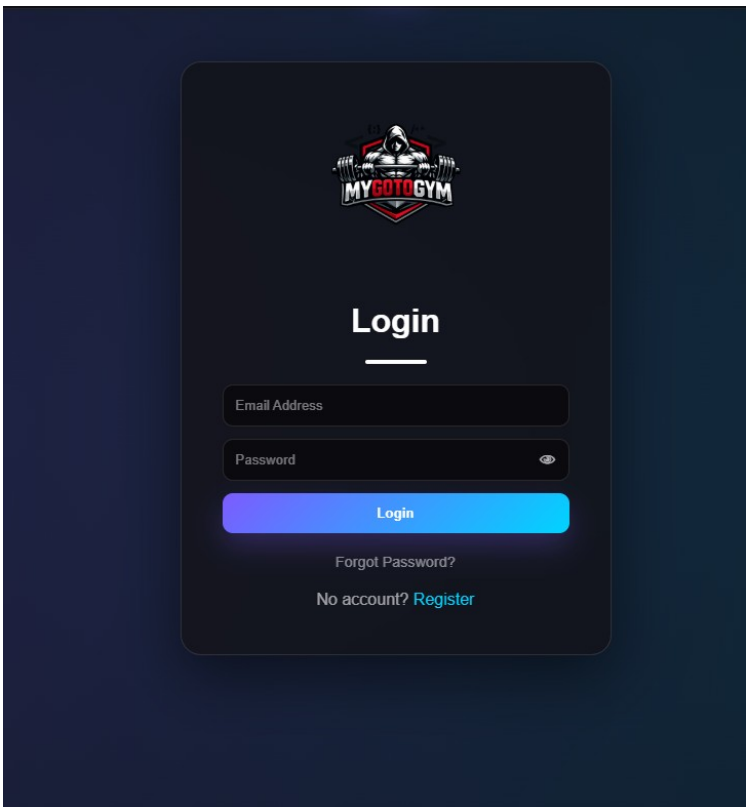
Klients var reģistrēties, pieteikties, apskatīt nodarbību kalendāru, iegādāties abonementu, rezervēt un atcelt nodarbības un pārvaldīt savu profilu.

Reģistrācija. Sākumlapā izvēlieties “Reģistrēties”, ievadiet vārdu, e-pasta adresi un paroli un apstipriniet formu (skat. 5.1. att.). Uz norādīto e-pastu tiks nosūtīta verifikācijas saite; pēc tās atvēršanas konts tiek aktivizēts.



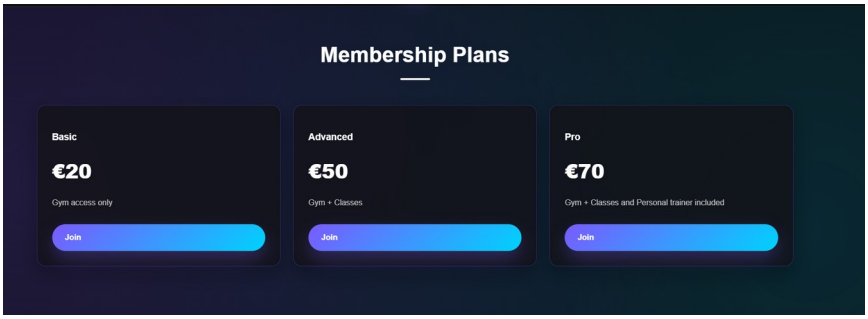
5.1. att. Reģistrācijas forma

Pieslēgšanās. Ievadiet e-pastu un paroli (skat. 5.2. att.). Ja parole aizmirsta, izmantojiet saiti “Aizmirsāt paroli?”, lai saņemtu paroles atjaunošanas saiti e-pastā.



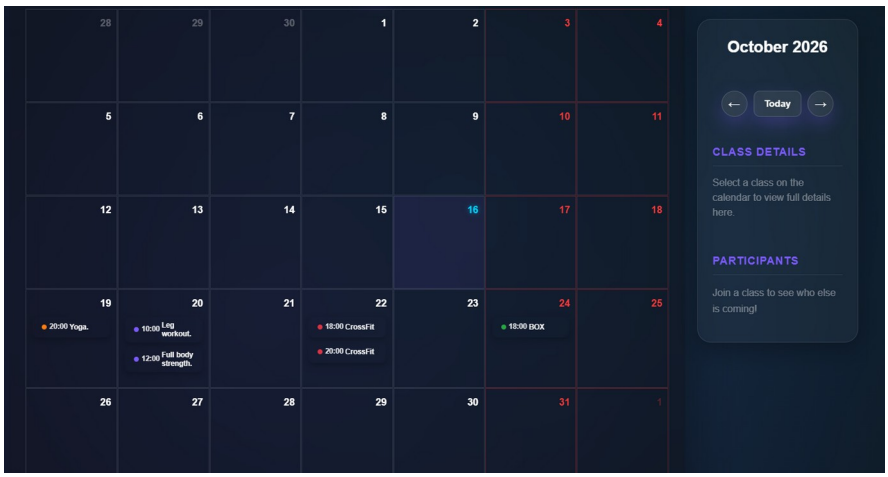
5.2. att. Pieslēgšanās forma

Abonementa iegāde. Sadaļā ar abonementu piedāvājumu izvēlieties plānu (Basic, Advanced vai Pro) un apstipriniet iegādi (skat. 5.3. att.). Rezervēt nodarbības var tikai ar Advanced vai Pro abonementu.

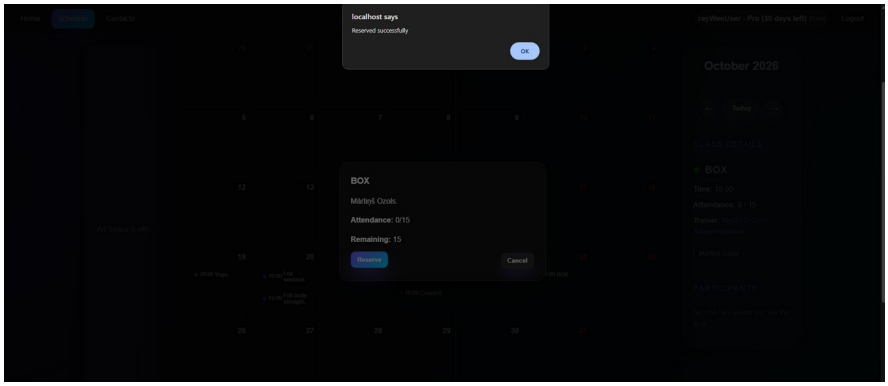


5.3. att. Abonementu izvēle

Kalendārs un rezervēšana. Kalendārā tiek attēlotas pieejamās nodarbības ar nosaukumu, laiku, treneri un brīvo vietu skaitu. Izvēloties nodarbību, tiek atvērta tās informācija un poga rezervācijai (skat. 5.4. un 5.5. att.). Rezervāciju var atcelt līdz nodarbības sākumam.

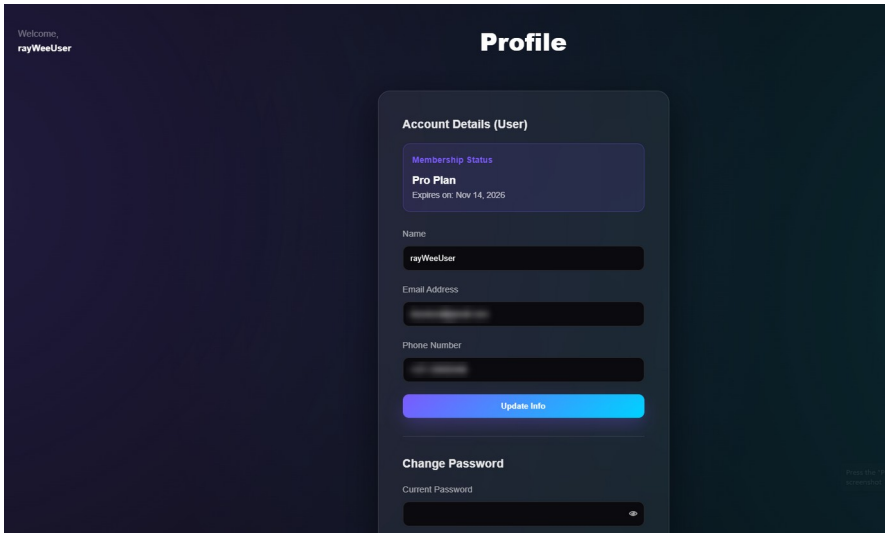


5.4. att. Nodarbību kalendārs



5.5. att. Nodarbības rezervēšana un atcelšana

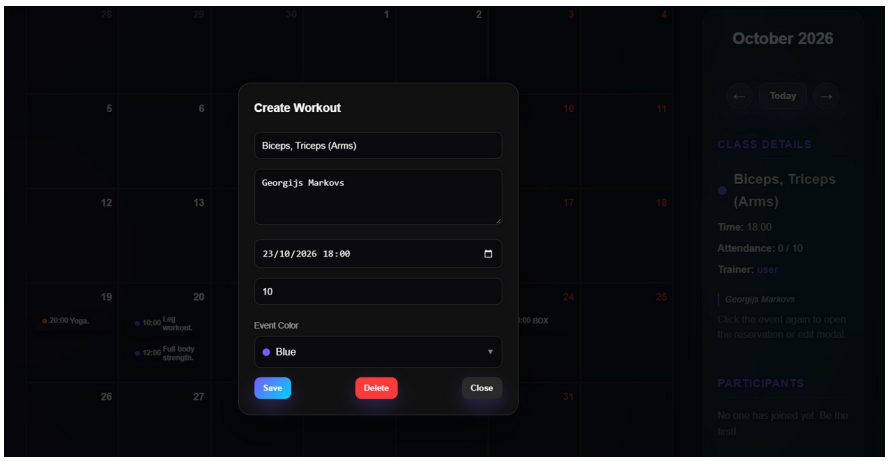
Profils. Profila lapā lietotājs var mainīt savus datus un apskatīt abonementa informāciju un derīguma termiņu (skat. 5.6. att.).



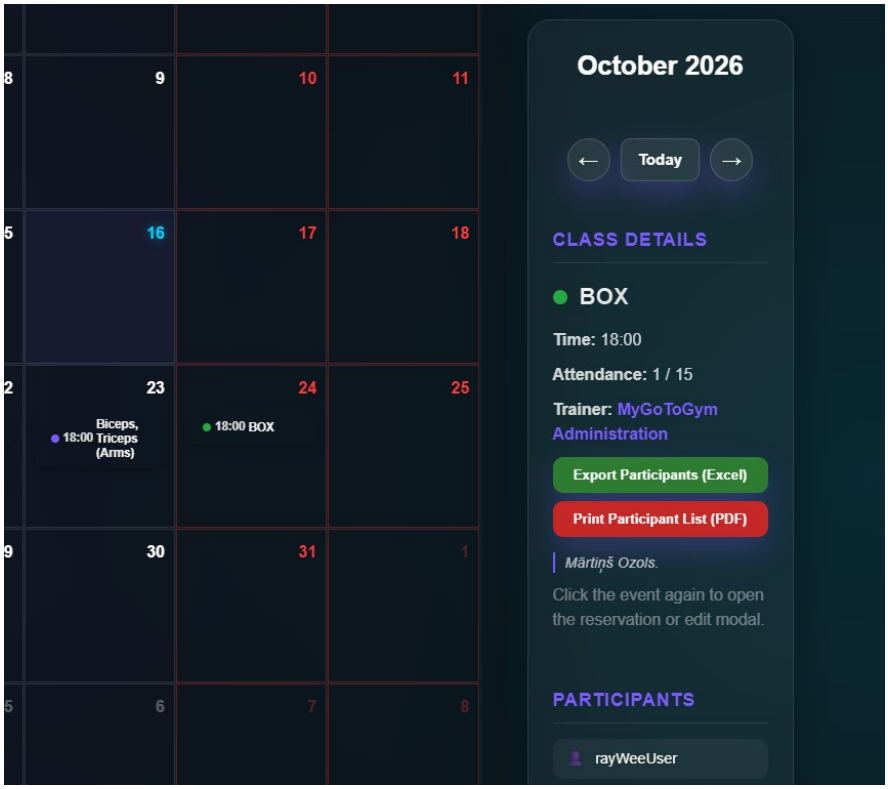
5.6. att. Lietotāja profils

5.3.2. Trenera darbības

Treneris var izveidot, rediģēt un dzēst tikai savas nodarbības, apskatīt tām pieteikušos lietotāju sarakstu un eksportēt to failā (skat. 5.7. un 5.8. att.).



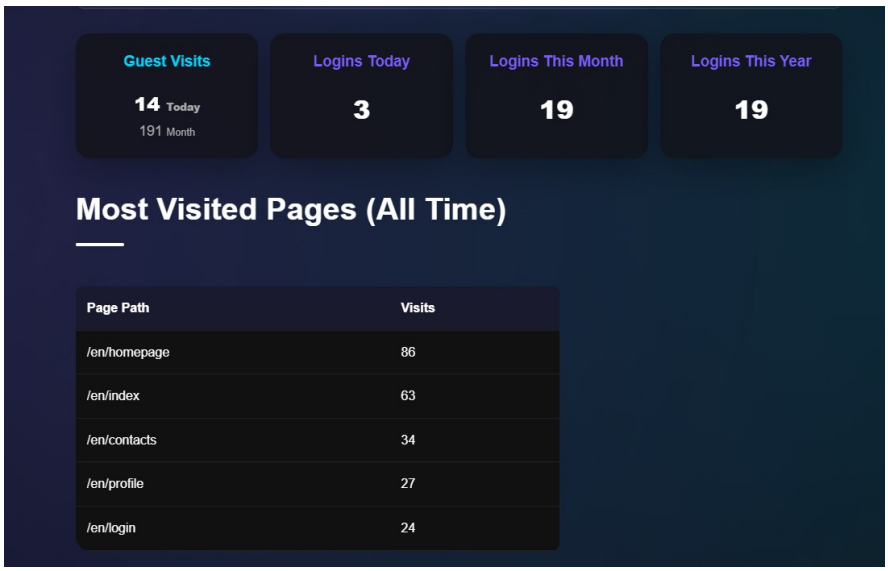
5.7. att. Nodarbības izveide (treneris)



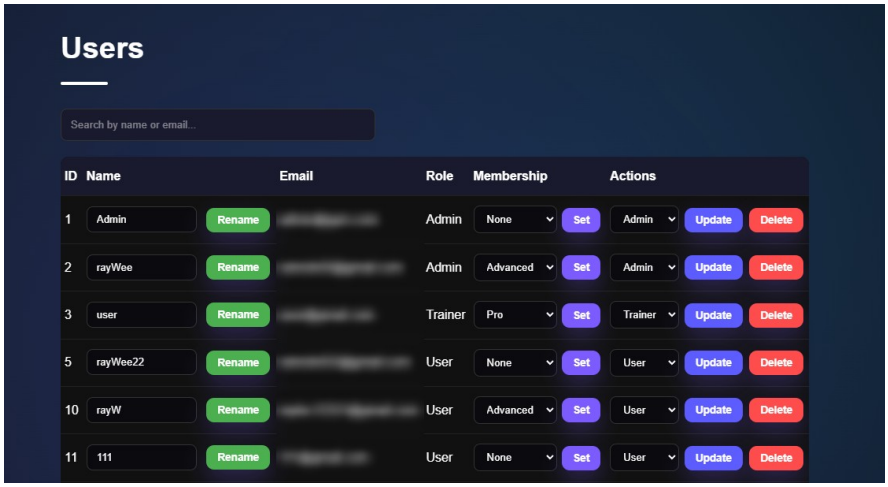
5.8. att. Dalībnieku saraksta eksports

5.3.3. Administratora darbības

Administrators piekļūst administrācijas panelim, kurā var apskatīt apmeklējuma statistiku, meklēt lietotājus, mainīt vārdu un lomu, piešķirt vai atcelt abonementu, dzēst lietotājus, kā arī pārvaldīt jebkuru nodarbību un eksportēt datus (skat. 5.9. un 5.10. att.).



5.9. att. Administrācijas panelis



5.10. att. Lietotāju un abonementu pārvaldība

6. TESTA PIEMĒRS

Programmatūras funkcionālā testēšana tika veikta, izpildot katrai funkcionālajai prasībai atbilstošus scenārijus. Katram scenārijam norādīts ievaddatu apraksts, sagaidāmais rezultāts, statuss un atsauce uz ekrānšāvumu, kas apliecina rezultātu. Scenāriji, kas pievienoti, balstoties uz sistēmas darbības loģiku, un vēl nav izpildīti, ir atzīmēti ar statusu “Jāpārbauda”. Testēšanas vide norādīta 6.1. tabulā.

6.1. tabula

Testēšanas vide

Parametrs	***Vērtība***
Operētājsistēma	Microsoft Windows 11 Pro, 24H2
Pārlūks	Google Chrome 147 (64 bitu)
PHP versija	8.2.12 (XAMPP 8.2.12, Apache 2.4.58)
Datubāze	MariaDB 10.4.32 (XAMPP), saderīga ar MySQL 8.0
Testēšanas datums	16.10.2026

Testu un prasību atbilstība norādīta 6.2. tabulā.

6.2. tabula

Prasību un testu atbilstība

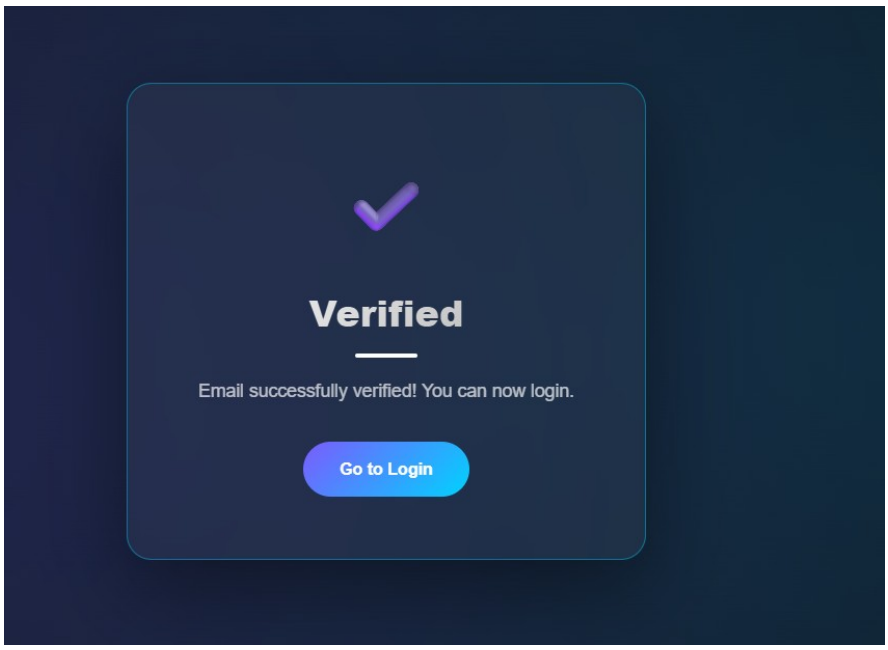
Prasība	***Testa tabula***	***Prasība***	***Testa tabula***
FP-1	6.3.	FP-6	6.6.
FP-2	6.3., 6.4.	FP-7, FP-8	6.7.
FP-3, FP-4, FP-5	6.5.	FP-9	6.8.

6.1. Reģistrācijas un autentifikācijas testēšana

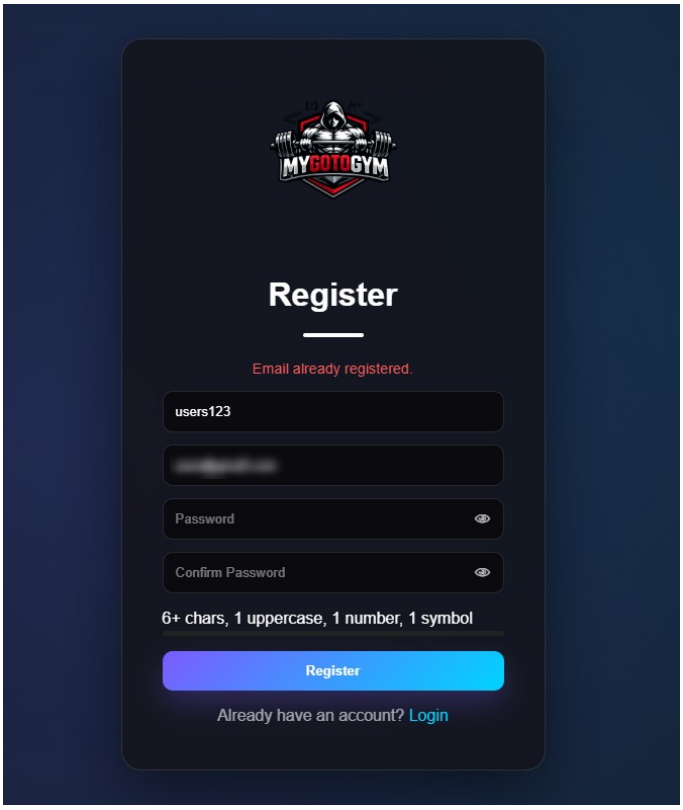
6.3. tabula

Lietotāju reģistrācijas, e-pasta verifikācijas un pieslēgšanās testēšana

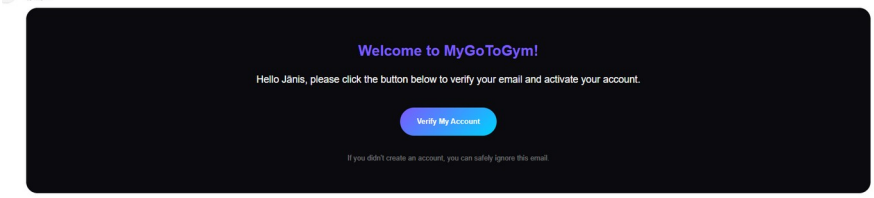
Lietotāja loma:	Viesis			
Prasības identifikators:	FP-1, FP-2			
Prasības apraksts				
Jāpārbauda, vai sistēma pareizi reģistrē jaunu lietotāju, nosūta e-pasta verifikācijas saiti un ierobežo pieslēgšanos bez verifikācijas un pēc atkārtotiem neveiksmīgiem mēģinājumiem.				
Nr.	***Ievaddati / situācijas apraksts***	***Sagaidāmais rezultāts***	***Statuss***	***Pierādījums***
1.	Lietotājs aizpilda reģistrācijas formu ar korektiem datiem.	Konts tiek izveidots, un uz e-pastu tiek nosūtīta verifikācijas saite.	Izpildīts	6.1. att.
2.	Lietotājs mēģina reģistrēties ar jau izmantotu e-pasta adresi.	Sistēma attēlo kļūdas paziņojumu par esošu e-pasta adresi.	Izpildīts	6.2. att.
3.	Lietotājs atver saņemto verifikācijas saiti.	Konts tiek veiksmīgi verificēts.	Izpildīts	6.3. att.
4.	Lietotājs mēģina pieslēgties bez e-pasta verifikācijas.	Sistēma liedz piekļuvi un pieprasa verificēt e-pastu.	Izpildīts	6.4. att.
5.	Lietotājs 5 reizes ievada nepareizu paroli.	Konts tiek bloķēts uz 15 minūtēm; tiek attēlots paziņojums.	Jāpārbauda	–



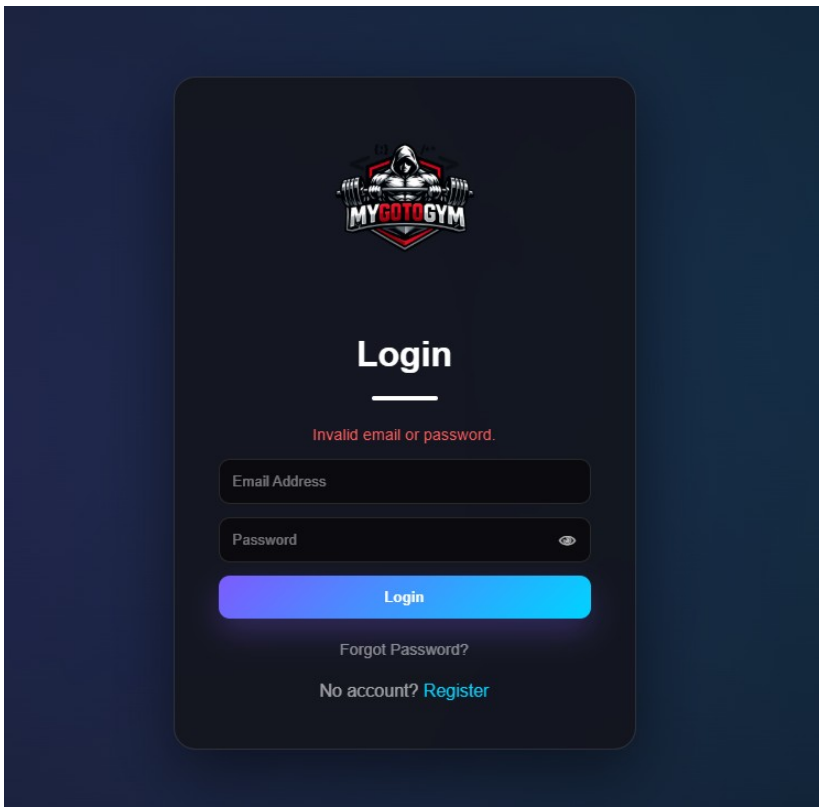
6.1. att. Veiksmīga reģistrācija



6.2. att. Reģistrācija ar jau esošu e-pastu



6.3. att. E-pasta verifikācija

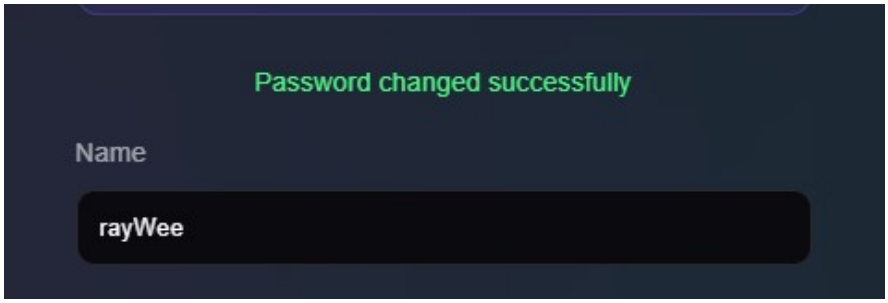


6.4. att. Pieslēgšanās bez verifikācijas

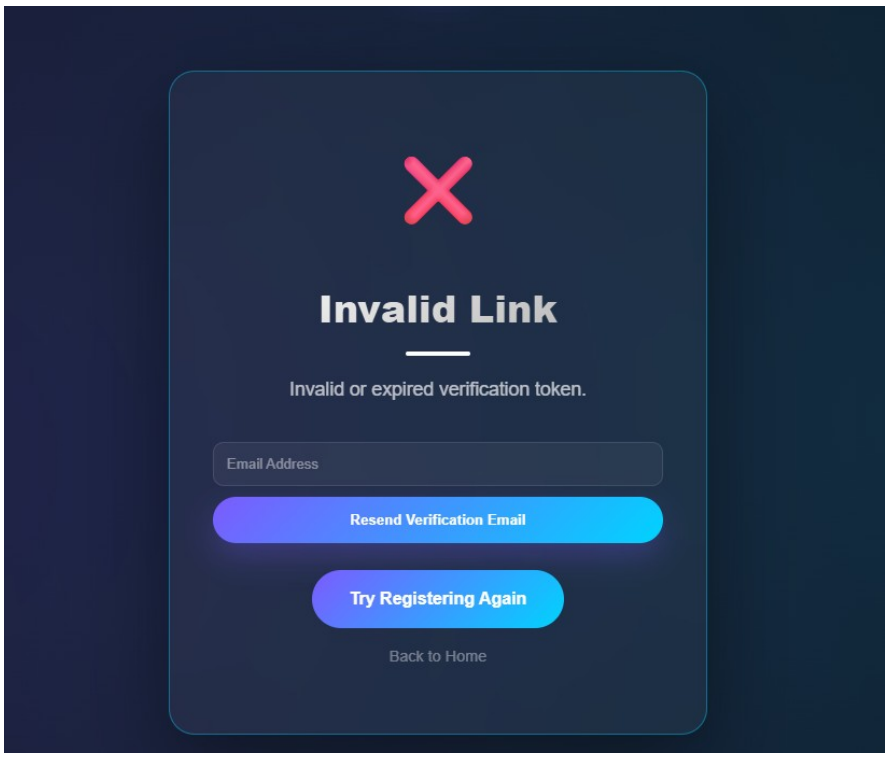
6.4. tabula

Paroles atjaunošanas testēšana

Lietotāja loma:	Reģistrēts lietotājs			
Prasības identifikators:	FP-2			
Prasības apraksts				
Jāpārbauda, vai sistēma nosūta paroles atjaunošanas saiti, ļauj iestatīt jaunu paroli un noraida nederīgas vai novecojušas saites.				
Nr.	***Ievaddati / situācijas apraksts***	***Sagaidāmais rezultāts***	***Statuss***	***Pierādījums***
1.	Lietotājs ievada reģistrētu e-pasta adresi paroles atjaunošanas formā.	Uz e-pastu tiek nosūtīta paroles atjaunošanas saite.	Izpildīts	6.5. att.
2.	Lietotājs atver derīgu paroles atjaunošanas saiti.	Tiek attēlota jaunas paroles ievades forma.	Izpildīts	–
3.	Lietotājs ievada jaunu derīgu paroli.	Parole tiek veiksmīgi nomainīta.	Izpildīts	–
4.	Lietotājs mēģina izmantot nederīgu vai novecojušu saiti.	Sistēma attēlo kļūdas paziņojumu.	Izpildīts	6.7. att.



6.5. att. Paroles atjaunošanas pieprasījums



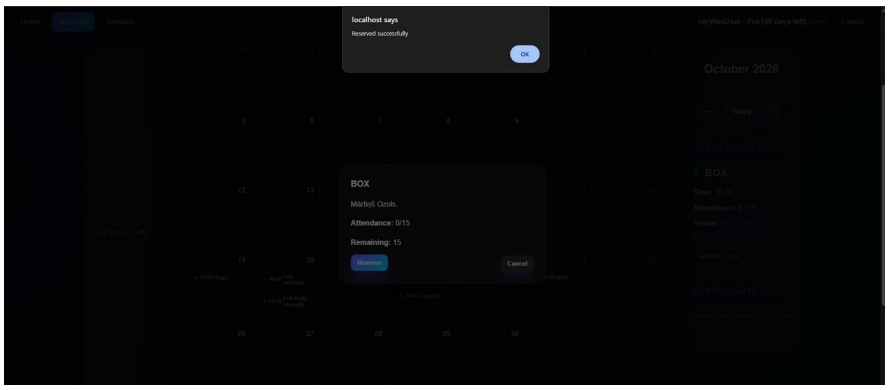
6.7. att. Nederīga atjaunošanas saite

6.2. Rezervēšanas testēšana

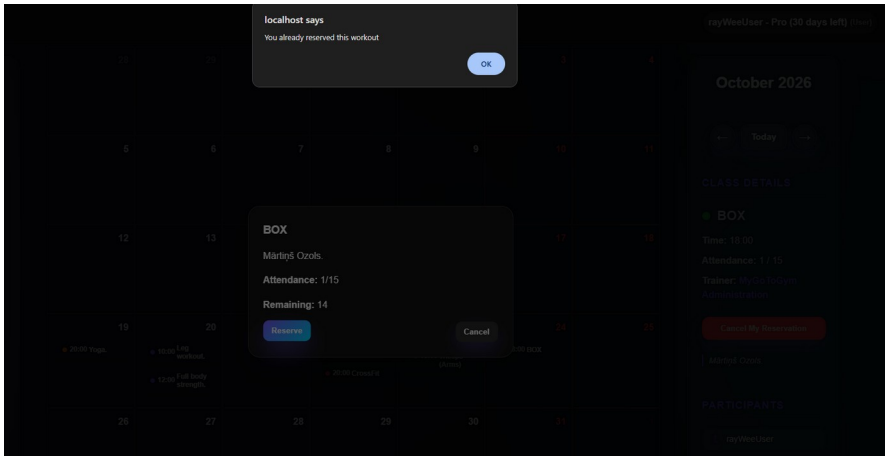
6.5. tabula

Nodarbību rezervēšanas un atcelšanas testēšana

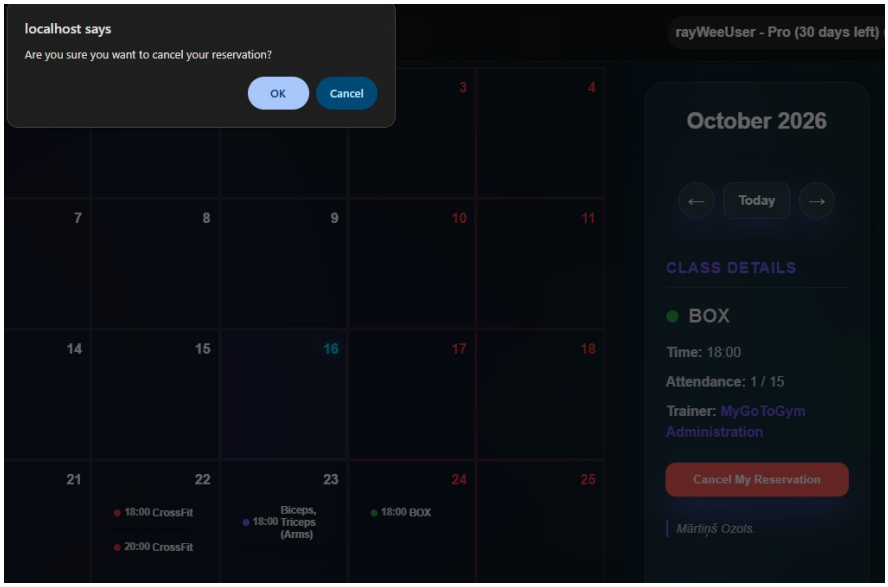
Lietotāja loma:	Reģistrēts lietotājs			
Prasības identifikators:	FP-3, FP-4, FP-5			
Prasības apraksts				
Jāpārbauda, vai klients var rezervēt un atcelt nodarbību, un vai sistēma ievēro vietu skaita, abonementa plāna un laika ierobežojumus.				
Nr.	***Ievaddati / situācijas apraksts***	***Sagaidāmais rezultāts***	***Statuss***	***Pierādījums***
1.	Lietotājs ar aktīvu Advanced/Pro abonementu rezervē pieejamu nodarbību.	Rezervācija tiek veiksmīgi izveidota; brīvo vietu skaits samazinās par vienu.	Izpildīts	6.8. att.
2.	Lietotājs mēģina rezervēt pilnībā aizņemtu nodarbību.	Sistēma attēlo paziņojumu, ka vietu nav.	Izpildīts	6.9. att.
3.	Lietotājs mēģina atkārtoti rezervēt to pašu nodarbību.	Sistēma neļauj izveidot dubultu rezervāciju.	Izpildīts	6.9. att.
4.	Lietotājs atceļ esošu rezervāciju pirms nodarbības sākuma.	Rezervācija tiek dzēsta, vieta atbrīvota.	Izpildīts	6.10. att.
5.	Lietotājs ar Basic abonementu mēģina rezervēt nodarbību.	Sistēma paziņo, ka plāns neietver rezervācijas.	Jāpārbauda	–
6.	Lietotājs mēģina atcelt rezervāciju nodarbībai, kas jau sākusies.	Sistēma neļauj atcelt rezervāciju.	Jāpārbauda	–



6.8. att. Veiksmīga rezervācija



6.9. att. Rezervācijas ierobežojumi



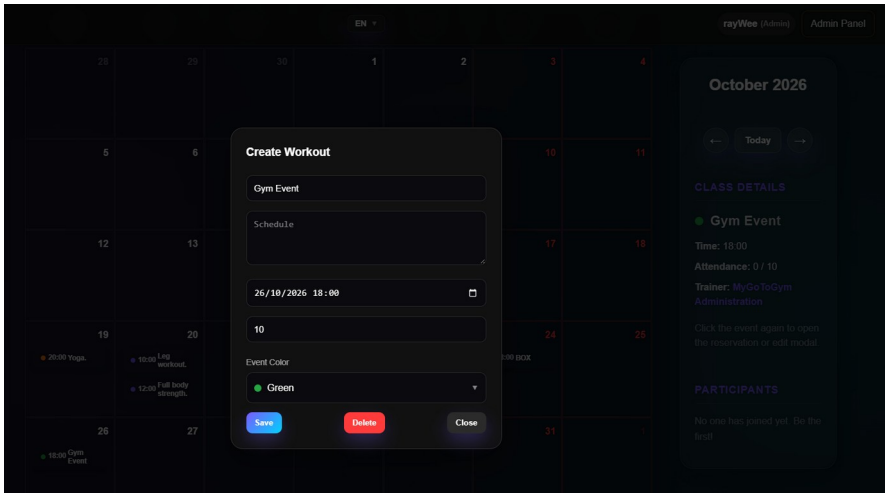
6.10. att. Rezervācijas atcelšana

6.3. Trenera un administratora funkciju testēšana

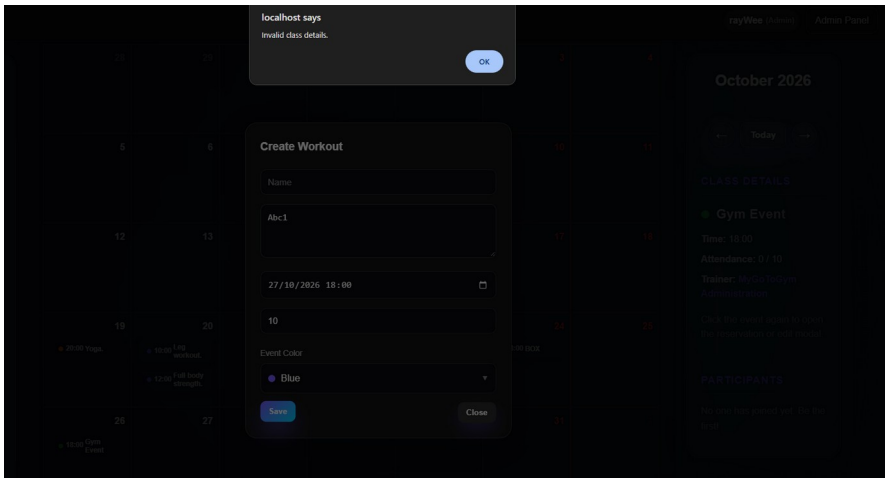
6.6. tabula

Trenera nodarbību pārvaldības testēšana

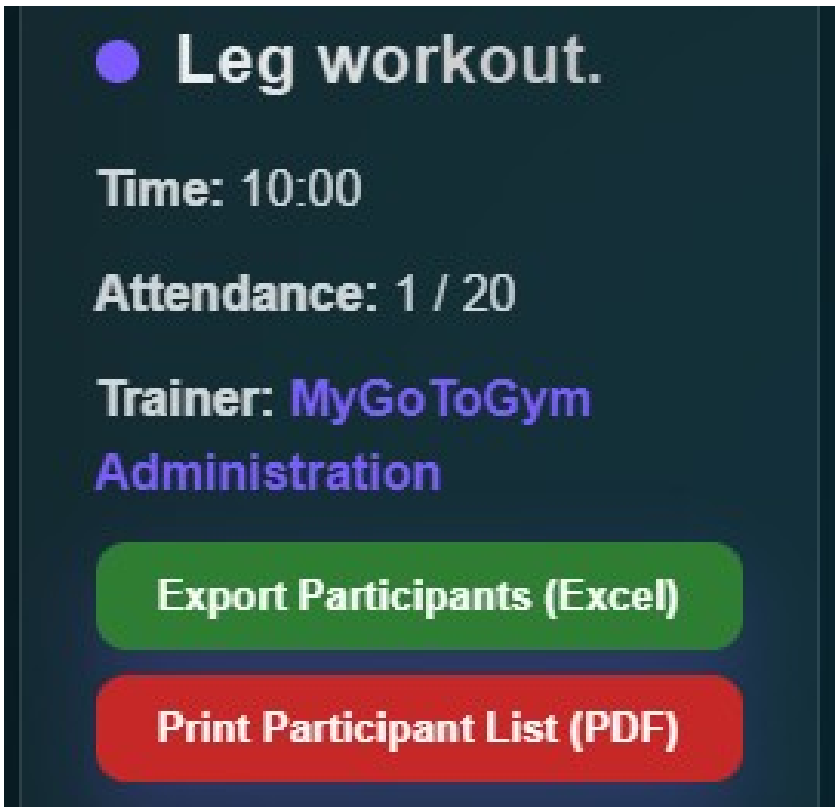
Lietotāja loma:	Treneris			
Prasības identifikators:	FP-6			
Prasības apraksts				
Jāpārbauda, vai treneris var izveidot, rediģēt un dzēst savas nodarbības un eksportēt dalībnieku sarakstu.				
Nr.	***Ievaddati / situācijas apraksts***	***Sagaidāmais rezultāts***	***Statuss***	***Pierādījums***
1.	Treneris izveido jaunu nodarbību, aizpildot visus obligātos laukus.	Nodarbība tiek veiksmīgi saglabāta kalendārā.	Izpildīts	6.11. att.
2.	Treneris mēģina izveidot nodarbību bez nosaukuma.	Sistēma attēlo kļūdas paziņojumu.	Izpildīts	6.12. att.
3.	Treneris rediģē savas nodarbības laiku.	Izmaiņas tiek veiksmīgi saglabātas.	Izpildīts	–
4.	Treneris dzēš savu nodarbību.	Nodarbība tiek izņemta no kalendāra.	Izpildīts	–
5.	Treneris eksportē rezervēto lietotāju sarakstu.	Tiek ģenerēts lejupielādējams fails ar dalībnieku sarakstu.	Izpildīts	6.13. att.



6.11. att. Nodarbības izveide



6.12. att. Nodarbības izveide bez nosaukuma

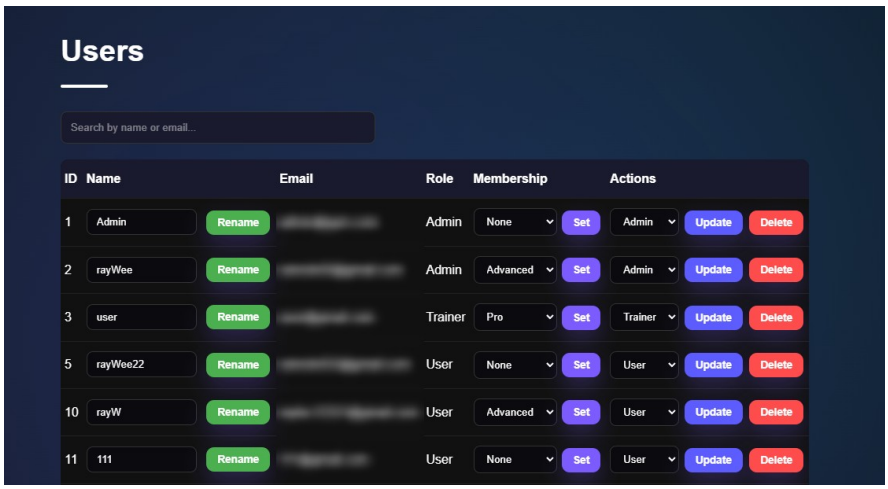


6.13. att. Dalībnieku saraksta eksports

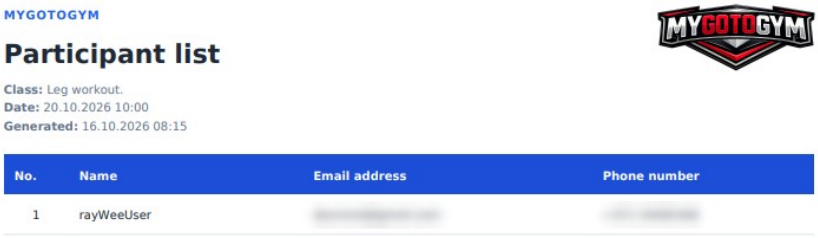
6.7. tabula

Administrācijas paneļa testēšana

Lietotāja loma:	Administrators			
Prasības identifikators:	FP-7, FP-8			
Prasības apraksts				
Jāpārbauda, vai administrators var pārvaldīt nodarbības un lietotājus, mainīt lomas un eksportēt datus.				
Nr.	***Ievaddati / situācijas apraksts***	***Sagaidāmais rezultāts***	***Statuss***	***Pierādījums***
1.	Administrators izveido jaunu nodarbību.	Nodarbība tiek veiksmīgi pievienota sistēmai.	Izpildīts	–
2.	Administrators dzēš jebkuru nodarbību.	Nodarbība tiek veiksmīgi dzēsta.	Izpildīts	–
3.	Administrators apskata lietotāju profilus.	Tiek attēlota lietotāju informācija.	Izpildīts	6.14. att.
4.	Administrators eksportē lietotāju sarakstu.	Tiek ģenerēts eksporta fails.	Izpildīts	6.15. att.
5.	Administrators maina lietotāja lomu.	Lietotāja loma tiek atjaunināta, mainās pieejamās funkcijas.	Jāpārbauda	–
6.	Administrators dzēš lietotāja kontu.	Lietotāja ieraksts tiek dzēsts, piekļuve sistēmai tiek liegta.	Jāpārbauda	–



6.14. att. Lietotāju saraksts



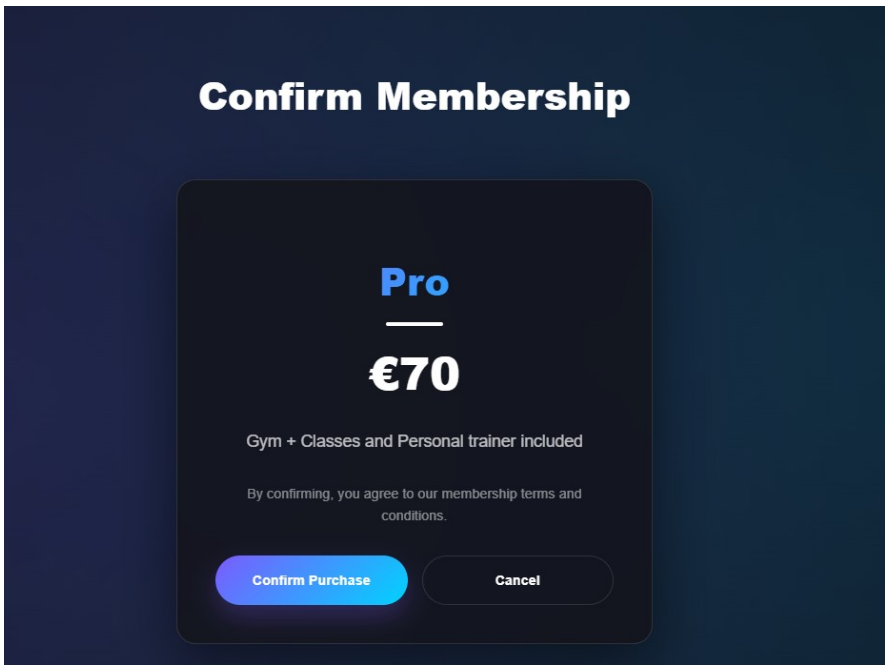
6.15. att. Lietotāju saraksta eksports

6.4. Abonementu testēšana

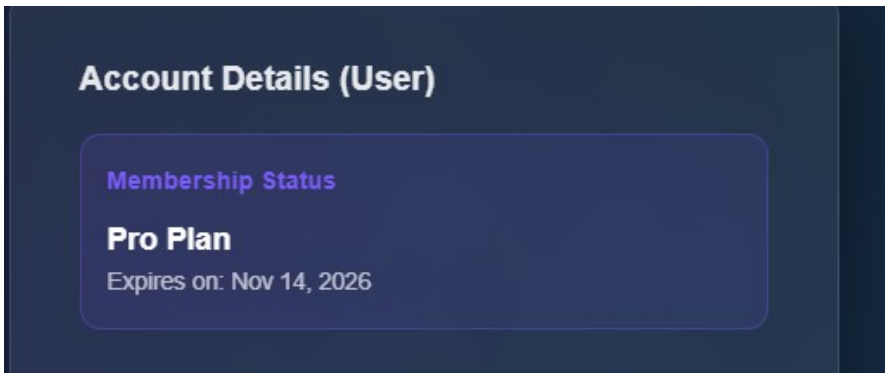
6.8. tabula

Abonementu funkcionalitātes testēšana

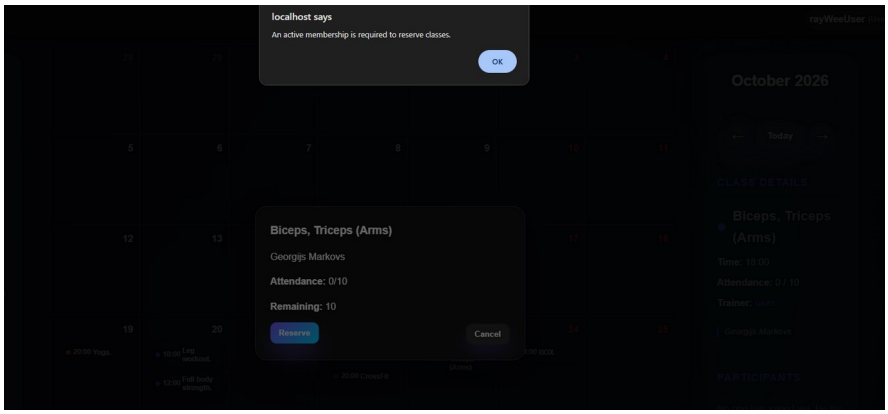
Lietotāja loma:	Reģistrēts lietotājs			
Prasības identifikators:	FP-9			
Prasības apraksts				
Jāpārbauda, vai sistēma aktivizē abonementu uz 31 dienu, attēlo tā derīguma termiņu un liedz rezervācijas pēc termiņa beigām.				
Nr.	***Ievaddati / situācijas apraksts***	***Sagaidāmais rezultāts***	***Statuss***	***Pierādījums***
1.	Lietotājs izvēlas abonementu un apstiprina iegādi.	Abonements tiek aktivizēts uz 31 dienu.	Izpildīts	6.16. att.
2.	Lietotājs atceļ iegādes procesu.	Abonements netiek aktivizēts.	Izpildīts	–
3.	Lietotājs apskata savu aktīvo abonementu.	Tiek attēlota abonementa informācija un derīguma termiņš.	Izpildīts	6.17. att.
4.	Lietotājam beidzas abonementa termiņš.	Sistēma liedz jaunas rezervācijas līdz abonementa atjaunošanai.	Izpildīts	6.18. att.



6.16. att. Abonementa iegāde



6.17. att. Aktīvā abonementa informācija



6.18. att. Rezervācija ar beigušos abonementu

6.5. Testēšanas rezultātu kopsavilkums

Visi izpildītie scenāriji (statuss “Izpildīts”) ir atbilduši sagaidāmajam rezultātam. Scenāriji ar statusu “Jāpārbauda” ir papildus sagatavoti, lai pārbaudītu sistēmas ierobežojumus (konta bloķēšana, abonementa plāna ierobežojums, atcelšana pēc nodarbības sākuma, lomu

maiņa un lietotāja dzēšana), un jāizpilda pirms darba iesniegšanas. NFP-1 un NFP-4 (veiktspēja un pieejamība) šajā testēšanā netika mērīti.

7. IEGULDĪJUMS DARBĀ

Kvalifikācijas darbs ir izstrādāts individuāli, tāpēc visi darba posmi ir autora paveikts darbs. Darba gaitā tika veikti šādi uzdevumi:

- esošo sporta klubu risinājumu izpēte un problēmas definēšana;
- funkcionālo un nefunkcionālo prasību izstrāde;
- datubāzes projektēšana un izveide (sešas tabulas ar relācijām);
- servera loģikas izstrāde PHP valodā: reģistrācija ar e-pasta verifikāciju, autentifikācija ar konta bloķēšanu, paroles atjaunošana, nodarbību un rezervāciju apstrāde, abonementu loģika, eksports;
- lietotāja saskarnes izstrāde ar HTML, CSS un JavaScript, tostarp nodarbību kalendārs un kalendāra atjaunošana ar SSE;
- drošības risinājumi: paroļu šifrēšana, CSRF aizsardzība, sagatavoti SQL izteikumi, sīkdatņu piekrišana;
- daudzvalodības atbalsta ieviešana (latviešu, angļu, krievu);
- testēšanas scenāriju sagatavošana un izpilde, dokumentācijas izstrāde.

SECINĀJUMI

Kvalifikācijas darba rezultātā tika izstrādāta sporta zāles nodarbību pārvaldības un rezervēšanas tīmekļa lietojumprogramma “Gym_Tracker2”, kuras mērķis ir atvieglot sporta klubu ikdienas darbu, nodarbību organizēšanu un klientu rezervāciju pārvaldību. Sistēma nodrošina centralizētu vidi nodarbību plānošanai, lietotāju pārvaldībai un abonementu administrēšanai.

Sasniegtie rezultāti. Izvirzītie darba uzdevumi ir izpildīti šādā apjomā:

- veikta esošo risinājumu izpēte un definētas 9 funkcionālās un 6 nefunkcionālās prasības;
- izvēlētas un pamatotas tehnoloģijas (PHP, MySQL/MariaDB, HTML, CSS, JavaScript, PHPMailer, Git);
- izstrādāta sistēmas arhitektūra no piecām apakšsistēmām, ER modelis, algoritmu shēmas un datu vārdnīca sešām datubāzes tabulām;
- realizētas trīs lietotāju lomas – klients, treneris un administrators;
- klienti var reģistrēties, verificēt e-pastu, atjaunot paroli, iegādāties abonementu, apskatīt nodarbību kalendāru un rezervēt vai atcelt nodarbības;
- treneri var izveidot, rediģēt un dzēst savas nodarbības un eksportēt dalībnieku sarakstus, administratori – pārvaldīt visus lietotājus, nodarbības un abonementus, apskatīt statistiku un eksportēt datus;
- sagatavots lietotāja ceļvedis un funkcionālās testēšanas scenāriji pa katru prasību grupu.

Testēšanas rezultāti. Izpildītie testa scenāriji apstiprināja reģistrācijas, paroles atjaunošanas, rezervēšanas, trenera funkciju, administrācijas paneļa un abonementu darbību atbilstoši specifikācijai. Papildu scenāriji ierobežojumu pārbaudei (konta bloķēšana, plāna ierobežojums, atcelšana pēc nodarbības sākuma, lomu maiņa, lietotāja dzēšana) jāizpilda un jādokumentē ar ekrānšāviem.

Ierobežojumi. Darba gaitā tika konstatēti šādi ierobežojumi: (1) abonementa iegāde šobrīd aktivizē abonementu bez pilnvērtīgas tiešsaistes maksājuma apstrādes – tabula “payments” un Stripe konfigurācija ir sagatavota, bet maksājumu integrācija nav pabeigta; (2) nav ieviesta gaidītāju rinda un ierobežojums atcelt rezervāciju noteiktu laiku pirms nodarbības – rezervāciju var atcelt līdz nodarbības sākumam; (3) automātiska atslēgšana pēc neaktivitātes nav realizēta – sesija beidzas, aizverot pārlūku; (4) veiktspējas (NFP-1) un pieejamības (NFP-4) rādītāji nav izmērīti; (5) sistēmas darbība ar HTTPS jānodrošina servera konfigurācijā.

Iespējamie uzlabojumi. Izstrādāto platformu nākotnē iespējams papildināt ar tiešsaistes maksājumu sistēmas integrāciju, gaidītāju rindu un rezervācijas atcelšanas laika ierobežojumu, automātiskiem atgādinājumiem par rezervētajām nodarbībām e-pastā vai īsziņās, detalizētāku statistikas un analītikas sadaļu, mobilo lietotni Android un iOS platformām, kā arī vairāku sporta klubu pārvaldību vienotā platformā.

Kopumā izstrādātā sistēma “Gym_Tracker2” ir funkcionāls un praktiski pielietojams risinājums sporta zāļu nodarbību pārvaldībai, kas palīdz digitalizēt rezervāciju procesus, samazināt administratīvo slodzi un uzlabot klientu apkalpošanas kvalitāti.

SAĪSINĀJUMU SARAKSTS

8.1. tabula

Saīsinājumi

Saīsinājums	*Atšifrējums*
API	Application Programming Interface – lietojumprogrammu saskarne
CRUD	Create, Read, Update, Delete – izveidot, nolasīt, mainīt, dzēst
CSRF	Cross-Site Request Forgery – starpvietņu pieprasījumu viltošana
CSS	Cascading Style Sheets – kaskadējošās stilu lapas
ER	Entity–Relationship – entītiju un attiecību modelis
FK / PK	Foreign Key / Primary Key – ārējā / primārā atslēga
FP / NFP	Funkcionālā / nefunkcionālā prasība
HTML	HyperText Markup Language – hiperteksta iezīmēšanas valoda
HTTPS	HyperText Transfer Protocol Secure – drošais hiperteksta pārraides protokols
JSON	JavaScript Object Notation – datu apmaiņas formāts
PHP	PHP: Hypertext Preprocessor – servera puses programmēšanas valoda
SMTP	Simple Mail Transfer Protocol – e-pasta pārsūtīšanas protokols
SQL	Structured Query Language – strukturēto vaicājumu valoda
SSE	Server-Sent Events – servera sūtītie notikumi

LITERATŪRAS SARAKSTS

1. Fizisko personu datu apstrādes likums: Latvijas Republikas likums. – Latvijas Vēstnesis / internets. – https://likumi.lv/ta/id/300099-fizisko-personu-datu-apstrades-likums – 16.10.2026.
2. Eiropas Parlamenta un Padomes Regula (ES) 2016/679 (2016. gada 27. aprīlis) par fizisku personu aizsardzību attiecībā uz personas datu apstrādi (Vispārīgā datu aizsardzības regula) / internets. – https://eur-lex.europa.eu/eli/reg/2016/679/oj – 16.10.2026.
3. Composer: Documentation / internets. – https://getcomposer.org/doc/ – 16.10.2026.
4. Git: Documentation / internets. – https://git-scm.com/doc – 16.10.2026.
5. Lemon Gym: oficiālā mājaslapa / internets. – https://www.lemongym.lv – 16.10.2026.
6. Microsoft: Visual Studio Code Documentation / internets. – https://code.visualstudio.com/docs – 16.10.2026.
7. Mozilla: MDN Web Docs – HTML, CSS, JavaScript / internets. – https://developer.mozilla.org/ – 16.10.2026.
8. Mozilla: Using server-sent events / MDN Web Docs / internets. – https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events/Using_server-sent_events – 16.10.2026.
9. MyFitness: oficiālā mājaslapa / internets. – https://www.myfitness.lv – 16.10.2026.
10. Oracle: MySQL 8.0 Reference Manual / internets. – https://dev.mysql.com/doc/refman/8.0/en/ – 16.10.2026.
11. OWASP: Cross-Site Request Forgery Prevention Cheat Sheet / internets. – https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html – 16.10.2026.
12. OWASP: Password Storage Cheat Sheet / internets. – https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html – 16.10.2026.
13. The PHP Group: PHP Manual / internets. – https://www.php.net/manual/en/ – 16.10.2026.
14. PHPMailer: A full-featured email creation and transfer class for PHP / internets. – https://github.com/PHPMailer/PHPMailer – 16.10.2026.
15. Stripe: Stripe Documentation / internets. – https://docs.stripe.com/ – 16.10.2026.

PIELIKUMI

1. pielikums. Failu struktūra

Projekta failu struktūra

```
gym_tracker2/
├── .env, .env.example, .gitignore, .htaccess
├── 403.php, 404.php, index.php, router.php, nav.php
├── composer.json, composer.lock
├── homepage.php, contacts.php, impressum.php, privacy.php
├── login.php, logout.php, register.php, profile.php
├── forgot_password.php, reset_password.php
├── verify_email.php, membership_offer.php
├── admin/
│   ├── classes_sse.php, dashboard.php, index.php
│   ├── delete_class.php, delete_user.php
│   └── rename_user.php, search_users.php, update_role.php
├── api/
│   ├── Mailer.php, config.php, db.php, csrf.php, session.php, lang.php
│   ├── admin_assign_membership.php, buy_membership.php, payment_success.php
│   ├── reserve_class.php, cancel_reservation.php
│   ├── save_class.php, delete_class.php, get_classes.php, get_workouts.php
│   ├── get_participants.php, track_analytics.php
│   ├── resend_verification.php, verify_email.php, update_profile.php
│   └── cookie_popup.php, footer.php, nav.php, index.php
├── assets/ (css/, images/, js/calendar.js)
├── export/
│   ├── export_class_participants.php, export_class_participants_pdf.php
│   └── export_excel.php, export_pdf.php, export_helpers.php, index.php
├── storage/sessions/
├── vendor/
└── views/
```

2. pielikums. Programmas pirmkods

Programmas pirmkods

Šajā pielikumā tiek iekļauts paša izstrādātais sistēmas pirmkods ar komentāriem (faili api/, admin/, export/, assets/js/ un galvenās lapas). Pirmkods ir pievienots arī kā atsevišķi programmatūras faili.

Fails **`403.php`** – kļūdas lapa “403 – piekļuve liegta”:

```
<?php

// If accessed directly with .php extension, redirect to clean URL
if (basename($_SERVER['PHP_SELF']) === '403.php') {
    header("Location: " . $basePath . "/403", true, 301);
    exit;
}

include 'api/db.php';
include 'views/403.html';
```

Fails **`404.php`** – kļūdas lapa “404 – lapa nav atrasta”:

```
<?php

// If accessed directly with .php extension, redirect to clean URL
if (basename($_SERVER['PHP_SELF']) === '404.php') {
    header("Location: " . $basePath . "/404", true, 301);
    exit;
}

include 'api/db.php';
include 'views/404.html';
```

Fails **`contacts.php`** – kontaktu lapa:

```
<?php
include 'api/db.php';

$name    = 'Demo Gym';
$email   = 'info@example.com';
$phone   = '+37120000000';
$address = 'Brīvības iela 1, Rīga';
$mapUrl = 'https://www.google.com/maps/search/?api=1&query=' . urlencode($address);
$mapEmbedUrl = 'https://maps.google.com/maps?output=embed&q=' . urlencode($address);

include 'views/contacts.html';
```

Fails **`forgot_password.php`** – paroles atjaunošanas pieprasījums (žetona izveide un e-pasta nosūtīšana):

```
<?php
include 'api/db.php';
require_once __DIR__ . '/api/csrf.php';

if (isset($_SESSION['user_id'])) {
    http_response_code(403);
    require __DIR__ . '/403.php';
    exit;
}
```

```
require_once 'api/Mailer.php';

require 'vendor/autoload.php';

$message = '';
$error = '';

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    if (!check_csrf($_POST['csrf_token'])) {
        $error = "Invalid CSRF token";
    } else {
        $email = strtolower(trim($_POST['email']));


        $stmt = $conn->prepare("SELECT id, name FROM users WHERE email = ? AND email_verified = 1");
        if (!$stmt) {
            die("Database error (SELECT): " . $conn->error);
        }

        $stmt->bind_param("s", $email);
        $stmt->execute();
        $res = $stmt->get_result();
        $user = $res->fetch_assoc();

        if ($user) {

            $token = bin2hex(random_bytes(32));
            $expiry = date("Y-m-d H:i:s", strtotime("+1 hour"));

            $stmt = $conn->prepare("REPLACE INTO password_resets (email, token, expiry) VALUES
(?, ?, ?)");
            if (!$stmt) {
                die("Database error (REPLACE): " . $conn->error);
            }

            $stmt->bind_param("sss", $email, $token, $expiry);
            $stmt->execute();

            $protocol = isset($_SERVER['HTTPS']) && $_SERVER['HTTPS'] === 'on' ? "https" : "http";
            $resetLink = "$protocol://" . $_SERVER['HTTP_HOST'] . $basePath . "/reset_password?
token=" .
                urlencode($token);

            \App\Mailer::sendResetEmail($email, $user['name'], $resetLink, $current_lang);
        }

        // Always show success message for security
        $message = $t['reset_link_sent'];
    }
}

$csrf_token = generate_csrf();

include 'views/forgot_password.html';
```

Fails **homepage.php** – sākumlapa:

```
<?php
require_once 'api/db.php';

$is_logged_in = isset($_SESSION['user_id']);
$buy_url = $is_logged_in ? $basePath . '/membership_offer?plan=' : $basePath . '/login';
$register_or_home = $is_logged_in ? $basePath . '/index' : $basePath . '/login';
```

```
include 'views/homepage.html';
```

Fails **`impressum.php`** – impresuma lapa:

```
<?php
include 'api/db.php';

if (basename($_SERVER['PHP_SELF']) === 'impressum.php') {
    header("Location: " . $basePath . "/impressum", true, 301);
    exit;
}

include 'views/impressum.html';
```

Fails **`index.php`** – ieejas punkts, maršrutēšana un sākuma lapas ielāde:

```
<?php

$requestPath = parse_url($_SERVER['REQUEST_URI'] ?? '/', PHP_URL_PATH) ?: '/';
$basePath = '';
$appRoot = str_replace('\\', '/', realpath(__DIR__) ?: __DIR__);
$documentRoot = str_replace('\\', '/', realpath($_SERVER['DOCUMENT_ROOT'] ?? '') ?: '');
if ($documentRoot !== '' && strpos($appRoot, $documentRoot) === 0) {
    $basePath = rtrim(substr($appRoot, strlen($documentRoot)), '/');
}

if (PHP_SAPI === 'cli-server') {
    $staticFile = __DIR__ . rawurldecode($requestPath);
    if (is_file($staticFile)) {
        return false;
    }
}

if ($basePath !== '' && strpos($requestPath, $basePath) === 0) {
    $requestPath = substr($requestPath, strlen($basePath)) ?: '/';
}

$route = trim(rawurldecode($requestPath), '/');

if (preg_match('#^(en|lv|ru)(?:/(.*))?$#', $route, $matches)) {
    $_GET['lang'] = $matches[1];
    $route = trim($matches[2] ?? '', '/');
}

if ($route === '' || strtolower($route) === 'index.php') {
    require __DIR__ . '/homepage.php';
    exit;
}

if (strpos($route, '..') !== false || strpos($route, "\0") !== false) {
    http_response_code(403);
    require __DIR__ . '/403.php';
    exit;
}

if (preg_match('#(^|/)(db|csrf|router|config)(\.php)?$#i', $route)
    || preg_match('#\.(sql|env|log|bak|ini|db)$#i', $route)
    || preg_match('#(^|/)vendor/#i', $route)
) {
    http_response_code(403);
    require __DIR__ . '/403.php';
    exit;
}
```

```
if (preg_match('/\.php$/i', $route)) {
    $cleanRoute = preg_replace('/\.php$/i', '', $route);
    $query = $_SERVER['QUERY_STRING'] ?? '';
    header('Location: ' . $basePath . '/' . $cleanRoute . ($query ? '?' . $query : ''), true, 301);
    exit;
}

if ($route !== 'index') {
    $phpFile = __DIR__ . '/' . $route . '.php';
    if (is_file($phpFile)) {
        $adminOnlyPrefixes = ['admin/', 'api/admin_', 'export/'];
        foreach ($adminOnlyPrefixes as $prefix) {
            if (strpos($route, $prefix) === 0) {
                if (session_status() === PHP_SESSION_NONE) {
                    require_once __DIR__ . '/api/session.php';
                    start_app_session();
                }

                if (($_SESSION['role'] ?? '') !== 'admin') {
                    http_response_code(403);
                    require __DIR__ . '/403.php';
                    exit;
                }

                break;
            }
        }

        require $phpFile;
        exit;
    }

    http_response_code(404);
    require __DIR__ . '/404.php';
    exit;
}

include 'api/db.php';
require_once __DIR__ . '/api/csrf.php';
```

Fails **`login.php`** – lietotāja pieslēgšanās ar konta bloķēšanu pēc 5 neveiksmīgiem mēģinājumiem:

```
<?php
include 'api/db.php';

if (basename($_SERVER['PHP_SELF']) === 'login.php') {
    header("Location: " . $basePath . "/login", true, 301);
    exit;
}

if (isset($_SESSION['user_id'])) {
    header("Location: " . $basePath . "/homepage");
    exit;
}

require_once __DIR__ . '/api/csrf.php';

$error = '';

if($_SERVER['REQUEST_METHOD']==='POST'){
    if(!check_csrf($_POST['csrf_token'])){
        $error = "Invalid CSRF token";
    } else {
```

```
        $email = strtolower(trim($_POST['email']));
        $password = $_POST['password'];

        $stmt = $conn->prepare("SELECT id,password,role,failed_logins,lock_until,email_verified FROM
users WHERE
            email=?");
        $stmt->bind_param("s",$email);
        $stmt->execute();
        $u = $stmt->get_result()->fetch_assoc();

        if($u){
            if($u['lock_until'] && strtotime($u['lock_until'])>time()){
                $error = "Account locked. Try later.";
            } elseif(!password_verify($password,$u['password'])){
                $failed = $u['failed_logins']+1;
                $lock = $failed>=5 ? date('Y-m-d H:i:s',strtotime('+15 minutes')) : NULL;
                $stmt2 = $conn->prepare("UPDATE users SET failed_logins=?, lock_until=? WHERE
id=?"); // This
                    uses $conn, consistent with db.php
                $stmt2->bind_param("isi",$failed,$lock,$u['id']);
                $stmt2->execute();
                $error = "Invalid email or password.";
            } elseif(!$u['email_verified']){
                $error = "Please verify your email first.";
                $showResend = true;
                $resendEmail = $email;
            } else {
                $stmt2 = $conn->prepare("UPDATE users SET failed_logins=0, lock_until=NULL WHERE
id=?");
                $stmt2->bind_param("i",$u['id']);
                $stmt2->execute();

                session_regenerate_id(true);
                $_SESSION['user_id'] = $u['id'];
                $_SESSION['role'] = $u['role'];


                if (isset($_COOKIE['cookie_consent']) && $_COOKIE['cookie_consent'] === 'accepted')
{
                    $track_stmt = $conn->prepare("INSERT INTO site_analytics (user_id, action)
VALUES (?,
                        'login')");
                    $track_stmt->bind_param("i", $u['id']);
                    $track_stmt->execute();
                }

                header("Location:" . $basePath . "/homepage");
                exit;
            }
        } else {
            $error = "Invalid email or password.";
        }
    }
}
$csrf_token = generate_csrf();

include 'views/login.html';
```

Fails **`logout.php`** – lietotāja izslēgšanās un sesijas beigšana:

```
<?php
require_once __DIR__ . '/api/session.php';
start_app_session();
```

```
$_SESSION = [];
setcookie(session_name(), '', time() - 42000, '/');
session_destroy();

if (basename($_SERVER['PHP_SELF']) === 'logout.php') {
    header("Location: " . $basePath . "/logout", true, 301);
    exit;
}

header("Location: " . $basePath . "/login");
exit;
```

Fails **`membership_offer.php`** – abonementu plānu piedāvājums un izvēle:

```
<?php
include 'api/db.php';
require_once __DIR__ . '/api/csrf.php';

if (basename($_SERVER['PHP_SELF']) === 'membership_offer.php') {
    header("Location: " . $basePath . "/membership_offer", true, 301);
    exit;
}

if (!isset($_SESSION['user_id'])) {
    header("Location: " . $basePath . "/login");
    exit;
}

$plan = $_GET['plan'] ?? '';
$valid_plans = ['basic', 'advanced', 'pro'];

if (!in_array($plan, $valid_plans)) {
    // Redirect to homepage or show an error if plan is invalid
    header("Location: " . $basePath . "/homepage");
    exit;
}

$plan_details = [
    'basic' => [
        'title' => $t['basic'],
        'price' => '€20',
        'description' => $t['gym_only']
    ],
    'advanced' => [
        'title' => $t['advanced'],
        'price' => '€50',
        'description' => $t['gym_classes']
    ],
    'pro' => [
        'title' => $t['pro'],
        'price' => '€70',
        'description' => $t['trainer_included']
    ]
];

$selected_plan = $plan_details[$plan];
$csrf_token = generate_csrf();


include 'views/membership_offer.html';
```

Fails **`nav.php`** – navigācijas josla:

```
<?php
require_once __DIR__ . '/api/session.php';
start_app_session();

$username = '';
$role = '';

if (isset($_SESSION['user_id']) && isset($conn)) {
    $stmt = $conn->prepare("SELECT name, role FROM users WHERE id=?");
    $stmt->bind_param("i", $_SESSION['user_id']);
    $stmt->execute();
    $user = $stmt->get_result()->fetch_assoc();

    if ($user) {
        $username = $user['name'];
        $role = $user['role'];
    }
}


$currentPage = basename($_SERVER['PHP_SELF']);
$queryString = $_SERVER['QUERY_STRING'] ?? '';

include 'views/nav.html';
?>
```

Fails **`privacy.php`** – privātuma politikas lapa:

```
<?php
include 'api/db.php';

if (basename($_SERVER['PHP_SELF']) === 'privacy.php') {
    header("Location: " . $basePath . "/privacy", true, 301);
    exit;
}

include 'views/privacy.html';
```

Fails **`profile.php`** – lietotāja profils un abonementa informācija:

```
<?php
include 'api/db.php';

if (basename($_SERVER['PHP_SELF']) === 'profile.php') {
    header("Location: " . $basePath . "/profile", true, 301);
    exit;
}

if (!isset($_SESSION['user_id'])) {
    header("Location: " . $basePath . "/login");
    exit;
}

require_once __DIR__ . '/api/csrf.php';

$stmt = $conn->prepare("SELECT name, email, pending_email, phone, role, membership_type,
membership_expiry FROM
    users WHERE id = ?");
$stmt->bind_param("i", $_SESSION['user_id']);
$stmt->execute();
$user = $stmt->get_result()->fetch_assoc();

if (!$user) {
```

```
    header("Location: logout.php");
    exit;
}

$username = $user['name'] ?? '';

$stmt_p = $conn->prepare("SELECT plan_type, amount, status, created_at FROM payments WHERE user_id =
? ORDER BY
    created_at DESC");
$stmt_p->bind_param("i", $_SESSION['user_id']);
$stmt_p->execute();
$payments = $stmt_p->get_result();

$csrf_token = generate_csrf();

include 'views/profile.html';
```

Fails **`register.php`** – jauna lietotāja reģistrācija ar e-pasta verifikāciju:

```
<?php
use PHPMailer\PHPMailer\PHPMailer;
use PHPMailer\PHPMailer\Exception;

require 'vendor/autoload.php';

include 'api/db.php';

if (basename($_SERVER['PHP_SELF']) === 'register.php') {
    header("Location: " . $basePath . "/register", true, 301);
    exit;
}
require_once 'api/Mailer.php';

if (isset($_SESSION['user_id'])) {
    header("Location: " . $basePath . "/index");
    exit;
}

require_once __DIR__ . '/api/csrf.php';

$error = '';
$name = '';
$email = '';
$developmentVerificationLink = '';

if($_SERVER['REQUEST_METHOD']==='POST'){
    if(!check_csrf($_POST['csrf_token'])){
        $error = "Invalid CSRF token.";
    } else {
        $name = trim($_POST['name']);
        $email = strtolower(trim($_POST['email']));
        $password = $_POST['password'];
        $confirm_password = $_POST['confirm_password'] ?? '';

        if(!$name || !$email || !$password || !$confirm_password){
            $error = "All fields are required.";
        } elseif(!filter_var($email,FILTER_VALIDATE_EMAIL)){
            $error = "Invalid email address.";
        } elseif($password !== $confirm_password){
            $error = "Passwords do not match.";
        } elseif(!preg_match('/^(?=.*[A-Z])(?=.*\d)(?=.*[\W_]).{6,}$/',$password)){
            $error = "Password must be 6+ chars, 1 uppercase, 1 number, 1 symbol.";
        } else {
            $stmt = $conn->prepare("SELECT id FROM users WHERE email=?");
```

```
            $stmt->bind_param("s",$email);
            $stmt->execute();

            if($stmt->get_result()->num_rows>0){
                $error = "Email already registered.";
            } else {
                $hashed = password_hash($password,PASSWORD_DEFAULT);
                $token = bin2hex(random_bytes(32));

                $stmt = $conn->prepare("INSERT INTO users
(name,email,password,role,email_verified,verification_token) VALUES (?,?,?,'user',0,?)"); // Set
email_verified
    to 0
                $stmt->bind_param("ssss",$name,$email,$hashed,$token);

                if($stmt->execute()){
                    // Dynamically generate the verification link using $basePath for clean URLs
                    $protocol = isset($_SERVER['HTTPS']) && $_SERVER['HTTPS'] === 'on' ? "https" :
"http";
                    $host = $_SERVER['HTTP_HOST'];
                    $verify_link = "$protocol://$host" . $basePath .
"/verfy_email?token=$token&lang=$current_lang";

                    if (\App\Mailer::sendVerificationEmail($email, $name, $verify_link,
$current_lang)) {
                        header("Location: " . $basePath . "/login?registered=1");
                        exit;
                    } else {
                        if (LOCAL_VERIFICATION_FALLBACK) {
                            $error = "Registration successful. Email delivery is unavailable
locally, so use the
                                verification link below.";
                            $developmentVerificationLink = $verify_link;
                        } else {
                            $error = "Registration successful, but verification email failed to
send.";
                        }
                        $showResend = true;
                        $resendEmail = $email;
                    }
                } else {
                    $error = "Registration failed, try again.";
                }
            }
        }
    }
}

$csrf_token = generate_csrf();

include 'views/register.html';
```

Fails **`reset_password.php`** – jaunas paroles iestatīšana pēc atjaunošanas saites:

```
<?php
include 'api/db.php';
require_once __DIR__ . '/api/csrf.php';

if (isset($_SESSION['user_id'])) {
    http_response_code(403);
    require __DIR__ . '/403.php';
    exit;
}
```

```php
$token = $_GET['token'] ?? '';
$error = '';
$success = false;

if (empty($token)) {
    http_response_code(404);
    require __DIR__ . '/404.php';
    exit;
}

$now = date("Y-m-d H:i:s");
$stmt = $conn->prepare("SELECT email FROM password_resets WHERE token = ? AND expiry > ?");
if (!$stmt) {
    die("Database error: " . $conn->error);
}

$stmt->bind_param("ss", $token, $now);
$stmt->execute();
$res = $stmt->get_result();
$resetRequest = $res->fetch_assoc();

if (!$resetRequest) {
    http_response_code(404);
    require __DIR__ . '/404.php';
    exit;
}

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    if (!check_csrf($_POST['csrf_token'])) {
        $error = "Invalid CSRF token";
    } else {
        $newPass = $_POST['password'];
        $confirmPass = $_POST['confirm_password'];
        if ($newPass !== $confirmPass) {
            $error = $t['password_mismatch'];
        } elseif (strlen($newPass) < 8) {
            $error = $t['password_min_length'];
        } else {
            $hashed = password_hash($newPass, PASSWORD_DEFAULT);
            $email = $resetRequest['email'];


            $conn->begin_transaction();
            try {
                $stmt = $conn->prepare("UPDATE users SET password = ? WHERE email = ?");
                $stmt->bind_param("ss", $hashed, $email);
                $stmt->execute();

                // Delete token
                $stmt = $conn->prepare("DELETE FROM password_resets WHERE token = ?");
                $stmt->bind_param("s", $token);
                $stmt->execute();

                $conn->commit();
                $success = true;
            } catch (Exception $e) {
                $conn->rollback();
                $error = "Database error: " . $e->getMessage();
            }
        }
    }
}
```

```
$csrf_token = generate_csrf();
include 'views/reset_password.html';
```

Fails **`router.php`** – maršrutētājs PHP iebūvētajam serverim:

```
<?php

$requestPath = parse_url($_SERVER['REQUEST_URI'] ?? '/', PHP_URL_PATH) ?: '/';
$appRoot = str_replace('\\', '/', realpath(__DIR__) ?: __DIR__);
$documentRoot = str_replace('\\', '/', realpath($_SERVER['DOCUMENT_ROOT'] ?? '') ?: '');
$basePath = ($documentRoot !== '' && strpos($appRoot, $documentRoot) === 0)
    ? rtrim(substr($appRoot, strlen($documentRoot)), '/')
    : '';
if ($basePath !== '' && strpos($requestPath, $basePath) === 0) {
    $requestPath = substr($requestPath, strlen($basePath)) ?: '/';
}

$uri = '/' . trim(rawurldecode($requestPath), '/');
$route = trim($uri, '/');

if (preg_match('#^(en|lv|ru)(?:/(.*))?$#', $route, $matches)) {
    $_GET['lang'] = $matches[1];
    $route = trim($matches[2] ?? '', '/');
}

if ($route === '') {
    require __DIR__ . '/homepage.php';
    exit;
}

if (strpos($route, '..') !== false || strpos($route, "\0") !== false) {
    http_response_code(403);
    require __DIR__ . '/403.php';
    exit;
}

if (preg_match('#(^|/)(db|csrf|router|config)(\.php)?$#i', $route)
    || preg_match('#\.(sql|env|log|bak|ini|db)$#i', $route)
    || preg_match('#(^|/)vendor/#i', $route)
) {
    http_response_code(403);
    require __DIR__ . '/403.php';
    exit;
}

$file = __DIR__ . '/' . $route;

if (is_file($file) && pathinfo($file, PATHINFO_EXTENSION) !== 'php') {
    return false;
}

if (preg_match('/\.php$/i', $route)) {
    $cleanRoute = preg_replace('/\.php$/i', '', $route);
    $query = $_SERVER['QUERY_STRING'] ?? '';
    header('Location: ' . $basePath . '/' . $cleanRoute . ($query ? '?' . $query : ''), true, 301);
    exit;
}

$phpFile = $file . '.php';
if (!is_file($phpFile)) {
    http_response_code(404);
    require __DIR__ . '/404.php';
    exit;
```

```
}

$adminOnlyPrefixes = ['admin/', 'api/admin_', 'export/'];
foreach ($adminOnlyPrefixes as $prefix) {
    if (strpos($route, $prefix) === 0) {
        if (session_status() === PHP_SESSION_NONE) {
            require_once __DIR__ . '/api/session.php';
            start_app_session();
        }

        if (($_SESSION['role'] ?? '') !== 'admin') {
            http_response_code(403);
            require __DIR__ . '/403.php';
            exit;
        }

        break;
    }
}

require $phpFile;
```

Fails **`verify_email.php`** – e-pasta verifikācija (verifikācijas žetona pārbaude):

```
<?php
include 'api/db.php';

$message = "";
$success = false;
$emailChangeConfirmed = false;

if (!isset($_GET['token']) || empty($_GET['token'])) {
    $message = $t['invalid_link_message_generic'];
} else {
    $token = $_GET['token'];

    $stmt = $conn->prepare("SELECT id, email, email_verified, pending_email, verification_token,
        (verification_expires IS NOT NULL AND verification_expires < NOW()) AS token_expired FROM
users WHERE
        verification_token = ?");
    $stmt->bind_param("s", $token);
    $stmt->execute();
    $user = $stmt->get_result()->fetch_assoc();

    if ($user) {
        if ((int) $user['token_expired'] === 1) {
            $stmt = $conn->prepare("UPDATE users SET pending_email = NULL, verification_token =
NULL,
                verification_expires = NULL WHERE id = ? AND verification_token = ?");
            $stmt->bind_param("is", $user['id'], $token);
            $stmt->execute();
            $message = $t['invalid_or_expired_token'];
        } elseif (!empty($user['email_verified']) && !empty($user['pending_email'])) {
            $newEmail = strtolower(trim($user['pending_email']));
            $stmt = $conn->prepare("SELECT id FROM users WHERE email = ? AND id != ?");
            $stmt->bind_param("si", $newEmail, $user['id']);
            $stmt->execute();

            if ($stmt->get_result()->num_rows > 0) {
                $stmt = $conn->prepare("UPDATE users SET pending_email = NULL, verification_token =
NULL,
                    verification_expires = NULL WHERE id = ? AND verification_token = ?");
                $stmt->bind_param("is", $user['id'], $token);
                $stmt->execute();
```

```
                $message = $t['email_change_taken'];
            } else {
                try {
                    $stmt = $conn->prepare("UPDATE users SET email = ?, pending_email = NULL,
verification_token
                        = NULL, verification_expires = NULL WHERE id = ? AND verification_token = ?
AND pending_email = ?");
                    $stmt->bind_param("siss", $newEmail, $user['id'], $token,
$user['pending_email']);

                    if ($stmt->execute() && $stmt->affected_rows === 1) {
                        $message = $t['email_change_verified'];
                        $success = true;
                        $emailChangeConfirmed = true;
                    } else {
                        $message = $t['invalid_or_expired_token'];
                    }
                } catch (mysqli_sql_exception $e) {
                    $message = $t['email_change_taken'];
                }
            }
        } elseif (empty($user['email_verified'])) {
            $stmt = $conn->prepare("UPDATE users SET email_verified = 1, verification_token = NULL,
                verification_expires = NULL WHERE id = ? AND verification_token = ?");
            $stmt->bind_param("is", $user['id'], $token);

            if ($stmt->execute() && $stmt->affected_rows === 1) {
                $message = $t['verified_message'];
                $success = true;
            } else {
                $message = $t['invalid_or_expired_token'];
            }
        } else {
            $message = $t['invalid_or_expired_token'];
        }
    } else {
         $message = $t['invalid_or_expired_token'];
     }
}
?>

<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Email Verification</title>
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<link href="assets/css/nav.css" rel="stylesheet">
<link href="assets/css/theme.css" rel="stylesheet">
<link href="assets/css/verify_email.css" rel="stylesheet">
</head>
<body>

     <div class="page-wrapper">
         <?php include 'api/nav.php'; ?>

        <div class="verify-wrapper">
            <div class="verify-card">
                <?php if ($success): ?>
                    <span class="icon">✔️</span>
                    <h2><?= $t['verified_status'] ?></h2>
                    <p><?= htmlspecialchars($message) ?></p>
```

```
                    <a href="<?= $basePath ?>/<?= $emailChangeConfirmed ? 'profile' : 'login' ?>"
class="btn-
                        verify"><?= $emailChangeConfirmed ? $t['return_to_profile'] :
$t['go_to_login'] ?></a>
                <?php else: ?>
                    <span class="icon">❌</span>
                    <h2><?= $t['invalid_link_status'] ?></h2>
                    <p><?= htmlspecialchars($message) ?></p>
                    <div style="display: flex; flex-direction: column; gap: 15px; align-items:
center;">
                        <div id="resendSection" style="width: 100%; margin-bottom: 10px;">
                            <input type="email" id="resendEmail" placeholder="<?=
$t['email_label'] ?>"
                                style="margin-bottom: 10px; background: rgba(255,255,255,0.05);
border: 1px solid rgba(255,255,255,0.1); color:
                                white; padding: 12px; border-radius: 10px; width: 100%;">
                            <button onclick="resendVerification()" class="btn-verify" style="width:
100%; border:
                                none; cursor: pointer; display: block;">
                                <?= $t['resend_verification'] ?>
                            </button>
                        </div>
                        <a href="<?= $basePath ?>/register" class="btn-verify"><?=
$t['try_registering_again'] ?></a>
                        <a href="<?= $basePath ?>/homepage" style="color: rgba(255,255,255,0.4);
text-decoration:
                            none; font-size: 0.9rem;"><?= $t['back_to_home'] ?></a>
                    </div>

                     <script>
                     function resendVerification() {
                         const email = document.getElementById('resendEmail').value;
                         if (!email) { alert("Please enter your email address."); return; }

                         fetch('<?= $basePath ?>/api/resend_verification', {
                             method: 'POST',
                             headers: { 'Content-Type': 'application/json' },
                             body: JSON.stringify({ email: email })
                         })
                         .then(r => r.json())
                         .then(data => {
                             alert(data.message);
                             if (data.success) {
                                 const section = document.getElementById('resendSection');
                                 section.textContent = '';
                                 const message = document.createElement('p');
                                 message.style.cssText = 'color: #4cff88; font-size: 1rem; margin:
10px 0;';
                                 message.textContent = data.message;
                                 section.appendChild(message);

                                 if (data.verification_link) {
                                     const link = document.createElement('a');
                                     link.href = data.verification_link;
                                        link.className = 'btn-verify';
                                        link.textContent = 'Verify this account locally';
                                        section.appendChild(link);
                                    }
                                }
                             })
                             .catch(err => {
                                 console.error(err);
                                 alert("An unexpected error occurred. Please try again.");
```

```
                        });
                    }
                    </script>
                <?php endif; ?>
            </div>
        </div>

        <?php include 'api/footer.php'; ?>
    </div>

</body>
</html>
```

Fails **`admin/classes_sse.php`** – servera sūtīto notikumu (SSE) straume nodarbību atjaunošanai:

```
<?php
require_once __DIR__ . '/../api/db.php';

if (($_SESSION['role'] ?? '') !== 'admin') {
    http_response_code(403);
    exit;
}
header('Content-Type: text/event-stream');
header('Cache-Control: no-cache');
$deadline = time() + 25;

$last_id = isset($_GET['last_id']) ? (int)$_GET['last_id'] : 0;

while (time() < $deadline) {

    $stmt = $conn->prepare("SELECT id, title, NOW() as date FROM classes WHERE id > ? ORDER BY id
ASC");
    $stmt->bind_param("i", $last_id);
    $stmt->execute();
    $result = $stmt->get_result();

    $newClasses = [];
    while($row = $result->fetch_assoc()) {
        $newClasses[] = [
            'id' => $row['id'],
            'title' => $row['title'],
            'date' => $row['date'] // you can replace with default date if needed
        ];
        $last_id = $row['id'];
    }

    if(count($newClasses) > 0) {
        echo "data: ".json_encode($newClasses)."\n\n";
        ob_flush();
        flush();
    }


    sleep(3);
}
```

Fails **`admin/dashboard.php`** – administrācijas panelis ar statistiku:

```
<?php
require_once __DIR__ . '/../api/db.php';
require_once __DIR__ . '/../api/csrf.php';

if (!isset($_SESSION['user_id'])) {
```

```
    header("Location: " . $basePath . "/login");
    exit;
}

$stmt = $conn->prepare("SELECT name, role FROM users WHERE id=?");
$stmt->bind_param("i", $_SESSION['user_id']);
$stmt->execute();
$result = $stmt->get_result()->fetch_assoc();

$username = $result['name'] ?? '';
$role = $result['role'] ?? 'user';
$_SESSION['role'] = $role;

if ($role !== 'admin') {
    http_response_code(403);
    require __DIR__ . '/../403.php';
    exit;
}

$csrf_token = generate_csrf();

function getCount($conn, $action, $period, $isGuest = false) {
    $sql = "SELECT COUNT(*) as total FROM site_analytics WHERE action = ?";
    if ($isGuest) $sql .= " AND user_id IS NULL";
    else if ($action === 'login') $sql .= " AND user_id IS NOT NULL";

    if ($period === 'today') $sql .= " AND DATE(created_at) = CURDATE()";
    elseif ($period === 'month') $sql .= " AND MONTH(created_at) = MONTH(CURDATE()) AND
YEAR(created_at) =
        YEAR(CURDATE())";
    elseif ($period === 'year') $sql .= " AND YEAR(created_at) = YEAR(CURDATE())";

    $stmt = $conn->prepare($sql);
    $stmt->bind_param("s", $action);
    $stmt->execute();
    return $stmt->get_result()->fetch_assoc()['total'];
}

$top_pages = $conn->query("SELECT page_path, COUNT(*) as count FROM site_analytics WHERE
action='visit' GROUP BY
    page_path ORDER BY count DESC LIMIT 5");

$limit = 10;
$page = isset($_GET['page']) ? max(1, (int)$_GET['page']) : 1;
$offset = ($page - 1) * $limit;

$total_res = $conn->query("SELECT COUNT(*) as total FROM users")->fetch_assoc();
$total_users = $total_res['total'];
$total_pages = ceil($total_users / $limit);

$stmt = $conn->prepare("SELECT id, name, email, role, membership_type FROM users LIMIT ? OFFSET ?");
$stmt->bind_param("ii", $limit, $offset);
$stmt->execute();
$users = $stmt->get_result();

include __DIR__ . '/../views/dashboard.html';
```

Fails **`admin/delete_class.php`** – nodarbības dzēšana (administrators):

```
<?php
require_once __DIR__ . '/../api/db.php';

if (!isset($_SESSION['user_id']) || $_SESSION['role']!=='admin') {
    echo json_encode(['success'=>false,'message'=>'Access denied']);
```

```
    exit;
}

$data = json_decode(file_get_contents('php://input'), true);
$id = $data['id'] ?? null;

if (!$id) {
    echo json_encode(['success'=>false,'message'=>'Invalid class ID']);
    exit;
}

$stmt = $conn->prepare("DELETE FROM classes WHERE id=?");
$stmt->bind_param("i", $id);
$success = $stmt->execute();

$stmt2 = $conn->prepare("DELETE FROM workouts WHERE title=(SELECT title FROM classes WHERE id=?)");
$stmt2->bind_param("i", $id);
$stmt2->execute();

echo json_encode([
    'success'=>$success,
    'message'=>$success ? "Class deleted." : "Failed to delete."
]);
```

Fails **`admin/delete_user.php`** – lietotāja dzēšana (administrators):

```
<?php
header('Content-Type: application/json');
require_once __DIR__ . '/../api/db.php';
require_once __DIR__ . '/../api/csrf.php';

if (!isset($_SESSION['user_id']) || ($_SESSION['role'] ?? '') !== 'admin') {
    echo json_encode(["success"=>false,"message"=>"Access denied"]);
    exit;
}

$data = json_decode(file_get_contents("php://input"), true);
if ($_SERVER['REQUEST_METHOD'] !== 'POST' || !is_array($data) || !check_csrf($data['csrf_token'] ??
'')) {
    http_response_code(403);
    echo json_encode(["success" => false, "message" => "Invalid security token"]);
    exit;
}
$id = intval($data['user_id'] ?? 0);

if ($id <= 0) {
    echo json_encode(["success" => false, "message" => "Invalid user"]);
    exit;
}

if($id == $_SESSION['user_id']){
    echo json_encode(["success"=>false,"message"=>"Cannot delete yourself"]);
    exit;
}

$stmt = $conn->prepare("DELETE FROM users WHERE id=?");
$stmt->bind_param("i",$id);

if($stmt->execute()){
    echo json_encode(["success"=>true,"message"=>"User deleted"]);
}else{
    echo json_encode(["success"=>false,"message"=>"Delete failed"]);
}
```

Fails **`admin/index.php`** – administrācijas paneļa ieejas punkts:

```
<?php

$requestPath = parse_url($_SERVER['REQUEST_URI'] ?? '/', PHP_URL_PATH) ?: '/';
$route = trim(rawurldecode($requestPath), '/');

if (strpos($route, 'admin/') === 0) {
    $route = substr($route, strlen('admin/'));
} elseif ($route === 'admin') {
    $route = 'dashboard';
}

$route = trim($route, '/');

if ($route === '' || $route === 'index' || $route === 'dashboard') {
    require __DIR__ . '/dashboard.php';
    exit;
}

if (strpos($route, '..') !== false || strpos($route, "\0") !== false) {
    http_response_code(403);
    require __DIR__ . '/../403.php';
    exit;
}

if (preg_match('/\.php$/i', $route)) {
    $cleanRoute = preg_replace('/\.php$/i', '', $route);
    $query = $_SERVER['QUERY_STRING'] ?? '';
    $basePath = rtrim(str_replace('\\', '/', dirname(dirname($_SERVER['SCRIPT_NAME'] ?? ''))), '/');
    header('Location: ' . $basePath . '/admin/' . $cleanRoute . ($query ? '?' . $query : ''), true,
301);
    exit;
}

$phpFile = __DIR__ . '/' . $route . '.php';
if (!is_file($phpFile)) {
    http_response_code(404);
    require __DIR__ . '/../404.php';
    exit;
}

require $phpFile;
```

Fails **`admin/rename_user.php`** – lietotāja vārda maiņa (administrators):

```
<?php
header('Content-Type: application/json');
require_once __DIR__ . '/../api/db.php';
require_once __DIR__ . '/../api/csrf.php';

if (!isset($_SESSION['user_id']) || ($_SESSION['role'] ?? '') !== 'admin') {
    echo json_encode(["success"=>false,"message"=>"Access denied"]);
    exit;
}

$data = json_decode(file_get_contents("php://input"), true);
if ($_SERVER['REQUEST_METHOD'] !== 'POST' || !is_array($data) || !check_csrf($data['csrf_token'] ??
'')) {
    http_response_code(403);
    echo json_encode(["success" => false, "message" => "Invalid security token"]);
    exit;
}
```

```
$id = intval($data['user_id'] ?? 0);
$name = trim($data['new_name'] ?? '');

if ($id <= 0 || $name === '' || mb_strlen($name) > 100) {
    echo json_encode(["success" => false, "message" => "Invalid user name"]);
    exit;
}

$stmt = $conn->prepare("UPDATE users SET name=? WHERE id=?");
$stmt->bind_param("si",$name,$id);

if($stmt->execute()){
    echo json_encode(["success"=>true,"message"=>"Name updated"]);
}else{
    echo json_encode(["success"=>false,"message"=>"Update failed"]);
}
```

Fails **`admin/search_users.php`** – lietotāju meklēšana (administrators):

```
<?php
header('Content-Type: application/json');
require_once __DIR__ . '/../api/db.php';

if (!isset($_SESSION['user_id']) || $_SESSION['role'] !== 'admin') {
    echo json_encode(['success' => false, 'message' => 'Access denied']);
    exit;
}

$query = isset($_GET['q']) ? trim($_GET['q']) : '';
$page = isset($_GET['page']) ? max(1, (int)$_GET['page']) : 1;
$limit = 10;
$offset = ($page - 1) * $limit;

$sql = "SELECT id, name, email, role, membership_type FROM users";
$countSql = "SELECT COUNT(*) as total FROM users";
$params = [];
$types = "";

if ($query !== '') {
    $where = " WHERE name LIKE ? OR email LIKE ?";
    $sql .= $where;
    $countSql .= $where;
    $searchTerm = "%$query%";
    $params = [$searchTerm, $searchTerm];
    $types = "ss";
}

$stmtCount = $conn->prepare($countSql);
if ($query !== '') {
    $stmtCount->bind_param($types, ...$params);
}
$stmtCount->execute();
$totalUsers = $stmtCount->get_result()->fetch_assoc()['total'];
$totalPages = ceil($totalUsers / $limit);

$sql .= " LIMIT ? OFFSET ?";
$stmt = $conn->prepare($sql);
$limitParam = $limit;
$offsetParam = $offset;
$allParams = array_merge($params, [$limitParam, $offsetParam]);
$stmt->bind_param($types . "ii", ...$allParams);
$stmt->execute();
$users = $stmt->get_result()->fetch_all(MYSQLI_ASSOC);
```

```
echo json_encode([
    'success' => true,
    'users' => $users,
    'totalPages' => $totalPages,
    'currentPage' => $page
]);
```

Fails **`admin/update_role.php`** – lietotāja lomas maiņa (administrators):

```
<?php
header('Content-Type: application/json');
require_once __DIR__ . '/../api/db.php';
require_once __DIR__ . '/../api/csrf.php';

if (!isset($_SESSION['user_id']) || ($_SESSION['role'] ?? '') !== 'admin') {
    echo json_encode(["success" => false, "message" => "Access denied"]);
    exit;
}

$data = json_decode(file_get_contents("php://input"), true);
if ($_SERVER['REQUEST_METHOD'] !== 'POST' || !is_array($data) || !check_csrf($data['csrf_token'] ??
'')) {
    http_response_code(403);
    echo json_encode(["success" => false, "message" => "Invalid security token"]);
    exit;
}

$user_id = intval($data['user_id'] ?? 0);
$new_role = $data['new_role'] ?? '';

// Define allowed roles
$valid_roles = ['user', 'trainer', 'admin'];
if ($user_id <= 0 || !in_array($new_role, $valid_roles, true)) {
    die(json_encode(["success" => false, "message" => "Invalid role selected"]));
}

$stmt = $conn->prepare("UPDATE users SET role=? WHERE id=?");
$stmt->bind_param("si", $new_role, $user_id);

$stmt->execute();

echo json_encode([
"success"=>true,
"message"=>"Role updated"
]);
```

Fails **`api/admin_assign_membership.php`** – abonementa piešķiršana vai atcelšana (administrators):

```
<?php
require_once 'db.php';
require_once 'csrf.php';
header('Content-Type: application/json');


if (!isset($_SESSION['user_id']) || $_SESSION['role'] !== 'admin') {
    echo json_encode(['success' => false, 'message' => 'Unauthorized access']);
    exit;
}

$data = json_decode(file_get_contents("php://input"), true);

if (!isset($data['csrf_token']) || !check_csrf($data['csrf_token'])) {
    echo json_encode(['success' => false, 'message' => 'Invalid security token']);
```

```
    exit;
}

$target_user_id = intval($data['user_id'] ?? 0);
$plan = $data['plan'] ?? '';

$valid_plans = ['basic', 'advanced', 'pro', 'none'];

if ($target_user_id > 0 && in_array($plan, $valid_plans)) {
    if ($plan === 'none') {
        // Remove membership
        $stmt = $conn->prepare("UPDATE users SET membership_type = NULL, membership_expiry = NULL
WHERE id = ?");
        $stmt->bind_param("i", $target_user_id);
    } else {
        // Assign membership for 31 days
        $expiry = date('Y-m-d H:i:s', strtotime('+31 days'));
        $stmt = $conn->prepare("UPDATE users SET membership_type = ?, membership_expiry = ? WHERE id
= ?");
        $stmt->bind_param("ssi", $plan, $expiry, $target_user_id);
    }

    if ($stmt->execute()) {
        echo json_encode(['success' => true, 'message' => 'Membership successfully assigned to user
ID ' .
$target_user_id]);
    } else {
        echo json_encode(['success' => false, 'message' => 'Database update failed']);
    }
} else {
    echo json_encode(['success' => false, 'message' => 'Invalid user ID or membership plan']);
}
exit;
```

Fails **`api/buy_membership.php`** – abonementa iegāde:

```
<?php
require_once __DIR__ . '/db.php';
require_once __DIR__ . '/csrf.php';

if (basename($_SERVER['PHP_SELF'] ?? '') === 'buy_membership.php') {
    $query = $_SERVER['QUERY_STRING'] ?? '';
    header("Location: " . $basePath . "/api/buy_membership" . ($query ? '?' . $query : ''), true,
301);
    exit;
}

if (!isset($_SESSION['user_id'])) {
    header("Location: " . $basePath . "/login");
    exit;
}

$plan = $_POST['plan'] ?? $_GET['plan'] ?? '';
$validPlans = ['basic', 'advanced', 'pro'];

if (!in_array($plan, $validPlans, true)) {
    header("Location: " . $basePath . "/homepage?error=invalid_plan");
    exit;
}

if ($_SERVER['REQUEST_METHOD'] === 'POST' && !check_csrf($_POST['csrf_token'] ?? '')) {
    header("Location: " . $basePath . "/membership_offer?plan=" . urlencode($plan) . "&error=csrf");
    exit;
}
```

```
$expiry = date('Y-m-d H:i:s', strtotime('+31 days'));
$stmt = $conn->prepare("
    UPDATE users
    SET membership_type = ?, membership_expiry = ?
    WHERE id = ?
");
$stmt->bind_param("ssi", $plan, $expiry, $_SESSION['user_id']);

if (!$stmt->execute()) {
    error_log('Membership update failed: ' . $conn->error);
    header("Location: " . $basePath . "/homepage?error=membership_failed");
    exit;
}

header("Location: " . $basePath . "/homepage?success=1");
exit;
```

Fails **`api/cancel_reservation.php`** – rezervācijas atcelšana:

```
<?php
header('Content-Type: application/json');
require 'db.php';
require_once 'csrf.php';

if (!isset($_SESSION['user_id'])) {
    echo json_encode(["success" => false, "message" => "Login required"]);
    exit;
}

$data = json_decode(file_get_contents("php://input"), true);
if ($_SERVER['REQUEST_METHOD'] !== 'POST' || !is_array($data) || !check_csrf($data['csrf_token'] ??
'')) {
    http_response_code(403);
    echo json_encode(["success" => false, "message" => "Invalid security token"]);
    exit;
}
$class_id = filter_var($data['class_id'] ?? null, FILTER_VALIDATE_INT, ['options' => ['min_range' =>
1]]) ?: 0;
if (!$class_id) {
    echo json_encode(["success" => false, "message" => "Invalid class"]);
    exit;
}
$user_id = $_SESSION['user_id'];


$stmt_check = $conn->prepare("SELECT start_datetime FROM classes WHERE id = ?");
$stmt_check->bind_param("i", $class_id);
$stmt_check->execute();
$class_row = $stmt_check->get_result()->fetch_assoc();
if(!$class_row || strtotime($class_row['start_datetime']) < time()){
    die(json_encode(["success" => false, "message" => "Cannot cancel reservation for a past
workout."]));
}

$stmt = $conn->prepare("DELETE FROM reservations WHERE user_id = ? AND class_id = ?");
$stmt->bind_param("ii", $user_id, $class_id);

if ($stmt->execute()) {
    echo json_encode(["success" => true, "message" => "Reservation cancelled!"]);
} else {
    echo json_encode(["success" => false, "message" => "Failed to cancel reservation."]);
}
?>
```

Fails **`api/config.php`** – konfigurācija un vides mainīgo ielāde:

```
<?php

function app_env(string $key, $default = null) {
    static $loaded = false;
    static $env = [];

    if (!$loaded) {
        $sharedEnvFile = dirname(dirname(__DIR__)) . '/.env';
        $envFiles = [$sharedEnvFile, dirname(__DIR__) . '/.env'];
        foreach ($envFiles as $envFile) {
            if (!is_file($envFile)) {
                continue;
            }

            $lines = file($envFile, FILE_IGNORE_NEW_LINES | FILE_SKIP_EMPTY_LINES);
            if (!$lines) {
                continue;
            }

            foreach ($lines as $line) {
                $trimmed = trim($line);
                if ($trimmed === '' || strpos($trimmed, '#') === 0) {
                    continue;
                }

                $parts = explode('=', $trimmed, 2);
                if (count($parts) !== 2) {
                    continue;
                }

                $name = trim($parts[0]);
                $value = trim($parts[1]);

                // The shared XAMPP .env belongs to another local project.
                // Only reuse its explicitly named email settings; never import
                // its database credentials into Gym Tracker.
                if ($envFile === $sharedEnvFile && !in_array($name, [
                    'BOOKING_EMAIL',
                    'BOOKING_SMTP_HOST',
                    'BOOKING_SMTP_PORT',
                    'BOOKING_SMTP_USER',
                    'BOOKING_SMTP_PASSWORD'
                ], true)) {
                    continue;
                }
                $env[$name] = trim($value, " \t\n\r\0\x0B\"'");
            }
        }
        $loaded = true;
    }

    if (array_key_exists($key, $env)) {
        return $env[$key];
    }

    $value = getenv($key);
    if ($value !== false) {
        return $value;
    }

    return $default;
}
```

```
function set_app_security_headers(): void {
    if (headers_sent()) {
        return;
    }

    header('X-Content-Type-Options: nosniff');
    header('X-Frame-Options: SAMEORIGIN');
    header('Referrer-Policy: strict-origin-when-cross-origin');
    header('Permissions-Policy: geolocation=(), microphone=(), camera=()');
    header("Content-Security-Policy: default-src 'self'; script-src 'self' 'unsafe-inline'
https://cdn.jsdelivr.net https://cdnjs.cloudflare.com; style-src 'self' 'unsafe-inline'
https://fonts.googleapis.com https://fonts.gstatic.com https://cdn.jsdelivr.net
https://cdnjs.cloudflare.com;
    img-src 'self' data: https:; font-src 'self' https://fonts.gstatic.com
https://fonts.googleapis.com data:;
    connect-src 'self'; frame-src https://maps.google.com https://www.google.com; object-src 'none';
base-uri 'self';
    frame-ancestors 'self';");

    if ((!empty($_SERVER['HTTPS']) && $_SERVER['HTTPS'] !== 'off') || ($_SERVER['SERVER_PORT'] ??
'') == '443') {
        header('Strict-Transport-Security: max-age=31536000; includeSubDomains');
    }
}

define('APP_ENV', app_env('APP_ENV', 'development'));
define('DB_HOST', app_env('DB_HOST', 'localhost'));
define('DB_NAME', app_env('DB_NAME', 'gym_tracker2'));
define('DB_USER', app_env('DB_USER', 'root'));
define('DB_PASS', app_env('DB_PASS', ''));
define('DB_PORT', (int) app_env('DB_PORT', 3306));

define('STRIPE_SECRET_KEY', app_env('STRIPE_SECRET_KEY', 'sk_test_replace_with_your_key'));
define('SMTP_HOST', app_env('SMTP_HOST', app_env('BOOKING_SMTP_HOST', 'smtp.gmail.com')));
define('SMTP_PORT', (int) app_env('SMTP_PORT', app_env('BOOKING_SMTP_PORT', 587)));
define('SMTP_USERNAME', app_env('SMTP_USERNAME', app_env('BOOKING_SMTP_USER',
app_env('BOOKING_EMAIL',
    'your_email@gmail.com'))));
define('SMTP_PASSWORD', app_env('SMTP_PASSWORD', app_env('BOOKING_SMTP_PASSWORD',
'replace_with_app_password')));
    define('LOCAL_VERIFICATION_FALLBACK', filter_var(app_env('LOCAL_VERIFICATION_FALLBACK', false),
FILTER_VALIDATE_BOOLEAN));
define('STRIPE_WEBHOOK_SECRET', app_env('STRIPE_WEBHOOK_SECRET',
'whsec_replace_with_your_webhook_secret'));
```

Fails **`api/cookie_popup.php`** – sīkdatņu piekrišanas logs:

```
<!-- Cookie Consent Popup -->
<div id="cookiePopup" class="cookie-popup">
    <div class="cookie-content">
        <p><?= $t['cookie_msg'] ?></p>
    </div>
    <div class="cookie-btns">
        <button onclick="setConsent('denied')" class="btn-deny"><?= $t['cookie_deny'] ?></button>
        <button onclick="setConsent('accepted')" class="btn-accept"><?= $t['cookie_accept']
?></button>
    </div>
</div>

<script>
function getCookie(name) {
    let value = "; " + document.cookie;
    let parts = value.split("; " + name + "=");
```

```
    if (parts.length == 2) return parts.pop().split(";").shift();
}

function setConsent(choice) {
    const d = new Date();
    d.setTime(d.getTime() + (365*24*60*60*1000));
    document.cookie = "cookie_consent=" + choice + ";expires=" + d.toUTCString() + ";path=/";
    document.getElementById('cookiePopup').classList.remove('show');

    if (choice === 'accepted') {
        trackEvent('visit');
    }
}

function trackEvent(action) {
    // Uses the basePath variable from your router to find the API correctly
    const apiPath = '<?= $basePath ?? "" ?>/api/track_analytics';
    fetch(apiPath, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
            action: action,
            page_path: window.location.pathname
        })
    });
}

document.addEventListener("DOMContentLoaded", function() {
    const consent = getCookie('cookie_consent');
    if (!consent) {
        document.getElementById('cookiePopup').classList.add('show');
    } else if (consent === 'accepted') {
        // Auto-track if already accepted
        trackEvent('visit');
    }
});
</script>
```

Fails **`api/csrf.php`** – CSRF žetonu ģenerēšana un pārbaude:

```
<?php
require_once __DIR__ . '/session.php';
start_app_session();

function generate_csrf() {
    if (empty($_SESSION['csrf_token'])) {
        $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
    }
    return $_SESSION['csrf_token'];
}

function check_csrf($token) {
    return isset($_SESSION['csrf_token']) && hash_equals($_SESSION['csrf_token'], $token);
}
```

Fails **`api/db.php`** – savienojums ar datubāzi:

```
<?php
require_once __DIR__ . '/config.php';
require_once __DIR__ . '/session.php';
set_app_security_headers();

if (!isset($basePath)) {
    $appRoot = str_replace('\\', '/', dirname(__DIR__));
```

```
    $documentRoot = str_replace('\\', '/', realpath($_SERVER['DOCUMENT_ROOT'] ?? '') ?: '');

    if ($documentRoot !== '' && strpos($appRoot, $documentRoot) === 0) {
        $basePath = substr($appRoot, strlen($documentRoot));
    } else {
        $scriptDir = str_replace('\\', '/', dirname($_SERVER['SCRIPT_NAME'] ?? ''));
        $basePath = preg_replace('#/(admin|api|export)$#', '', $scriptDir);
    }

    $basePath = rtrim($basePath, '/');
}

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);

$conn = new mysqli(DB_HOST, DB_USER, DB_PASS, DB_NAME, DB_PORT);
if ($conn->connect_error) {
    error_log('Database connection failed: ' . $conn->connect_error);
    http_response_code(503);
    exit('Database unavailable.');
}
$conn->set_charset('utf8mb4');

// Start session if not already started
if (session_status() === PHP_SESSION_NONE) {
    start_app_session();
}

if (isset($_GET['lang'])) {
    $l = strtolower($_GET['lang']);
    if (in_array($l, ['en', 'lv', 'ru'], true)) {
        $_SESSION['lang'] = $l;
    }
}
$current_lang = $_SESSION['lang'] ?? 'en';
$localizedBase = $basePath . '/' . $current_lang;
$txt = include __DIR__ . '/lang.php';
$t = $txt[$current_lang];
?>
```

Fails **`api/delete_class.php`** – nodarbības dzēšana:

```
<?php
header('Content-Type: application/json');
include 'db.php';
require_once 'csrf.php';

if ($_SERVER['REQUEST_METHOD'] !== 'POST' || !isset($_SESSION['user_id']) || !
in_array($_SESSION['role'] ?? '',
    ['admin', 'trainer'], true)) {
    die(json_encode(["success"=>false,"message"=>"Unauthorized access"]));
}

$data = json_decode(file_get_contents("php://input"), true);
if (!is_array($data) || !check_csrf($data['csrf_token'] ?? '')) {
    http_response_code(403);
    die(json_encode(["success" => false, "message" => "Invalid request."]));
}
$id = filter_var($data['id'] ?? null, FILTER_VALIDATE_INT, ['options' => ['min_range' => 1]]) ?: 0;
if (!$id) {
    die(json_encode(["success" => false, "message" => "Invalid class."]));
}

$roleStmt = $conn->prepare('SELECT role FROM users WHERE id = ?');
$roleStmt->bind_param('i', $_SESSION['user_id']);
```

```
$roleStmt->execute();
$currentRole = $roleStmt->get_result()->fetch_assoc()['role'] ?? '';
if (!in_array($currentRole, ['admin', 'trainer'], true)) {
    http_response_code(403);
    die(json_encode(["success" => false, "message" => "Unauthorized access"]));
}
$_SESSION['role'] = $currentRole;

if ($currentRole === 'trainer') {
    $owner = $conn->prepare('SELECT id FROM classes WHERE id = ? AND created_by = ?');
    $owner->bind_param('ii', $id, $_SESSION['user_id']);
    $owner->execute();
    if ($owner->get_result()->num_rows === 0) {
        http_response_code(403);
        die(json_encode(["success" => false, "message" => "You can only delete your own
classes."]));
    }
}

try {
    $conn->begin_transaction();
    $stmt = $conn->prepare("DELETE FROM reservations WHERE class_id=?");
    $stmt->bind_param("i", $id);
    $stmt->execute();

    $stmt2 = $conn->prepare("DELETE FROM classes WHERE id=?");
    $stmt2->bind_param("i", $id);
    $stmt2->execute();
    $conn->commit();
    echo json_encode(["success" => true]);
} catch (mysqli_sql_exception $e) {
    $conn->rollback();
    error_log('Delete class failed: ' . $e->getMessage());
    echo json_encode(["success" => false, "message" => "Unable to delete the class."]);
}
```

Fails **`api/footer.php`** – lapas kājene:

```
<footer>
    <div>
        © <?= date('Y') ?> MyGoToGym |
        <a href="<?= $localizedBase ?>/privacy">
            <?= $t['privacy_policy'] ?>
        </a>
        | <a href="<?= $localizedBase ?>/impressum">
            <?= $t['impressum_title'] ?>
        </a>
    </div>

    <div class="social-icons">
        <a href="https://www.facebook.com/demo.gym" target="_blank" class="social-link" aria-
label="<?=
$t['facebook'] ?>" data-tooltip="<?= $t['facebook'] ?>">
            <svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24" fill="currentColor">
                <path d="M24 12.073c0-6.627-5.373-12-12-12s-12 5.373-12 12c0 5.99 3.656 11.127 8.812
13.226v-
                    9.357h-3.393v-3.869h3.393v-2.949c0-3.35 2.043-5.183 5.032-5.183 1.432 0
2.929.256 2.929.256v3.22h-1.65c-1.659 0-
                    2.176 1.03-2.176 2.088v2.588h3.628l-.58 3.869h-3.048v9.357c5.156-2.099 8.812-
7.236 8.812-13.226z"/>
            </svg>
        </a>
        <a href="https://www.instagram.com/rayweee/" target="_blank" class="social-link" aria-
label="<?=
```

```
$t['instagram'] ?>" data-tooltip="<?= $t['instagram'] ?>">
            <svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24" fill="currentColor">
                <path d="M12 2.163c3.204 0 3.584.012 4.85.07 3.252.148 4.771 1.691 4.919 4.919.058
1.265.07
                    1.645.07 4.85s-.012 3.584-.07 4.85c-.149 3.225-1.664 4.771-4.919 4.919-
1.266.058-1.644.07-4.85.07s-3.584-.012-
                    4.85-.07c-3.251-.148-4.77-1.691-4.919-4.919-.058-1.265-.07-1.644-.07-4.85s.012-
3.584.07-4.85c.149-3.227 1.664-
                    4.771 4.919-4.919 1.266-.057 1.645-.07 4.85-.07zm0 2.163c-3.204 0-3.584.012-
4.85.07-2.703.123-3.95 1.409-4.071
                    4.071-.058 1.265-.07 1.644-.07 4.85s.012 3.584.07 4.85c.121 2.662 1.368 3.948
4.071 4.071 1.265.058 1.644.07
                    4.85.07s3.584-.012 4.85-.07c2.703-.123 3.95-1.409 4.071-4.071.058-1.265.07-
1.644.07-4.85s-.012-3.584-.07-
                    4.85c-.121-2.662-1.368-3.948-4.071-4.071-1.265-.058-1.644-.07-4.85-.07zm0
3.627c-2.34 0-4.24 1.9-4.24 4.24s1.9
                    4.24 4.24 4.24 4.24-1.9 4.24-4.24-1.9-4.24-4.24-4.24zm0 2.163c1.17 0 2.077.907
2.077 2.077s-.907 2.077-2.077
                    2.077-2.077-.907-2.077-2.077.907-2.077 2.077-2.077zm5.504-5.162c0 .667-.543
1.21-1.21 1.21s-1.21-.543-1.21-
                    1.21.543-1.21 1.21-1.21 1.21.543 1.21 1.21z"/>
            </svg>
        </a>
        <a href="https://x.com/rayWee1337" target="_blank" class="social-link" aria-label="<?=
$t['twitter'] ?>"
            data-tooltip="<?= $t['twitter'] ?>">
            <svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24" fill="currentColor">
                <path d="M18.901 1.153h3.68l-8.04 9.19L24 22.846h-7.406l-5.8-7.584-6.638
7.584H.474l8.6-9.83L0
                    1.154h7.594l5.243 6.932 6.064-6.932zm-1.294 19.497h2.039L6.486 3.24H4.298l13.309
17.41z"/>
            </svg>
        </a>
    </div>
</footer>

<button id="backToTopBtn" class="back-to-top-btn" title="Go to top">↑</button>

<link rel="stylesheet" href="<?= $basePath ?>/assets/css/footer.css">
<link rel="stylesheet" href="<?= $basePath ?>/assets/css/cookie_popup.css">
<?php include __DIR__ . '/cookie_popup.php'; ?>

<script>

    let backToTopBtn = document.getElementById("backToTopBtn");


    window.onscroll = function() {
        if (document.body.scrollTop > 300 || document.documentElement.scrollTop > 300) {
            backToTopBtn.classList.add("show");
        } else {
            backToTopBtn.classList.remove("show");
        }
    };


    backToTopBtn.onclick = function() {
        window.scrollTo({
            top: 0,
            behavior: 'smooth'
        });
    };
</script>
```

Fails **`api/get_classes.php`** – nodarbību saraksta iegūšana kalendāram:

```
<?php
header('Content-Type: application/json');
require 'db.php';

function calendar_error_response(string $message): void {
    http_response_code(500);
    echo json_encode([
        'error' => true,
        'message' => $message
    ]);
    exit;
}

$sql = "
SELECT
  c.id,
  c.title,
  c.description,
  c.start_datetime,
  c.capacity,
  c.color,
  u.name AS trainer_name,
  u.role AS creator_role,
  COUNT(r.id) AS booked
FROM classes c
LEFT JOIN users u ON c.created_by = u.id
LEFT JOIN reservations r ON c.id = r.class_id
GROUP BY c.id
";

try {
    $result = $conn->query($sql);
} catch (mysqli_sql_exception $e) {
    error_log('Calendar classes query failed: ' . $e->getMessage());
    calendar_error_response('Calendar classes could not be loaded from the database.');
}

if (!$result) {
    error_log('Calendar classes query failed: ' . $conn->error);
    calendar_error_response('Calendar classes could not be loaded from the database.');
}

$colorMap = [
    'Blue'   => '#7c5cff',
    'Green' => '#28a745',
    'Red'    => '#dc3545',
    'Orange' => '#fd7e14',
    'Purple' => '#6f42c1',
    'Teal'   => '#20c997'
];

$events = [];

while ($row = $result->fetch_assoc()) {
    $dbColor = !empty($row["color"]) ? $row["color"] : "Blue";
    $hexColor = $colorMap[$dbColor] ?? $colorMap['Blue'];

    $events[] = [
        "id" => $row["id"],
        "title" => $row["title"],
        "start" => $row["start_datetime"],
        "color" => $hexColor,
```

```
        "extendedProps" => [
            "description" => $row["description"],
            "capacity" => (int)$row["capacity"],
            "booked" => (int)$row["booked"],
            "original_color" => $dbColor, // This will be the name (e.g., "Blue")
            "trainer_name" => $row["trainer_name"],
             "creator_role" => $row["creator_role"]
         ]
    ];
}

echo json_encode($events);
```

Fails **`api/get_participants.php`** – nodarbības dalībnieku saraksta iegūšana:

```
<?php
header('Content-Type: application/json');
require 'db.php';

if (!isset($_SESSION['user_id'])) {
    http_response_code(401);
    echo json_encode(['success' => false, 'message' => 'Login required.']);
    exit;
}
if (!isset($_GET['class_id']) || !ctype_digit((string) $_GET['class_id'])) {
    echo json_encode([]);
    exit;
}

$class_id = (int) $_GET['class_id'];

$stmt = $conn->prepare("
    SELECT u.id, u.name
    FROM users u
    JOIN reservations r ON u.id = r.user_id
    WHERE r.class_id = ? AND r.status = 'reserved'
");
$stmt->bind_param("i", $class_id);
$stmt->execute();
$result = $stmt->get_result();

$participants = [];
while($row = $result->fetch_assoc()){
    $participants[] = $row;
}

echo json_encode($participants);
```

Fails **`api/get_workouts.php`** – lietotāja nodarbību saraksta iegūšana:

```
<?php
require __DIR__ . '/db.php';

header('Content-Type: application/json');

if (!isset($_SESSION['user_id'])) {
    http_response_code(401);
    echo json_encode([]);
    exit;
}

$stmt = $conn->prepare("SELECT workout_date, muscles, weight, reps FROM workouts WHERE user_id=?");
```

```
$stmt->bind_param("i", $_SESSION['user_id']);
$stmt->execute();
$result = $stmt->get_result();

$events = [];

while ($row = $result->fetch_assoc()) {
    $events[] = [
        'title' => $row['muscles'] . ' - ' . $row['weight'] . 'kg x ' . $row['reps'],
        'start' => $row['workout_date'],
        'allDay' => true
    ];
}

echo json_encode($events);
```

Fails **`api/index.php`** – API mapes aizsargfails:

```
<?php

$requestPath = parse_url($_SERVER['REQUEST_URI'] ?? '/', PHP_URL_PATH) ?: '/';
$route = trim(rawurldecode($requestPath), '/');

if (strpos($route, 'api/') === 0) {
    $route = substr($route, strlen('api/'));
} elseif ($route === 'api') {
    http_response_code(404);
    require __DIR__ . '/../404.php';
    exit;
}

$route = trim($route, '/');

if ($route === '' || strpos($route, '..') !== false || strpos($route, "\0") !== false) {
    http_response_code(403);
    require __DIR__ . '/../403.php';
    exit;
}

if (preg_match('#^(db|csrf|config|Mailer)(\.php)?$#i', $route)) {
    http_response_code(403);
    require __DIR__ . '/../403.php';
    exit;
}

if (preg_match('/\.php$/i', $route)) {
    $cleanRoute = preg_replace('/\.php$/i', '', $route);
    $query = $_SERVER['QUERY_STRING'] ?? '';
    $basePath = rtrim(str_replace('\\', '/', dirname(dirname($_SERVER['SCRIPT_NAME'] ?? ''))), '/');
    header('Location: ' . $basePath . '/api/' . $cleanRoute . ($query ? '?' . $query : ''), true,
301);
    exit;
}

$phpFile = __DIR__ . '/' . $route . '.php';
if (!is_file($phpFile)) {
    http_response_code(404);
    require __DIR__ . '/../404.php';
    exit;
}

require $phpFile;
```

Fails **`api/lang.php`** – daudzvalodības vārdnīca (latviešu, angļu, krievu):

```
<?php
return [
    'en' => [
        'home' => 'Home', 'schedule' => 'Schedule', 'contacts' => 'Contacts',
        'login' => 'Login', 'register' => 'Register', 'logout' => 'Logout',
        'profile' => 'Profile', 'admin_panel' => 'Admin Panel',
        'welcome' => 'Welcome', 'name' => 'Name', 'email' => 'Email',
        'phone' => 'Phone', 'address' => 'Address', 'password' => 'Password',
        'invalid_phone' => 'Enter a valid international phone number with a + country code.',
        'confirm_pass' => 'Confirm Password', 'current_pass' => 'Current Password',
        'new_pass' => 'New Password', 'update_info' => 'Update Info',
        'profile_info_updated' => 'Profile information updated.',
        'phone_updated' => 'Phone number updated.',
        'phone_and_email_change_confirmation_sent' => 'Phone number updated. We sent a confirmation
link to your
            new email. Your current email stays active until confirmation.',
        'email_change_confirmation_sent' => 'We sent a confirmation link to your new email. Your
current email
            stays active until confirmation.',
        'phone_updated_email_change_confirmation_failed' => 'Phone updated, but we could not send
the
            confirmation email. Your current email remains unchanged. Please try again.',
        'email_change_confirmation_failed' => 'We could not send the confirmation email. Your
current email
            remains unchanged. Please try again.',
        'email_change_local_fallback' => 'Email delivery is unavailable. Use this local link to
confirm the
            address change.',
        'email_already_in_use' => 'That email address is already in use.',
        'email_change_pending' => 'Confirmation pending for',
        'local_email_verification_link' => 'Verify new email',
        'update_pass' => 'Update Password', 'class_details' => 'Class Details',
        'participants' => 'Participants', 'hero_h1' => 'Train Like an Athlete.',
        'hero_p' => 'Premium fitness experience with elite trainers and science-based programs.',
        'join_now' => 'Join Now', 'view_plans' => 'View Plans',
        'select_class' => 'Select a class on the calendar to view full details here.',
        'join_msg' => 'Join a class to see who else is coming!',
        'why_choose_us' => 'Why Choose Us', 'elite_equip' => 'Elite Equipment',
        'expert_coaches' => 'Expert Coaches', 'flex_training' => 'Flexible Training',
        'our_trainers' => 'Our Trainers', 'pop_classes' => 'Popular Classes',
        'member_plans' => 'Membership Plans', 'start_trans' => 'Start Your Transformation Today',
        'get_started' => 'Get Started', 'name_label' => 'Name', 'email_label' => 'Email Address',
        'phone_label' => 'Phone Number', 'address_label' => 'Address',
        'time' => 'Time', 'attendance' => 'Attendance', 'class_full' => 'Class Full',
        'loading_athletes' => 'Loading athletes...', 'no_participants' => 'No one has joined yet. Be
the first!',
        'total_joined' => 'Total joined', 'cancel_my_res' => 'Cancel My Reservation',
        'export_excel' => 'Export Participants (Excel)', 'print_pdf' => 'Print Participant List
(PDF)',
        'create_workout' => 'Create Workout', 'save' => 'Save', 'delete' => 'Delete', 'close' =>
'Close',
        'forgot_password' => 'Forgot Password?',
        'cancel' => 'Cancel', 'login_to_view' => 'Please login to view participants.', 'no_account'
=> 'No
            account?', 'already_account' => 'Already have an account?',
        'gym_only' => 'Gym access only', 'gym_classes' => 'Gym + Classes', 'trainer_included' =>
'Gym + Classes
            and Personal trainer included',
        'join' => 'Join', 'basic' => 'Basic', 'advanced' => 'Advanced', 'pro' => 'Pro',
        'acc_details' => 'Account Details', 'change_pass' => 'Change Password',
        'pass_hint' => '6+ chars, 1 uppercase, 1 number, 1 symbol',
        'membership_status' => 'Membership Status',
```

```
        'no_active_membership' => 'No Active Membership',
        'expires_on' => 'Expires on:',
        'days_left' => 'days left', 'boxing' => 'Boxing', 'yoga' => 'Yoga', 'crossfit' =>
'CrossFit',
        'none' => 'None',
        'strength_coach' => 'Strength Coach', 'yoga_inst' => 'Yoga Instructor', 'cf_coach' =>
'CrossFit Coach',
        'res_cancelled' => 'Reservation cancelled. Select again to refresh.',
        'confirm_cancel' => 'Are you sure you want to cancel your reservation?',
        'today' => 'Today',
        'remaining' => 'Remaining',
        'reserve' => 'Reserve',
        'assign_membership' => 'Assign Membership',
        'select_user' => 'Select User',
        'confirm_delete' => 'Delete class?',
        'save_failed' => 'Save failed',
        'sidebar_hint' => 'Click the event again to open the reservation or edit modal.',
        'elite_p' => 'World-class machines designed for performance and safety.', // Corrected typo
from previous
            context
        'expert_p' => 'Certified professionals guiding your transformation.', // Corrected typo from
previous
            context
          'flex_p' => 'Boxing, yoga, crossfit and personal coaching programs.', // Corrected typo
from previous
              context
        'cta_members' => '500+ Members',
        'cta_booking' => 'Live Booking',
        'cta_results' => 'Pro Results',
        'privacy_policy' => 'Privacy Policy',
        'facebook' => 'Facebook',
        'instagram' => 'Instagram',
        'twitter' => 'X (Twitter)',
        'cookie_msg' => 'We use cookies to improve your experience and collect site usage
statistics.',
        'cookie_deny' => 'Deny',
        'cookie_accept' => 'Accept',
        'privacy_title' => 'Privacy Policy',
        'privacy_intro' => 'This Privacy Policy describes how MyGoToGym collects, uses, and
discloses your
            Personal Information when you visit or make a purchase from the Site.',
        'p_sec1_t' => '1. Information We Collect',
        'p_sec1_c' => 'We collect personal information you provide directly to us, such as your
name, email
            address, phone number, and membership details when you register or use our services.',
        'p_sec2_t' => '2. How We Use Your Information',
        'p_sec2_c' => 'We use the information we collect to provide, maintain, and improve our
services, process
            transactions, send you notifications, and for marketing purposes.',
        'p_sec3_t' => '3. Sharing Your Personal Information',
        'p_sec3_c' => 'We do not share your personal information with third parties except as
necessary to
            provide our services, comply with the law, or protect our rights.',
        'p_sec4_t' => '4. Your Rights',
        'p_sec4_c' => 'You have the right to access, correct, or delete your personal information.
Please contact
            us to exercise these rights.',
        'p_sec5_t' => '5. Changes to This Policy',
        'p_sec5_c' => 'We may update this privacy policy from time to time. We will notify you of
any changes by
            posting the new policy on this page.',
        'p_contact' => 'If you have any questions about this Privacy Policy, please contact us at
            info@mygotogym.com.',
```

```
        'confirm_membership' => 'Confirm Membership',
        'confirm_purchase' => 'Confirm Purchase',
        'membership_terms' => 'By confirming, you agree to our membership terms and conditions.',
        'payment_history' => 'Payment History',
        'impressum_title' => 'Impressum',
        'impressum_operator' => 'Legal Operator',
        'impressum_contact' => 'Contact Information',
        'impressum_disclaimer' => 'Disclaimer',
        'impressum_disclaimer_text' => 'The content of our pages was created with great care.
However, we cannot
            guarantee the accuracy, completeness, or timeliness of the content.',
        'date' => 'Date',
        'amount' => 'Amount',
        'status' => 'Status',
        'no_payments' => 'No payment history found.',
        'holiday_error' => 'Workouts cannot be scheduled on public holidays.',
        'trainer_label' => 'Trainer',
        'admin_name' => 'MyGoToGym Administration',
        'verified_status' => 'Verified',
        'verified_message' => 'Email successfully verified! You can now login.',
        'email_change_verified' => 'Your new email address has been confirmed.',
        'email_change_taken' => 'That email address is already in use. Your current email was not
changed.',
        'return_to_profile' => 'Return to Profile',
        'go_to_login' => 'Go to Login',
        'invalid_link_status' => 'Invalid Link',
        'invalid_link_message_generic' => 'Invalid verification link.',
        'invalid_or_expired_token' => 'Invalid or expired verification token.',
        'try_registering_again' => 'Try Registering Again',
        'back_to_home' => 'Back to Home',
        'resend_verification' => 'Resend Verification Email',
        'reset_link_sent' => 'If that email is registered and verified, a reset link has been
sent.',
        'pass_reset_subject' => 'Reset your MyGoToGym Password',
        'pass_reset_welcome' => 'Password Reset',
        'pass_reset_body' => 'You requested a password reset. Click the button below to set a new
password. This
            link expires in 1 hour.',
        'pass_reset_button' => 'Reset Password',
        'pass_reset_footer' => 'If you did not request this, please ignore this email.',
        'password_mismatch' => 'Passwords do not match.',
        'password_min_length' => 'Password must be at least 8 characters.',
        'password_updated_success' => 'Password updated successfully!',
        'token_invalid' => 'Token invalid or expired.',
        'enter_email_to_reset' => 'Enter your email to receive a reset link.',
        'reset_cooldown' => 'Please wait 5 minutes before requesting another reset link.',
        'send_reset_link' => 'Send Reset Link'
    ],
    'lv' => [
        'home' => 'Sākums', 'schedule' => 'Grafiks', 'contacts' => 'Kontakti',
        'login' => 'Ienākt', 'register' => 'Reģistrēties', 'logout' => 'Iziet',
        'profile' => 'Profils', 'admin_panel' => 'Admin Panelis',
        'welcome' => 'Sveicināti', 'name' => 'Vārds', 'email' => 'E-pasts',
        'phone' => 'Telefons', 'address' => 'Adrese', 'password' => 'Parole',
        'invalid_phone' => 'Ievadiet derīgu starptautisko tālruņa numuru ar valsts kodu +.',
        'confirm_pass' => 'Apstiprināt paroli', 'current_pass' => 'Pašreizējā parole',
        'new_pass' => 'Jaunā parole', 'update_info' => 'Atjaunot info',
        'profile_info_updated' => 'Profila informācija atjaunināta.',
        'phone_updated' => 'Tālruņa numurs atjaunināts.',
        'phone_and_email_change_confirmation_sent' => 'Tālrunis atjaunināts. Uz jauno e-pastu
nosūtīta
            apstiprinājuma saite. Pašreizējais e-pasts būs aktīvs līdz apstiprināšanai.',
```

```
        'email_change_confirmation_sent' => 'Uz jauno e-pastu nosūtīta apstiprinājuma saite.
Pašreizējais e-pasts
            būs aktīvs līdz apstiprināšanai.',
        'phone_updated_email_change_confirmation_failed' => 'Tālrunis atjaunināts, bet neizdevās
nosūtīt
            apstiprinājuma e-pastu. Pašreizējais e-pasts nav mainīts. Mēģiniet vēlreiz.',
        'email_change_confirmation_failed' => 'Neizdevās nosūtīt apstiprinājuma e-pastu.
Pašreizējais e-pasts nav
            mainīts. Mēģiniet vēlreiz.',
        'email_change_local_fallback' => 'E-pasta piegāde nav pieejama. Izmantojiet šo lokālo saiti,
lai
            apstiprinātu adreses maiņu.',
        'email_already_in_use' => 'Šī e-pasta adrese jau tiek izmantota.',
        'email_change_pending' => 'Apstiprinājums gaida šo adresi:',
        'local_email_verification_link' => 'Apstiprināt jauno e-pastu',
        'update_pass' => 'Mainīt paroli', 'class_details' => 'Nodarbības detaļas',
        'participants' => 'Dalībnieki', 'hero_h1' => 'Trenējies kā atlēts.',
        'hero_p' => 'Premium fitnesa pieredze ar elites treneriem un zinātnē balstītām programmām.',
        'join_now' => 'Pievienoties', 'view_plans' => 'Plāni',
        'select_class' => 'Izvēlieties nodarbību kalendārā, lai redzētu detaļas šeit.',
        'join_msg' => 'Pievienojies nodarbībai, lai redzētu, kas vēl nāks!',
        'why_choose_us' => 'Kāpēc izvēlēties mūs', 'elite_equip' => 'Elites aprīkojums',
        'expert_coaches' => 'Ekspertu treneri', 'flex_training' => 'Fleksibli treniņi',
        'our_trainers' => 'Mūsu treneri', 'pop_classes' => 'Populāras nodarbības',
        'member_plans' => 'Abonementu plāni', 'start_trans' => 'Sāc savas pārmaiņas šodien',
        'get_started' => 'Sākt tūlīt', 'name_label' => 'Vārds', 'email_label' => 'E-pasta adrese',
        'phone_label' => 'Telefona numurs', 'address_label' => 'Adrese',
        'time' => 'Laiks', 'attendance' => 'Apmeklējums', 'class_full' => 'Grupa pilna',
        'loading_athletes' => 'Ielādē atlētus...', 'no_participants' => 'Neviens vēl nav pieteicies.
Esi
            pirmais!',
        'total_joined' => 'Kopā pieteikušies', 'cancel_my_res' => 'Atcelt manu rezervāciju',
        'export_excel' => 'Eksportēt dalībniekus (Excel)', 'print_pdf' => 'Drukāt sarakstu (PDF)',
        'create_workout' => 'Izveidot treniņu', 'save' => 'Saglabāt', 'delete' => 'Dzēst', 'close'
=> 'Aizvērt',
        'forgot_password' => 'Aizmirsu paroli?',
        'cancel' => 'Atcelt', 'login_to_view' => 'Lūdzu, ienāciet, lai redzētu dalībniekus.',
'no_account' =>
            'Nav konta?', 'already_account' => 'Jau ir konts?',
        'gym_only' => 'Tikai trenažieru zāle', 'gym_classes' => 'Zāle + Nodarbības',
'trainer_included' => 'Zāle
            + Nodarbības un personīgais treneris',
        'join' => 'Pievienoties', 'basic' => 'Pamata', 'advanced' => 'Paplašinātais', 'pro' =>
'Pro',
        'acc_details' => 'Konta informācija', 'change_pass' => 'Mainīt paroli',
        'pass_hint' => '6+ zīmes, 1 lielais burts, 1 skaitlis, 1 simbols',
        'membership_status' => 'Abonementa statuss',
        'no_active_membership' => 'Nav aktīva abonementa',
        'expires_on' => 'Derīgs līdz:',
        'days_left' => 'dienas atlikušas', 'boxing' => 'Bokss', 'yoga' => 'Joga', 'crossfit' =>
'Krossfits',
        'none' => 'Nav',
        'strength_coach' => 'Spēka treneris', 'yoga_inst' => 'Jogas instruktore', 'cf_coach' =>
'Krossfita
            treneris',
        'res_cancelled' => 'Rezervācija atcelta. Izvēlieties vēlreiz, lai atjauninātu.',
        'confirm_cancel' => 'Vai tiešām vēlaties atcelt rezervāciju?',
        'today' => 'Šodien',
        'remaining' => 'Atlicis',
        'reserve' => 'Rezervēt',
        'assign_membership' => 'Piešķirt abonementu',
        'select_user' => 'Izvēlēties lietotāju',
        'confirm_delete' => 'Dzēst nodarbību?',
```

```
        'save_failed' => 'Saglabāšana neizdevās',
        'sidebar_hint' => 'Noklikšķiniet vēlreiz, lai atvērtu rezervācijas vai rediģēšanas logu.',
          'elite_p' => 'Pasaules līmeņa trenažieri veiktspējai un drošībai.', // Corrected typo from
previous
              context
          'expert_p' => 'Sertificēti profesionāļi jūsu izaugsmei.', // Corrected typo from previous
context
          'flex_p' => 'Boksa, jogas, krossfita un personīgo treniņu programmas.', // Corrected typo
from previous
              context
        'cta_members' => '500+ Dalībnieki',
        'cta_booking' => 'Tiešsaistes rezervācija',
        'cta_results' => 'Profesionāli rezultāti',
        'privacy_policy' => 'Privātuma politika',
        'facebook' => 'Facebook',
        'instagram' => 'Instagram',
        'twitter' => 'X (Twitter)',
        'cookie_msg' => 'Mēs izmantojam sīkdatnes, lai uzlabotu jūsu pieredzi un apkopotu vietnes
lietošanas
            statistiku.',
        'cookie_deny' => 'Noraidīt',
        'cookie_accept' => 'Pieņemt',
        'privacy_title' => 'Privātuma Politika',
        'privacy_intro' => 'Šī Privātuma Politika apraksta, kā MyGoToGym vāc, izmanto un atklāj jūsu
Personisko
            Informāciju, kad apmeklējat vai veicat pirkumu vietnē.',
        'p_sec1_t' => '1. Mūsu vāktā informācija',
        'p_sec1_c' => 'Mēs apkopojam personisko informāciju, ko jūs mums sniedzat tieši, piemēram,
jūsu vārdu, e-
            pasta adresi, tālruņa numuru un dalības informāciju, kad reģistrējaties vai izmantojat
mūsu pakalpojumus.',
        'p_sec2_t' => '2. Kā mēs izmantojam jūsu informāciju',
        'p_sec2_c' => 'Mēs izmantojam apkopoto informāciju, lai nodrošinātu, uzturētu un uzlabotu
mūsu
            pakalpojumus, apstrādātu darījumus, sūtītu jums paziņojumus un mārketinga nolūkos.',
        'p_sec3_t' => '3. Personiskās informācijas koplietošana',
        'p_sec3_c' => 'Mēs nekopīgojam jūsu personisko informāciju ar trešajām personām, izņemot
gadījumus, kad
            tas ir nepieciešams mūsu pakalpojumu sniegšanai, likumu ievērošanai vai mūsu tiesību
aizsardzībai.',
        'p_sec4_t' => '4. Jūsu tiesības',
        'p_sec4_c' => 'Jums ir tiesības piekļūt, labot vai dzēst savu personisko informāciju. Lūdzu,
sazinieties
            ar mums, lai izmantotu šīs tiesības.',
        'p_sec5_t' => '5. Izmaiņas šajā politikā',
        'p_sec5_c' => 'Mēs laiku pa laikam varam atjaunināt šo privātuma politiku. Mēs jūs
informēsim par
            jebkurām izmaiņām, publicējot jauno politiku šajā lapā.',
        'p_contact' => 'Ja jums ir kādi jautājumi par šo Privātuma Politiku, lūdzu, sazinieties ar
mums pa e-
            pastu info@mygotogym.com.',
        'confirm_membership' => 'Apstiprināt dalību',
        'confirm_purchase' => 'Apstiprināt pirkumu',
        'membership_terms' => 'Apstiprinot, jūs piekrītat mūsu dalības noteikumiem un
nosacījumiem.',
        'payment_history' => 'Maksājumu vēsture',
        'impressum_title' => 'Impresums',
        'impressum_operator' => 'Juridiskais operators',
        'impressum_contact' => 'Kontaktinformācija',
        'impressum_disclaimer' => 'Atruna',
        'impressum_disclaimer_text' => 'Mūsu lapu saturs ir izveidots ar lielu rūpību. Tomēr mēs
nevaram garantēt
            satura pareizību, pilnīgumu vai savlaicīgumu.',
```

```
        'date' => 'Datums',
        'amount' => 'Summa',
        'status' => 'Statuss',
        'no_payments' => 'Maksājumu vēsture nav atrasta.',
        'holiday_error' => 'Nodarbības nevar plānot svētku dienās.',
        'trainer_label' => 'Treneris',
        'admin_name' => 'MyGoToGym administrācija',
        'verified_status' => 'Apstiprināts',
        'verified_message' => 'E-pasts veiksmīgi apstiprināts! Tagad varat pieteikties.',
        'email_change_verified' => 'Jaunā e-pasta adrese ir apstiprināta.',
        'email_change_taken' => 'Šī e-pasta adrese jau tiek izmantota. Pašreizējā adrese netika
mainīta.',
        'return_to_profile' => 'Atgriezties profilā',
        'go_to_login' => 'Doties uz pieteikšanos',
        'invalid_link_status' => 'Nederīga saite',
        'invalid_link_message_generic' => 'Nederīga verifikācijas saite.',
        'invalid_or_expired_token' => 'Nederīga vai beidzies verifikācijas žetons.',
        'try_registering_again' => 'Mēģiniet reģistrēties vēlreiz',
        'back_to_home' => 'Atpakaļ uz sākumu',
        'resend_verification' => 'Nosūtīt verifikācijas e-pastu vēlreiz',
        'reset_link_sent' => 'Ja šis e-pasts ir reģistrēts un apstiprināts, atiestatīšanas saite ir
nosūtīta.',
        'pass_reset_subject' => 'Atiestatiet savu MyGoToGym paroli',
        'pass_reset_welcome' => 'Paroles atiestatīšana',
        'pass_reset_body' => 'Jūs pieprasījāt paroles atiestatīšanu. Noklikšķiniet uz zemāk esošās
pogas, lai
            iestatītu jaunu paroli. Šī saite ir derīga 1 stundu.',
        'pass_reset_button' => 'Atiestatīt paroli',
        'pass_reset_footer' => 'Ja jūs to nepieprasījāt, lūdzu, ignorējiet šo e-pastu.',
        'password_mismatch' => 'Paroles nesakrīt.',
        'password_min_length' => 'Parolei jābūt vismaz 8 rakstzīmēm.',
        'password_updated_success' => 'Parole veiksmīgi atjaunota!',
        'token_invalid' => 'Žetons ir nederīgs vai beidzies.',
        'enter_email_to_reset' => 'Ievadiet savu e-pastu, lai saņemtu atiestatīšanas saiti.',
        'reset_cooldown' => 'Lūdzu, uzgaidiet 5 minūtes pirms pieprasāt jaunu saiti.',
        'send_reset_link' => 'Sūtīt atiestatīšanas saiti'
    ],
    'ru' => [
        'home' => 'Главная', 'schedule' => 'Расписание', 'contacts' => 'Контакты',
        'login' => 'Войти', 'register' => 'Регистрация', 'logout' => 'Выход',
        'profile' => 'Профиль', 'admin_panel' => 'Админ Панель',
        'welcome' => 'Добро пожаловать', 'name' => 'Имя', 'email' => 'Эл. почта',
        'phone' => 'Телефон', 'address' => 'Адрес', 'password' => 'Пароль',
        'invalid_phone' => 'Введите действительный международный номер телефона с кодом страны,
начиная с +.',
        'confirm_pass' => 'Подтвердите пароль', 'current_pass' => 'Текущий пароль',
        'new_pass' => 'Новый пароль', 'update_info' => 'Обновить инфо',
        'profile_info_updated' => 'Данные профиля обновлены.',
        'phone_updated' => 'Номер телефона обновлен.',
        'phone_and_email_change_confirmation_sent' => 'Номер телефона обновлен. Мы отправили ссылку
для
            подтверждения на новый адрес. Текущий адрес останется активным до подтверждения.',
        'email_change_confirmation_sent' => 'Мы отправили ссылку для подтверждения на новый адрес.
Текущий адрес
            останется активным до подтверждения.',
        'phone_updated_email_change_confirmation_failed' => 'Телефон обновлен, но письмо с
подтверждением
            отправить не удалось. Текущий адрес не изменен. Попробуйте еще раз.',
        'email_change_confirmation_failed' => 'Письмо с подтверждением отправить не удалось. Текущий
адрес не
            изменен. Попробуйте еще раз.',
        'email_change_local_fallback' => 'Доставка почты недоступна. Используйте локальную ссылку
для
```

```
            подтверждения нового адреса.',
        'email_already_in_use' => 'Этот адрес электронной почты уже используется.',
        'email_change_pending' => 'Ожидается подтверждение для адреса:',
        'local_email_verification_link' => 'Подтвердить новый адрес',
        'update_pass' => 'Сменить пароль', 'class_details' => 'Детали занятия',
        'participants' => 'Участники', 'hero_h1' => 'Тренируйся как атлет.',
        'hero_p' => 'Премиальный фитнес с элитными тренерами и научными программами.',
        'join_now' => 'Записаться', 'view_plans' => 'Планы',
        'select_class' => 'Выберите занятие в календаре для деталей здесь.',
        'join_msg' => 'Присоединяйтесь, чтобы увидеть список участников!',
        'why_choose_us' => 'Почему мы', 'elite_equip' => 'Элитное оборудование',
        'expert_coaches' => 'Экспертные тренеры', 'flex_training' => 'Гибкие тренировки',
        'our_trainers' => 'Наши тренеры', 'pop_classes' => 'Популярные занятия',
        'member_plans' => 'Планы подписки', 'start_trans' => 'Начни трансформацию сегодня',
        'get_started' => 'Начать', 'name_label' => 'Имя', 'email_label' => 'Эл. почта',
        'phone_label' => 'Номер телефона', 'address_label' => 'Адрес',
        'time' => 'Время', 'attendance' => 'Посещаемость', 'class_full' => 'Группа полна',
        'loading_athletes' => 'Загрузка атлетов...', 'no_participants' => 'Никто еще не записался.
Будь первым!',
        'total_joined' => 'Всего участников', 'cancel_my_res' => 'Отменить мою запись',
        'export_excel' => 'Экспорт участников (Excel)', 'print_pdf' => 'Печать списка (PDF)',
        'create_workout' => 'Создать тренировку', 'save' => 'Сохранить', 'delete' => 'Удалить',
'close' =>
            'Закрыть',
        'forgot_password' => 'Забыли пароль?',
        'cancel' => 'Отмена', 'login_to_view' => 'Пожалуйста, войдите, чтобы увидеть участников.',
'no_account'
            => 'Нет аккаунта?', 'already_account' => 'Уже есть аккаунт?',
        'gym_only' => 'Только тренажерный зал', 'gym_classes' => 'Зал + Занятия', 'trainer_included'
=> 'Зал +
            Занятия и персональный тренер включен',
        'join' => 'Вступить', 'basic' => 'Базовый', 'advanced' => 'Продвинутый', 'pro' => 'Про',
        'acc_details' => 'Данные аккаунта', 'change_pass' => 'Сменить пароль',
        'pass_hint' => '6+ симв., 1 заглавная, 1 цифра, 1 символ',
        'membership_status' => 'Статус подписки',
        'no_active_membership' => 'Нет активной подписки',
        'expires_on' => 'Истекает:',
        'days_left' => 'дней осталось', 'boxing' => 'Бокс', 'yoga' => 'Йога', 'crossfit' =>
'Кроссфит',
        'none' => 'Нет',
        'strength_coach' => 'Тренер по силе', 'yoga_inst' => 'Инструктор по йоге', 'cf_coach' =>
'Тренер по
            кроссфиту',
        'res_cancelled' => 'Запись отменена. Выберите снова для обновления.',
        'confirm_cancel' => 'Вы уверены, что хотите отменить запись?',
        'today' => 'Сегодня',
        'remaining' => 'Осталось',
        'reserve' => 'Забронировать',
        'assign_membership' => 'Назначить подписку',
        'select_user' => 'Выбрать пользователя',
        'confirm_delete' => 'Удалить занятие?',
        'save_failed' => 'Ошибка сохранения',
        'sidebar_hint' => 'Нажмите на событие еще раз, чтобы открыть окно бронирования или
редактирования.',
        'elite_p' => 'Тренажеры мирового класса для производительности и безопасности.', //
Corrected typo from
            previous context
        'expert_p' => 'Сертифицированные профи для вашего прогресса.', // Corrected typo from
previous context
        'flex_p' => 'Программы бокса, йоги, кроссфита и личного коучинга.', // Corrected typo from
previous
            context
        'cta_members' => '500+ Участников',
```

```
        'cta_booking' => 'Онлайн бронирование',
        'cta_results' => 'Профессиональные результаты',
        'privacy_policy' => 'Политика конфиденциальности',
        'facebook' => 'Facebook',
        'instagram' => 'Instagram',
        'twitter' => 'X (Twitter)',
        'cookie_msg' => 'Мы используем файлы cookie для улучшения вашего взаимодействия с сайтом и
сбора
            статистики использования.',
        'cookie_deny' => 'Отклонить',
        'cookie_accept' => 'Принять',
        'privacy_title' => 'Политика конфиденциальности',
        'privacy_intro' => 'Настоящая Политика конфиденциальности описывает, как MyGoToGym собирает,
использует и
            раскрывает вашу Личную информацию, когда вы посещаете или совершаете покупку на Сайте.',
        'p_sec1_t' => '1. Собираемая нами информация',
        'p_sec1_c' => 'Мы собираем личную информацию, которую вы предоставляете нам напрямую, такую
как ваше имя,
            адрес электронной почты, номер телефона и данные о членстве, когда вы регистрируетесь
или используете наши
            услуги.',
        'p_sec2_t' => '2. Как мы используем вашу информацию',
        'p_sec2_c' => 'Мы используем собранную информацию для предоставления, поддержки и улучшения
наших услуг,
            обработки транзакций, отправки вам уведомлений и в маркетинговых целях.',
        'p_sec3_t' => '3. Передача вашей личной информации',
        'p_sec3_c' => 'Мы не передаем вашу личную информацию третьим лицам, за исключением случаев,
когда это
            необходимо для предоставления наших услуг, соблюдения законодательства или защиты наших
прав.',
        'p_sec4_t' => '4. Ваши права',
        'p_sec4_c' => 'Вы имеете право на доступ, исправление или удаление вашей личной информации.
Пожалуйста,
            свяжитесь с нами, чтобы воспользоваться этими правами.',
        'p_sec5_t' => '5. Изменения в этой политике',
        'p_sec5_c' => 'Мы можем время от времени обновлять настоящую политику конфиденциальности. Мы
уведомим вас
            о любых изменениях, разместив новую политику на этой странице.',
        'p_contact' => 'Если у вас есть какие-либо вопросы относительно настоящей Политики
конфиденциальности,
            пожалуйста, свяжитесь с нами по адресу info@mygotogym.com.',
        'confirm_membership' => 'Подтвердить членство',
        'confirm_purchase' => 'Подтвердить покупку',
        'membership_terms' => 'Подтверждая, вы соглашаетесь с нашими условиями членства.',
        'payment_history' => 'История платежей',
        'impressum_title' => 'Выходные данные',
        'impressum_operator' => 'Юридический оператор',
        'impressum_contact' => 'Контактная информация',
        'impressum_disclaimer' => 'Отказ от ответственности',
        'impressum_disclaimer_text' => 'Содержимое наших страниц было создано с большой
тщательностью. Однако мы
            не можем гарантировать точность, полноту или своевременность контента.',
        'date' => 'Дата',
        'amount' => 'Сумма',
        'status' => 'Статус',
        'no_payments' => 'История платежей не найдена.',
        'holiday_error' => 'Тренировки нельзя планировать в праздничные дни.',
        'trainer_label' => 'Тренер',
        'admin_name' => 'Администрация MyGoToGym',
        'verified_status' => 'Подтверждено',
        'verified_message' => 'Электронная почта успешно подтверждена! Теперь вы можете войти.',
        'email_change_verified' => 'Новый адрес электронной почты подтвержден.',
        'email_change_taken' => 'Этот адрес уже используется. Текущий адрес не изменен.',
```

```
        'return_to_profile' => 'Вернуться в профиль',
        'go_to_login' => 'Перейти к входу',
        'invalid_link_status' => 'Неверная ссылка',
        'invalid_link_message_generic' => 'Неверная ссылка для подтверждения.',
        'invalid_or_expired_token' => 'Неверный или просроченный токен подтверждения.',
        'try_registering_again' => 'Попробуйте зарегистрироваться снова',
        'back_to_home' => 'Вернуться на главную',
        'resend_verification' => 'Отправить письмо с подтверждением еще раз',
        'reset_link_sent' => 'Если этот адрес зарегистрирован и подтвержден, ссылка для сброса
отправлена.',
        'pass_reset_subject' => 'Сброс пароля вашего аккаунта MyGoToGym',
        'pass_reset_welcome' => 'Сброс пароля',
        'pass_reset_body' => 'Вы запросили сброс пароля. Нажмите кнопку ниже, чтобы установить новый
пароль.
            Ссылка действительна в течение 1 часа.',
        'pass_reset_button' => 'Сбросить пароль',
        'pass_reset_footer' => 'Если вы не запрашивали это, просто проигнорируйте письмо.',
        'password_mismatch' => 'Пароли не совпадают.',
        'password_min_length' => 'Пароль должен содержать не менее 8 символов.',
        'password_updated_success' => 'Пароль успешно обновлен!',
        'token_invalid' => 'Токен недействителен или просрочен.',
        'enter_email_to_reset' => 'Введите свой e-mail, чтобы получить ссылку для сброса.',
        'reset_cooldown' => 'Пожалуйста, подождите 5 минут перед повторным запросом.',
        'send_reset_link' => 'Отправить ссылку для сброса'
    ]
];
```

Fails **`api/Mailer.php`** – e-pasta sūtīšana ar PHPMailer:

```
<?php
namespace App;

use PHPMailer\PHPMailer\PHPMailer;
use PHPMailer\PHPMailer\Exception;
require_once __DIR__ . '/config.php';

class Mailer {
    public static function sendVerificationEmail($email, $name, $link, $lang = 'en') {
        $mail = new PHPMailer(true);
        try {
            // Localized Content
            $dict = [
                'en' => [
                    'subject' => 'Verify your MyGoToGym Account',
                    'welcome' => 'Welcome to MyGoToGym!',
                    'body'    => "Hello $name, please click the button below to verify your email
and activate
                       your account.",
                    'button' => 'Verify My Account',
                    'footer' => "If you didn't create an account, you can safely ignore this email."
                ],
                'lv' => [
                    'subject' => 'Apstipriniet savu MyGoToGym kontu',
                    'welcome' => 'Laipni lūdzam MyGoToGym!',
                    'body'    => "Sveiki $name, lūdzu, noklikšķiniet uz zemāk esošās pogas, lai
apstiprinātu savu
                       e-pastu un aktivizētu kontu.",
                    'button' => 'Apstiprināt manu kontu',
                    'footer' => "Ja neizveidojāt kontu, varat droši ignorēt šo e-pastu."
                ],
                'ru' => [
                    'subject' => 'Подтвердите свою учетную запись MyGoToGym',
                    'welcome' => 'Добро пожаловать в MyGoToGym!',
```

```
                    'body'    => "Здравствуйте, $name, пожалуйста, нажмите на кнопку ниже, чтобы
подтвердить свой
                        адрес электронной почты и активировать вашу учетную запись.",
                    'button' => 'Подтвердить мой аккаунт',
                    'footer' => "Если вы не создавали учетную запись, вы можете спокойно
игнорировать это
                        письмо."
                ]
            ];


            $content = $dict[$lang] ?? $dict['en'];

            $mail->isSMTP();
            $mail->Host       = SMTP_HOST;
            $mail->SMTPAuth   = true;
            $mail->Username   = SMTP_USERNAME;
            $mail->Password   = SMTP_PASSWORD;
            $mail->SMTPSecure = PHPMailer::ENCRYPTION_STARTTLS;
            $mail->Port       = SMTP_PORT;
            $mail->CharSet    = 'UTF-8'; // Ensure proper character encoding for international
characters


            $mail->setFrom(SMTP_USERNAME, 'MyGoToGym');
            $mail->addAddress($email, $name);

            $mail->isHTML(true);
            $mail->Subject = $content['subject'];


            $mail->Body    = "
                <div style='background: #0b0b0f; color: white; padding: 40px; font-family: sans-
serif; border-
                    radius: 20px; text-align: center;'>
                    <h1 style='color: #7c5cff;'>{$content['welcome']}</h1>
                    <p style='font-size: 1.1rem;'>{$content['body']}</p>
                    <br>
                    <a href='$link' style='background: linear-gradient(135deg, #7c5cff, #00d4ff);
color: white;
                        padding: 15px 30px; text-decoration: none; border-radius: 50px; font-weight:
bold; display: inline-
                        block;'>{$content['button']}</a>
                    <p style='margin-top: 30px; color: rgba(255,255,255,0.4); font-size:
0.8rem;'>{$content['footer']}</p>
                </div>
            ";

            // Plain text AltBody for email clients that don't support HTML
            $mail->AltBody = "{$content['welcome']}\n\n{$content['body']}\n\nLink:
$link\n\n{$content['footer']}";

            return $mail->send();
        } catch (Exception $e) {
            error_log("Mailer Error: {$mail->ErrorInfo}");
            return false;
        }
    }

    public static function sendEmailChangeVerification($email, $name, $link, $lang = 'en') {
        $contentByLanguage = [
            'en' => [
                'subject' => 'Confirm your new MyGoToGym email address',
```

```
                'heading' => 'Confirm your new email address',
                'body' => 'Hello %s, a request was made to change the email address on your
MyGoToGym account.
                    Confirm this new address to apply the change. Your current email will remain
active until you confirm.',
                'button' => 'Confirm email address',
                'footer' => 'If you did not request this change, you can ignore this email. Your
current address
                    will remain unchanged.'
            ],
            'lv' => [
                'subject' => 'Apstipriniet savu jauno MyGoToGym e-pasta adresi',
                'heading' => 'Apstipriniet jauno e-pasta adresi',
                'body' => 'Sveiki, %s! Tika pieprasīts mainīt jūsu MyGoToGym konta e-pasta adresi.
Apstipriniet
                    jauno adresi, lai veiktu izmaiņas. Pašreizējā adrese būs aktīva līdz
apstiprināšanai.',
                'button' => 'Apstiprināt e-pasta adresi',
                'footer' => 'Ja nepieprasījāt šīs izmaiņas, ignorējiet šo e-pastu. Pašreizējā adrese
netiks
                    mainīta.'
            ],
            'ru' => [
                'subject' => 'Подтвердите новый адрес MyGoToGym',
                'heading' => 'Подтвердите новый адрес электронной почты',
                'body' => 'Здравствуйте, %s! Поступил запрос на изменение адреса электронной почты
вашей учетной
                    записи MyGoToGym. Подтвердите новый адрес, чтобы применить изменение. Текущий
адрес останется активным до
                    подтверждения.',
                'button' => 'Подтвердить адрес',
                'footer' => 'Если вы не запрашивали это изменение, проигнорируйте письмо. Текущий
адрес останется
                    без изменений.'
            ]
        ];

        $content = $contentByLanguage[$lang] ?? $contentByLanguage['en'];
        $safeName = htmlspecialchars($name, ENT_QUOTES, 'UTF-8');
        $safeLink = htmlspecialchars($link, ENT_QUOTES, 'UTF-8');
        $body = sprintf($content['body'], $safeName);
        $plainBody = sprintf($content['body'], $name);
        $mail = new PHPMailer(true);

        try {
            $mail->isSMTP();
            $mail->Host = SMTP_HOST;
            $mail->SMTPAuth = true;
            $mail->Username = SMTP_USERNAME;
            $mail->Password = SMTP_PASSWORD;
            $mail->SMTPSecure = PHPMailer::ENCRYPTION_STARTTLS;
            $mail->Port = SMTP_PORT;
            $mail->CharSet = 'UTF-8';
            $mail->setFrom(SMTP_USERNAME, 'MyGoToGym');
            $mail->addAddress($email, $name);
            $mail->isHTML(true);
            $mail->Subject = $content['subject'];
            $mail->Body = "<div style='background:#0b0b0f;color:#fff;padding:40px;font-family:sans-
serif;border-
                radius:20px;text-align:center'><h1
style='color:#7c5cff'>{$content['heading']}</h1><p style='font-
                size:1.1rem'>{$body}</p><a href='{$safeLink}' style='background:linear-
```

```
gradient(135deg,#7c5cff,#00d4ff);color:#fff;padding:15px 30px;text-decoration:none;border-
radius:50px;font-
    weight:bold;display:inline-block'>{$content['button']}</a><p style='margin-
top:30px;color:rgba(255,255,255,.5);font-size:.85rem'>{$content['footer']}</p></div>";
            $mail->AltBody = $plainBody . "\n\n" . $link . "\n\n" . $content['footer'];

            return $mail->send();
        } catch (Exception $e) {
            error_log("Email change verification mail failed: {$mail->ErrorInfo}");
            return false;
        }
    }

    public static function sendResetEmail($email, $name, $link, $lang = 'en') {
        $mail = new PHPMailer(true);
        try {
            $dict = [
                'en' => [
                    'subject' => 'Reset your MyGoToGym Password',
                    'welcome' => 'Password Reset',
                    'body'    => 'You requested a password reset. Click the button below to set a
new password.
                        This link expires in 1 hour.',
                    'button' => 'Reset Password',
                    'footer' => 'If you did not request this, please ignore this email.'
                ],
                'lv' => [
                    'subject' => 'Atiestatiet savu MyGoToGym paroli',
                    'welcome' => 'Paroles atiestatīšana',
                    'body'    => 'Jūs pieprasījāt paroles atiestatīšanu. Noklikšķiniet uz pogas,
lai iestatītu
                        jaunu paroli. Šī saite ir derīga 1 stundu.',
                    'button' => 'Atiestatīt paroli',
                    'footer' => 'Ja jūs to nepieprasījāt, ignorējiet šo e-pastu.'
                ],
                'ru' => [
                    'subject' => 'Сброс пароля MyGoToGym',
                    'welcome' => 'Сброс пароля',
                    'body'    => 'Вы запросили сброс пароля. Нажмите кнопку ниже, чтобы установить
новый пароль.
                        Ссылка действительна 1 час.',
                    'button' => 'Сбросить пароль',
                    'footer' => 'Если вы не запрашивали это, просто проигнорируйте письмо.'
                ]
            ];

            $content = $dict[$lang] ?? $dict['en'];

            $mail->isSMTP();
            $mail->Host       = SMTP_HOST;
            $mail->SMTPAuth   = true;
            $mail->Username   = SMTP_USERNAME;
            $mail->Password   = SMTP_PASSWORD;
            $mail->SMTPSecure = PHPMailer::ENCRYPTION_STARTTLS;
            $mail->Port       = SMTP_PORT;
            $mail->CharSet    = 'UTF-8';

            $mail->setFrom(SMTP_USERNAME, 'MyGoToGym');
            $mail->addAddress($email, $name);

            $mail->isHTML(true);
            $mail->Subject = $content['subject'];
            $mail->Body    = "
```

```
                <div style='background: #0b0b0f; color: white; padding: 40px; font-family: sans-
serif; border-
                    radius: 20px; text-align: center;'>
                    <h1 style='color: #7c5cff;'>{$content['welcome']}</h1>
                    <p style='font-size: 1.1rem;'>{$content['body']}</p>
                    <br>
                    <a href='$link' style='background: linear-gradient(135deg, #7c5cff, #00d4ff);
color: white;
                        padding: 15px 30px; text-decoration: none; border-radius: 50px; font-weight:
bold; display: inline-
                        block;'>{$content['button']}</a>
                    <p style='margin-top: 30px; color: rgba(255,255,255,0.4); font-size:
0.8rem;'>{$content['footer']}</p>
                </div>";

            return $mail->send();
        } catch (Exception $e) {
            return false;
        }
    }
}
```

Fails **`api/nav.php`** – navigācijas josla un valodas izvēle:

```
<?php
require_once __DIR__ . '/db.php';

$username = '';
$role = '';
$membership_info = '';

if (isset($_SESSION['user_id'])) {
    $stmt = $conn->prepare("SELECT name, role, membership_type, membership_expiry FROM users WHERE
id=?");
    $stmt->bind_param("i", $_SESSION['user_id']);
    $stmt->execute();
    $u = $stmt->get_result()->fetch_assoc();

    if ($u) {
        $username = $u['name'];
        $role = $u['role'];

        if (!empty($u['membership_type']) && !empty($u['membership_expiry'])) {
            $expiry = strtotime($u['membership_expiry']);
            $remaining = $expiry - time();
            if ($remaining > 0) { // Only show if membership is active
                $days = ceil($remaining / (60 * 60 * 24)); // Calculate remaining days
                $membership_info = ' - ' . ucfirst($t[$u['membership_type']]) . " ($days " .
$t['days_left'] .
                    ")";
            }
        }
    }
}

$requestUri = parse_url($_SERVER['REQUEST_URI'], PHP_URL_PATH);
$currentPage = str_replace('.php', '', basename($requestUri));
$routeForLanguage = trim($requestUri, '/');
if ($basePath !== '' && strpos('/' . $routeForLanguage, $basePath . '/') === 0) {
    $routeForLanguage = ltrim(substr('/' . $routeForLanguage, strlen($basePath)), '/');
}
$routeForLanguage = preg_replace('#^(en|lv|ru)/#', '', $routeForLanguage);
if ($routeForLanguage === '' || $routeForLanguage === 'router') {
    $routeForLanguage = 'homepage';
```

```
}

// If the URL is just the folder path (like /abc/), default to 'homepage'
if ($currentPage == 'abc' || $currentPage == 'L' || $currentPage == '' || $currentPage == 'router')
{
    $currentPage = 'homepage';
}

// Preserve all existing URL parameters (like 'token') when switching languages
$queryParams = $_GET;
unset($queryParams['lang']); // Remove existing lang to prevent duplicates
$qStr = http_build_query($queryParams);
$langQuery = $qStr ? '?' . $qStr : '';
?>

<nav>
<div class="nav-left">
    <a href="<?= $localizedBase ?>/homepage" class="<?= $currentPage == 'homepage' ? 'active' : '' ?
>"><?=
$t['home'] ?></a>
    <a href="<?= $localizedBase ?>/index" class="<?= $currentPage == 'index' ? 'active' : '' ?>"><?=
$t['schedule'] ?></a>
    <a href="<?= $localizedBase ?>/contacts" class="<?= $currentPage == 'contacts' ? 'active' : '' ?
>"><?=
$t['contacts'] ?></a>
</div>

<div class="nav-center">
    <div class="lang-switcher">
        <button class="lang-btn" onclick="toggleLangMenu(event)">
            <?= $current_lang === 'en' ? 'EN' : ($current_lang === 'lv' ? 'LV' : 'RU') ?>
            <span style="font-size: 0.6rem; opacity: 0.5;">&#9660;</span>
        </button>
        <div class="lang-dropdown-content" id="langMenuDropdown">
            <a href="<?= $basePath ?>/en/<?= htmlspecialchars($routeForLanguage) ?><?= $langQuery ?
>">English</a>
            <a href="<?= $basePath ?>/lv/<?= htmlspecialchars($routeForLanguage) ?><?=
$langQuery ?>">Latviešu</a>
            <a href="<?= $basePath ?>/ru/<?= htmlspecialchars($routeForLanguage) ?><?= $langQuery ?
>">Русский</a>
        </div>
    </div>
</div>

<button class="hamburger" onclick="toggleMenu()">☰</button>
<div class="nav-right" id="navMenu">
<?php if(isset($_SESSION['user_id'])): ?>
    <?php if($username): // Ensure $username is not empty before displaying ?>
        <a href="<?= $localizedBase ?>/profile" class="nav-profile-link">
            <span><?= htmlspecialchars($username . $membership_info) ?></span>
            <span class="nav-role">(<?= ucfirst($role) ?>)</span>
        </a>
    <?php endif; ?>
    <?php if($role === 'admin'): ?>
        <a href="<?= $basePath ?>/admin/dashboard" class="<?= $currentPage == 'dashboard' ? 'active'
: '' ?>"><?=
$t['admin_panel'] ?></a>
    <?php endif; ?>
    <a href="<?= $basePath ?>/logout"><?= $t['logout'] ?></a>
<?php else: ?>
    <a href="<?= $localizedBase ?>/login" class="<?= in_array($currentPage, ['login',
'forgot_password',
        'reset_password']) ? 'active' : '' ?>"><?= $t['login'] ?></a>
```

```
    <a href="<?= $localizedBase ?>/register" class="<?= $currentPage == 'register' ? 'active' : '' ?
>"><?=
$t['register'] ?></a>
<?php endif; ?>
</div>
</nav>
<script>
function toggleMenu() {
    document.getElementById("navMenu").classList.toggle("show");
}

function toggleLangMenu(event) {
    event.stopPropagation();
    document.getElementById("langMenuDropdown").classList.toggle("show");
}

const languageScrollStorageKey = "language-switch-scroll-position";
const languageDropdown = document.getElementById("langMenuDropdown");

languageDropdown.querySelectorAll("a").forEach((link) => {
    link.addEventListener("click", (event) => {
        if (event.button !== 0 || event.ctrlKey || event.metaKey || event.shiftKey || event.altKey)
{
            return;
        }

            const target = new URL(link.href, window.location.href);
            sessionStorage.setItem(languageScrollStorageKey, JSON.stringify({
                pathname: target.pathname,
                search: target.search,
                scrollY: window.scrollY
            }));
      });
});

const savedLanguageScroll = sessionStorage.getItem(languageScrollStorageKey);
if (savedLanguageScroll) {
    sessionStorage.removeItem(languageScrollStorageKey);

      try {
          const position = JSON.parse(savedLanguageScroll);
          if (
              position.pathname === window.location.pathname &&
              position.search === window.location.search &&
              Number.isFinite(position.scrollY)
          ) {
              const restoreScroll = () => window.scrollTo(0, position.scrollY);
              if (document.readyState === "complete") {
                  requestAnimationFrame(restoreScroll);
              } else {
                window.addEventListener("load", restoreScroll, { once: true });
              }
        }
    } catch (error) {
        console.error("Unable to restore scroll position after language change.", error);
    }
}
</script>
```

Fails **`api/payment_success.php`** – maksājuma apstiprinājuma lapa:

```
<?php
require_once 'db.php';
```

```
if (!isset($_SESSION['user_id'])) {
    header("Location: " . $basePath . "/login");
    exit;
}
```

Fails **`api/resend_verification.php`** – verifikācijas e-pasta atkārtota nosūtīšana:

```
<?php
use PHPMailer\PHPMailer\PHPMailer;
use PHPMailer\PHPMailer\Exception;

require_once __DIR__ . '/../vendor/autoload.php';
header('Content-Type: application/json');
require_once 'config.php'; // Load SMTP constants
require 'db.php';
require_once __DIR__ . '/Mailer.php';

$data = json_decode(file_get_contents("php://input"), true);
$email = strtolower(trim($data['email'] ?? ''));

if (!$email) {
    echo json_encode(['success' => false, 'message' => 'Email is required.']);
    exit;
}

$stmt = $conn->prepare("SELECT id, name, verification_token, email_verified FROM users WHERE email =
?");
$stmt->bind_param("s", $email);
$stmt->execute();
$user = $stmt->get_result()->fetch_assoc();

if (!$user) {
    echo json_encode(['success' => false, 'message' => 'User not found.']);
    exit;
}

if ($user['email_verified']) {
    echo json_encode(['success' => false, 'message' => 'Email is already verified.']);
    exit;
}

$name = $user['name'];
$token = $user['verification_token'];

if (!$token) {
    $token = bin2hex(random_bytes(32));
    $upd = $conn->prepare("UPDATE users SET verification_token = ? WHERE id = ?");
    $upd->bind_param("si", $token, $user['id']);
    $upd->execute();
}

$protocol = isset($_SERVER['HTTPS']) && $_SERVER['HTTPS'] === 'on' ? "https" : "http";
$host = $_SERVER['HTTP_HOST'];
$verify_link = "$protocol://$host" . $basePath . "/verfy_email?token=$token&lang=$current_lang"; //
Use $basePath
    and clean URL

try {

    if (\App\Mailer::sendVerificationEmail($email, $name, $verify_link, $current_lang)) {
        echo json_encode(['success' => true, 'message' => 'Verification email resent
successfully!']);
    } elseif (LOCAL_VERIFICATION_FALLBACK) {
        echo json_encode([
```

```
            'success' => true,
            'message' => 'Email delivery is unavailable locally. Use the local verification link.',
            'verification_link' => $verify_link
        ]);
    } else {

        echo json_encode(['success' => false, 'message' => 'Failed to resend verification email.']);
    }

} catch (Exception $e) {

    error_log("Resend Mailer Error: " . $e->getMessage());

    echo json_encode(['success' => false, 'message' => "An unexpected error occurred while sending
the email."]);
}
```

Fails **`api/reserve_class.php`** – nodarbības rezervēšana (datubāzes transakcija ar bloķēšanu):

```
<?php
header('Content-Type: application/json');
include 'db.php';
require_once 'csrf.php';

if(!isset($_SESSION['user_id'])){
    die(json_encode(["success"=>false,"message"=>"You must be logged in to reserve."]));
}

$data = json_decode(file_get_contents("php://input"), true);
if ($_SERVER['REQUEST_METHOD'] !== 'POST' || !is_array($data) || !check_csrf($data['csrf_token'] ??
'')) {
    http_response_code(403);
    die(json_encode(["success" => false, "message" => "Invalid security token"]));
}
$class_id = filter_var($data['class_id'] ?? null, FILTER_VALIDATE_INT, ['options' => ['min_range' =>
1]]) ?: 0;
if (!$class_id) {
    die(json_encode(["success" => false, "message" => "Invalid class"]));
}
$user_id = $_SESSION['user_id'];

$conn->begin_transaction();
try {


$stmt_check = $conn->prepare("SELECT start_datetime, capacity FROM classes WHERE id = ? FOR
UPDATE");
$stmt_check->bind_param("i", $class_id);
$stmt_check->execute();
$class_row = $stmt_check->get_result()->fetch_assoc();
if(!$class_row || strtotime($class_row['start_datetime']) < time()){
    $conn->rollback();
    die(json_encode(["success" => false, "message" => "This workout has already started or
passed."]));
}

$mem_stmt = $conn->prepare("SELECT membership_expiry, membership_type FROM users WHERE id = ?");
$mem_stmt->bind_param("i", $user_id);
$mem_stmt->execute();
$user_mem = $mem_stmt->get_result()->fetch_assoc();

if (!$user_mem || empty($user_mem['membership_expiry']) || strtotime($user_mem['membership_expiry'])
< time()) {
```

```
    $conn->rollback();
    die(json_encode(["success" => false, "message" => "An active membership is required to reserve
classes."]));
}

if (!in_array($user_mem['membership_type'], ['advanced', 'pro'])) {
    $conn->rollback();
    die(json_encode(["success" => false, "message" => "Your current plan does not include class
reservations.
        Please upgrade to Advanced or Pro."]));
}


$check = $conn->prepare("
SELECT id FROM reservations
WHERE user_id=? AND class_id=?
");
$check->bind_param("ii", $user_id, $class_id);
$check->execute();

if($check->get_result()->num_rows > 0){
    $conn->rollback();
    die(json_encode([
"success"=>false,
"message"=>"You already reserved this workout"
]));
}

$cap_stmt = $conn->prepare("
SELECT c.capacity, COUNT(r.id) AS booked
FROM classes c
LEFT JOIN reservations r ON c.id = r.class_id
WHERE c.id = ?
GROUP BY c.id
");
$cap_stmt->bind_param("i", $class_id);
$cap_stmt->execute();
$cap = $cap_stmt->get_result()->fetch_assoc();

if(!$cap || $cap['booked'] >= $cap['capacity']){
    $conn->rollback();
die(json_encode([
"success"=>false,
"message"=>"Class is full"
]));
}

$stmt = $conn->prepare("
INSERT INTO reservations(class_id,user_id)
VALUES(?,?)
");

$stmt->bind_param("ii",$class_id,$user_id);

if ($stmt->execute()) {
    $conn->commit();
    echo json_encode([
        "success"=>true,
        "message"=>"Reserved successfully",
    ]);
} else {
    $conn->rollback();
    echo json_encode([
```

```
        "success"=>false,
        "message"=>"Reservation failed."
    ]);
}
} catch (mysqli_sql_exception $e) {
    $conn->rollback();
    error_log('Reservation failed: ' . $e->getMessage());
    echo json_encode(["success" => false, "message" => "Reservation failed."]);
}
```

Fails **`api/save_class.php`** – nodarbības izveide un rediģēšana:

```
<?php
header('Content-Type: application/json');
include 'db.php';
require_once 'csrf.php';

// Check if user is logged in and has the correct role
if ($_SERVER['REQUEST_METHOD'] !== 'POST' || !isset($_SESSION['user_id']) || !
in_array($_SESSION['role'] ?? '',
    ['admin', 'trainer'], true)) {
    die(json_encode(["success"=>false,"message"=>"Unauthorized access"]));
}

$data = json_decode(file_get_contents("php://input"), true);
if (!is_array($data) || !check_csrf($data['csrf_token'] ?? '')) {
    http_response_code(403);
    die(json_encode(["success" => false, "message" => "Invalid security token"]));
}

$roleStmt = $conn->prepare('SELECT role FROM users WHERE id = ?');
$roleStmt->bind_param('i', $_SESSION['user_id']);
$roleStmt->execute();
$currentRole = $roleStmt->get_result()->fetch_assoc()['role'] ?? '';
if (!in_array($currentRole, ['admin', 'trainer'], true)) {
    http_response_code(403);
    die(json_encode(["success" => false, "message" => "Unauthorized access"]));
}
$_SESSION['role'] = $currentRole;

$id = filter_var($data['id'] ?? null, FILTER_VALIDATE_INT, ['options' => ['min_range' => 1]]) ?: 0;
$title = trim((string) ($data['title'] ?? ''));
$description = trim((string) ($data['description'] ?? ''));
$start = (string) ($data['start_datetime'] ?? '');
$capacity = filter_var($data['capacity'] ?? null, FILTER_VALIDATE_INT, ['options' => ['min_range' =>
1,
    'max_range' => 1000]]);
$color = (string) ($data['color'] ?? 'Blue');
$validColors = ['Blue', 'Green', 'Red', 'Orange', 'Purple', 'Teal'];

if ($title === '' || mb_strlen($title) > 150 || mb_strlen($description) > 10000 || $capacity ===
false
    || !in_array($color, $validColors, true)
    || !DateTime::createFromFormat('Y-m-d\\TH:i', $start)) {
    die(json_encode(["success" => false, "message" => "Invalid class details."]));
}

$check_ts = strtotime($start);
$year = (int)date('Y', $check_ts);
$md = date('m-d', $check_ts);
$full_date = date('Y-m-d', $check_ts);

$fixed_holidays = [
    '01-01', // New Year
```

```
    '05-01', // Labor Day
    '05-04', // Restoration of Independence
    '06-23', // Līgo
    '06-24', // Jāņi
    '11-18', // Proclamation Day
    '12-24', // Christmas Eve
    '12-25', // Christmas
    '12-26', // 2nd Christmas
    '12-31', // New Year's Eve
];

$is_holiday = in_array($md, $fixed_holidays);

if (!$is_holiday) {
    // Variable date holidays (Easter)
    $easter_ts = easter_date($year);
    $good_friday = date('Y-m-d', strtotime('-2 days', $easter_ts));
    $easter_sunday = date('Y-m-d', $easter_ts);
    $easter_monday = date('Y-m-d', strtotime('+1 day', $easter_ts));
    if (in_array($full_date, [$good_friday, $easter_sunday, $easter_monday])) {
        $is_holiday = true;
    }
}

if ($is_holiday) {
    die(json_encode(["success" => false, "message" => $t['holiday_error']]));
}

try {
    if ($id) {
        if ($currentRole === 'trainer') {
            $owner = $conn->prepare('SELECT id FROM classes WHERE id = ? AND created_by = ?');
            $owner->bind_param('ii', $id, $_SESSION['user_id']);
            $owner->execute();
            if ($owner->get_result()->num_rows === 0) {
                http_response_code(403);
                exit(json_encode(["success" => false, "message" => "You can only edit your own
classes."]));
            }
        }
        $conn->begin_transaction();
        $lock = $conn->prepare('SELECT id FROM classes WHERE id = ? FOR UPDATE');
        $lock->bind_param('i', $id); $lock->execute();
        if ($lock->get_result()->num_rows === 0) { throw new RuntimeException('Class not found.'); }
        $count = $conn->prepare("SELECT COUNT(*) AS total FROM reservations WHERE class_id = ? AND
status =
            'reserved'");
        $count->bind_param('i', $id); $count->execute();
        $booked = (int) $count->get_result()->fetch_assoc()['total'];
        if ($capacity < $booked) { throw new RuntimeException("Capacity cannot be lower than the
$booked existing
            reservation(s)."); }
        $stmt = $conn->prepare('UPDATE classes SET title=?, description=?, start_datetime=?,
capacity=?, color=?
            WHERE id=?');
        $stmt->bind_param('sssisi', $title, $description, $start, $capacity, $color, $id);
    } else {
        $stmt = $conn->prepare('INSERT INTO
classes(title,description,start_datetime,capacity,color,created_by)
            VALUES(?,?,?,?,?,?)');
        $stmt->bind_param('sssisi', $title, $description, $start, $capacity, $color,
$_SESSION['user_id']);
    }
```

```
    $stmt->execute();
    if ($id) { $conn->commit(); }
    echo json_encode(["success" => true]);
} catch (Throwable $e) {
    if ($id && $conn->errno === 0) { $conn->rollback(); }
    error_log('Save class failed: ' . $e->getMessage());
    echo json_encode(["success" => false, "message" => $e instanceof RuntimeException ? $e-
>getMessage() :
        'Unable to save the class.']);
}
```

Fails **`api/session.php`** – drošas sesijas palaišana:

```
<?php

function start_app_session(): void
{
    if (session_status() !== PHP_SESSION_NONE) {
        return;
    }

    $tempDirectory = getenv('TEMP') ?: sys_get_temp_dir();
    $sessionPath = rtrim($tempDirectory, "\\/") . DIRECTORY_SEPARATOR . 'gym_tracker2_sessions';
    if (!is_dir($sessionPath) && !mkdir($sessionPath, 0770, true) && !is_dir($sessionPath)) {
        error_log('Unable to create session storage directory.');
        http_response_code(500);
        exit('Session storage is unavailable.');
    }

    session_save_path($sessionPath);
    $secure = (!empty($_SERVER['HTTPS']) && $_SERVER['HTTPS'] !== 'off') || ($_SERVER['SERVER_PORT']
?? '') ===
        '443';
    session_set_cookie_params([
        'lifetime' => 0,
        'path' => '/',
        'secure' => $secure,
        'httponly' => true,
        'samesite' => 'Lax'
    ]);
    session_start();
}
```

Fails **`api/track_analytics.php`** – apmeklējuma statistikas uzkrāšana:

```
<?php
include 'db.php';


$data = json_decode(file_get_contents("php://input"), true);
$action = $data['action'] ?? 'visit';
$page_path = $data['page_path'] ?? null;

if (isset($_COOKIE['cookie_consent']) && $_COOKIE['cookie_consent'] === 'accepted') {
    $user_id = $_SESSION['user_id'] ?? null;

    $stmt = $conn->prepare("INSERT INTO site_analytics (user_id, action, page_path) VALUES
(?, ?, ?)");
    $stmt->bind_param("iss", $user_id, $action, $page_path);

    if ($stmt->execute()) {
        echo json_encode(['success' => true]);
    } else {
        echo json_encode(['success' => false]);
```

```
    }
} else {
    echo json_encode(['success' => false, 'message' => 'No consent']);
}
```

Fails **`api/update_profile.php`** – profila datu atjaunināšana:

```
<?php
header('Content-Type: application/json');
include 'db.php';
require_once 'csrf.php';
require_once __DIR__ . '/../vendor/autoload.php';

if (!isset($_SESSION['user_id'])) {
    echo json_encode(['success' => false, 'message' => 'Unauthorized']);
    exit;
}

$data = json_decode(file_get_contents("php://input"), true);
$action = $_GET['action'] ?? '';

if ($_SERVER['REQUEST_METHOD'] !== 'POST' || !is_array($data) || !check_csrf($data['csrf_token'] ??
'')) {
    echo json_encode(['success' => false, 'message' => 'Invalid CSRF token']);
    exit;
}

$user_id = $_SESSION['user_id'];

if ($action === 'info') {
    if (array_key_exists('name', $data)) {
        http_response_code(400);
        echo json_encode(['success' => false, 'message' => 'Name cannot be changed from profile
settings']);
        exit;
    }

    $email = strtolower(trim($data['email'] ?? ''));
    $phone = trim($data['phone'] ?? '');

    if (!filter_var($email, FILTER_VALIDATE_EMAIL) || mb_strlen($email) > 150) {
        echo json_encode(['success' => false, 'message' => 'Invalid email']);
        exit;
    }

    if (strlen($phone) > 100) {
        http_response_code(422);
        echo json_encode(['success' => false, 'message' => $t['invalid_phone']]);
        exit;
    }

    $phoneUtil = \libphonenumber\PhoneNumberUtil::getInstance();
    if ($phone !== '') {
        if ($phone[0] !== '+') {
            http_response_code(422);
            echo json_encode(['success' => false, 'message' => $t['invalid_phone']]);
            exit;
        }

        try {
            $parsedPhone = $phoneUtil->parse($phone, null);
            if (!$phoneUtil->isValidNumber($parsedPhone)) {
                throw new \libphonenumber\NumberParseException(0, 'Invalid phone number');
            }
```

```
            $phone = $phoneUtil->format($parsedPhone, \libphonenumber\PhoneNumberFormat::E164);
        } catch (\libphonenumber\NumberParseException $e) {
            http_response_code(422);
            echo json_encode(['success' => false, 'message' => $t['invalid_phone']]);
            exit;
        }
    }

    $stmt = $conn->prepare("SELECT name, email, email_verified, phone FROM users WHERE id = ?");
    $stmt->bind_param("i", $user_id);
    $stmt->execute();
    $profile = $stmt->get_result()->fetch_assoc();
    if (!$profile || empty($profile['email_verified'])) {
        http_response_code(403);
        echo json_encode(['success' => false, 'message' => 'Verified account required']);
        exit;
    }

    $currentEmail = strtolower($profile['email']);
    $currentPhone = trim((string) ($profile['phone'] ?? ''));
    if ($currentPhone !== '' && $currentPhone[0] === '+') {
        try {
            $currentParsedPhone = $phoneUtil->parse($currentPhone, null);
            if ($phoneUtil->isValidNumber($currentParsedPhone)) {
                $currentPhone = $phoneUtil->format($currentParsedPhone, \libphonenumber\
PhoneNumberFormat::E164);
            }
        } catch (\libphonenumber\NumberParseException $e) {

        }
    }
    $phoneChanged = $phone !== $currentPhone;
    if ($email !== $currentEmail) {
        $stmt = $conn->prepare("SELECT id FROM users WHERE email = ? AND id != ?");
        $stmt->bind_param("si", $email, $user_id);
        $stmt->execute();
        if ($stmt->get_result()->num_rows > 0) {
            http_response_code(409);
            echo json_encode(['success' => false, 'message' => $t['email_already_in_use']]);
            exit;
        }
    }

    if ($email === $currentEmail) {
        if (!$phoneChanged) {
            echo json_encode(['success' => true, 'message' => '']);
            exit;
        }

        $stmt = $conn->prepare("UPDATE users SET phone = ? WHERE id = ?");
        $stmt->bind_param("si", $phone, $user_id);
        echo json_encode($stmt->execute()
            ? ['success' => true, 'message' => $t['phone_updated']]
            : ['success' => false, 'message' => 'Update failed']);
        exit;
    }

    $verificationToken = bin2hex(random_bytes(32));
    if ($phoneChanged) {
        $stmt = $conn->prepare("UPDATE users SET phone = ?, pending_email = ?, verification_token
= ?,
            verification_expires = DATE_ADD(NOW(), INTERVAL 1 DAY) WHERE id = ?");
        $stmt->bind_param("sssi", $phone, $email, $verificationToken, $user_id);
```

```
    } else {
        $stmt = $conn->prepare("UPDATE users SET pending_email = ?, verification_token = ?,
verification_expires
            = DATE_ADD(NOW(), INTERVAL 1 DAY) WHERE id = ?");
        $stmt->bind_param("ssi", $email, $verificationToken, $user_id);
    }
    if (!$stmt->execute()) {
        echo json_encode(['success' => false, 'message' => 'Update failed']);
        exit;
    }

    $protocol = (!empty($_SERVER['HTTPS']) && $_SERVER['HTTPS'] !== 'off') ? 'https' : 'http';
    $host = $_SERVER['HTTP_HOST'] ?? '';
    $verificationLink = $protocol . '://' . $host . $basePath . '/verfy_email?token=' .
urlencode($verificationToken) . '&lang=' . urlencode($current_lang);

    require_once __DIR__ . '/../vendor/autoload.php';
    require_once __DIR__ . '/Mailer.php';
    if (\App\Mailer::sendEmailChangeVerification($email, $profile['name'], $verificationLink,
$current_lang)) {
        $message = $phoneChanged ? $t['phone_and_email_change_confirmation_sent'] :
$t['email_change_confirmation_sent'];
        echo json_encode(['success' => true, 'message' => $message]);
    } elseif (LOCAL_VERIFICATION_FALLBACK) {
        echo json_encode([
            'success' => true,
            'message' => $phoneChanged ? $t['phone_updated_email_change_confirmation_failed'] :
$t['email_change_confirmation_failed'],
            'verification_link' => $verificationLink,
            'verification_link_label' => $t['local_email_verification_link']
        ]);
    } else {
        $stmt = $conn->prepare("UPDATE users SET pending_email = NULL, verification_token = NULL,
            verification_expires = NULL WHERE id = ? AND verification_token = ?");
        $stmt->bind_param("is", $user_id, $verificationToken);
        $stmt->execute();
        $message = $phoneChanged ? $t['phone_updated_email_change_confirmation_failed'] :
$t['email_change_confirmation_failed'];
        echo json_encode(['success' => false, 'message' => $message]);
    }

} elseif ($action === 'password') {
    $current = $data['current_password'] ?? '';
    $new = $data['new_password'] ?? '';

    if (!preg_match('/^(?=.*[A-Z])(?=.*\d)(?=.*[\W_]).{8,}$/', $new)) {
        echo json_encode(['success' => false, 'message' => 'New password does not meet
requirements.']);
        exit;
    }

    $stmt = $conn->prepare("SELECT password FROM users WHERE id = ?");
    $stmt->bind_param("i", $user_id);
    $stmt->execute();
    $user = $stmt->get_result()->fetch_assoc();

    if (!password_verify($current, $user['password'])) {
        echo json_encode(['success' => false, 'message' => 'Current password incorrect']);
        exit;
    }

    $hashed = password_hash($new, PASSWORD_DEFAULT);
    $stmt = $conn->prepare("UPDATE users SET password = ? WHERE id = ?");
```

```
    $stmt->bind_param("si", $hashed, $user_id);

    if ($stmt->execute()) {
        echo json_encode(['success' => true, 'message' => 'Password changed successfully']);
    } else {
        echo json_encode(['success' => false, 'message' => 'Password update failed']);
    }
} else {
    echo json_encode(['success' => false, 'message' => 'Invalid action']);
}
```

Fails **`assets/js/calendar.js`** – nodarbību kalendārs, formu validācija un paziņojumi (JavaScript):

```
document.addEventListener('DOMContentLoaded', () => {

function escapeHtml(value) {
    return String(value ?? '').replace(/&/g, '&amp;').replace(/</g, '&lt;').replace(/>/g,
'&gt;').replace(/"/g,
        '&quot;').replace(/'/g, '&#039;');
}

const calendarEl = document.getElementById('calendar');
const modal = document.getElementById('workoutModal');

const canManageClasses = window.USER_ROLE === "admin" || window.USER_ROLE === "trainer";
const userId = parseInt(window.USER_ID || 0);

const titleInput = document.getElementById('classTitle');
const descInput = document.getElementById('classDesc');
const dateInput = document.getElementById('classDate');
const capInput = document.getElementById('classCap');
const colorInput = document.getElementById('classColor');
const classIdInput = document.getElementById('classId');

const saveBtn = document.getElementById('saveClassBtn');
const deleteBtn = document.getElementById('deleteClassBtn');
const closeBtn = document.getElementById('closeModal');

let selectedEvent = null;

function formatMonthYearCustom(date, locale) {

    const bcp47Locale = {
        'en': 'en-US', // Or 'en-GB', etc.
        'lv': 'lv-LV',
        'ru': 'ru-RU'
    }[locale] || locale; // Fallback to original if not mapped


    let monthName = new Intl.DateTimeFormat(bcp47Locale, { month: 'long' }).format(date);


    let year = new Intl.DateTimeFormat(bcp47Locale, { year: 'numeric' }).format(date);


    if (locale === 'lv') {
        year = year.replace(/\.\s*g\.?$/i, '').trim(); // Remove "g." or "g"
    } else if (locale === 'ru') {
        year = year.replace(/\s*r\.?$/i, '').trim(); // Remove "r." or "r"
    }


    monthName = monthName.charAt(0).toUpperCase() + monthName.slice(1);
```

```
    return `${monthName} ${year}`;
}

if (!document.getElementById('modalUserContent')) {
    modal.insertAdjacentHTML('beforeend', '<div id="modalUserContent" class="modal-content"
style="display:none;"></div>');
}

function openModal(){
    modal.style.display = "flex";
}

function closeModal(){
    modal.style.display = "none";
}

closeBtn.onclick = closeModal;

const colorPicker = document.getElementById('colorPicker');
const colorOptions = document.getElementById('colorOptions');
const selectedDot = document.getElementById('selectedDot');
const selectedLabel = document.getElementById('selectedLabel');

const colorMap = {
    'Blue': '#7c5cff',
    'Green': '#28a745',
    'Red': '#dc3545',
    'Orange': '#fd7e14',
    'Purple': '#6f42c1',
    'Teal': '#20c997'
};

function updateColorPicker(name) {
    // Default to Blue if name is missing or invalid
    const validName = colorMap[name] ? name : "Blue";
    colorInput.value = validName;
    selectedDot.style.background = colorMap[validName];
    selectedLabel.textContent = validName;
}

if(colorPicker) {
    colorPicker.onclick = (e) => {
        e.stopPropagation();
        colorOptions.style.display = colorOptions.style.display === 'block' ? 'none' : 'block';
    };
}

document.querySelectorAll('.dropdown-option').forEach(opt => {
    opt.onclick = (e) => {
        e.stopPropagation();
        updateColorPicker(opt.dataset.value);
        colorOptions.style.display = 'none';
    };
});

window.addEventListener('click', () => { if(colorOptions) colorOptions.style.display = 'none'; });

window.calendar = new FullCalendar.Calendar(calendarEl, {

locale: window.I18N.locale,
firstDay: 1,
```

```
initialView: 'dayGridMonth',
dayHeaderFormat: {
    weekday: 'short'
},
dayHeaderContent: function(arg) {

    if (window.I18N.locale === 'ru') {
        return arg.text.charAt(0).toUpperCase() + arg.text.slice(1);
    }


},
initialLocale: window.I18N.locale,
headerToolbar: false,
height: 'auto',

dayCellClassNames: function(arg) {
    const m = arg.date.getMonth() + 1;
    const d = arg.date.getDate();
    const holidays = ['1-1', '5-1', '5-4', '6-23', '6-24', '11-18', '12-24', '12-25', '12-26', '12-
31'];
    if (holidays.includes(`${m}-${d}`)) return ['fc-day-holiday'];
    return [];
},

eventSources: [
    {
        url: 'api/get_classes'
    },
    {
        events: function(fetchInfo, successCallback) {
             const year = fetchInfo.start.getFullYear();
             const holidayList = [
                 { title: 'Jaunais gads', start: `${year}-01-01` },
                 { title: 'Darba svētki', start: `${year}-05-01` },
                 { title: 'Neatkarības atjaunošana', start: `${year}-05-04` },
                 { title: 'Līgo', start: `${year}-06-23` },
                 { title: 'Jāņi', start: `${year}-06-24` },
                 { title: 'LR proklamēšanas diena', start: `${year}-11-18` },
                 { title: 'Ziemassvētku vakars', start: `${year}-12-24` },
                 { title: 'Ziemassvētki', start: `${year}-12-25` },
                 { title: 'Otrie Ziemassvētki', start: `${year}-12-26` },
                 { title: 'Vecgada vakars', start: `${year}-12-31` }
             ];


             const nextYear = year + 1;
             holidayList.push(
                 { title: 'Jaunais gads', start: `${nextYear}-01-01` },
                 { title: 'Latvijas neatkarība', start: `${nextYear}-05-04` }
             );

             successCallback(holidayList.map(h => ({
                 ...h,
                 allDay: true,
                 display: 'list-item', // Shows as a small text label
                 color: '#ff3b3b',
                 interactive: false, // Users can't click holidays
                 extendedProps: { isHoliday: true }
             })));
         }
     }
],
```

```
eventSourceFailure: function(error) {
    console.error('Calendar events failed to load:', error);
    const infoBox = document.getElementById("classInfo");
    if (infoBox) {
        infoBox.innerHTML = '<p class="error">Calendar classes could not be loaded. Please check the
database
            tables.</p>';
    }
},

eventDataTransform: function(eventData) {

     if (eventData.extendedProps && eventData.extendedProps.isHoliday) {
         return eventData;
     }

     const now = new Date();
     const eventDate = new Date(eventData.start);


     if (eventDate < now) {
         // Using specific grey shades for past events
         eventData.color = '#555555';       // Changes the dot color
         eventData.textColor = '#888888';   // Fades the title text
         eventData.backgroundColor = 'transparent';
         eventData.borderColor = '#444444';
     }

     return eventData;
},

eventTimeFormat: {
    hour: '2-digit',
    minute: '2-digit',
    hour12: false
},

dateClick(info){

     if(!canManageClasses) return;
     const date = new Date(info.dateStr);
     const m = date.getMonth() + 1;
     const d = date.getDate();
     const holidays = ['1-1', '5-1', '5-4', '6-23', '6-24', '11-18', '12-24', '12-25', '12-26', '12-
31'];
     if (holidays.includes(`${m}-${d}`)) return;

     classIdInput.value = "";
     titleInput.value = "";
     descInput.value = "";
     capInput.value = 10;
     updateColorPicker("Blue");
     dateInput.value = info.dateStr + "T18:00";

     document.querySelector('.modal-content').style.display = "block";
     document.getElementById('modalUserContent').style.display = "none";
     deleteBtn.style.display = "none";

     openModal();
},

eventClick(info){
```

```
      const event = info.event;


      if (typeof window.updateSidebar === 'function') {
          window.updateSidebar(event);
      }




      const id = event.id;
      const title = event.title;
      const desc = event.extendedProps.description;
      const cap = event.extendedProps.capacity;
      const booked = event.extendedProps.booked;



      const colorName = event.extendedProps.original_color || "Blue";

      selectedEvent = id;


      if(canManageClasses){

          classIdInput.value = id;
          titleInput.value = title;
          descInput.value = desc;
          capInput.value = cap;
          updateColorPicker(colorName);
          dateInput.value = event.startStr.slice(0,16);

          document.querySelector('.modal-content').style.display = "block";
          document.getElementById('modalUserContent').style.display = "none";
          deleteBtn.style.display = "inline-block";

          openModal();
          return;
      }


      let remaining = cap - booked;
      const isFull = booked >= cap;
      const isPast = event.start < new Date();
      const userContainer = document.getElementById('modalUserContent');

      userContainer.innerHTML = `
          <h3>${escapeHtml(title)}</h3>
         <p>${escapeHtml(desc)}</p>
         <p><b>${window.I18N.attendance}:</b> ${booked}/${cap}</p>
         <p><b>${window.I18N.remaining}:</b> ${remaining}</p>
         <div class="modal-actions">
             <button id="reserveBtn" ${(isFull || isPast) ? 'disabled style="background: #333; color:
#777;
                 cursor: not-allowed; border: 1px solid #444;"' : ''}>
                 ${isPast ? 'Passed' : (isFull ? window.I18N.classFull : window.I18N.reserve)}
             </button>
             <button id="closeUser" style="background: #2b2b2b; color: white;">$
{window.I18N.cancel}</button>
     </div>`;

       document.querySelector('.modal-content').style.display = "none";
```

```
      userContainer.style.display = "block";
      openModal();

      document.getElementById('closeUser').onclick = closeModal;

      document.getElementById('reserveBtn').onclick = () => {

           fetch('api/reserve_class', {
               method: 'POST',
               headers: {'Content-Type':'application/json'},
               body: JSON.stringify({
                   class_id: id,
                   csrf_token: window.CSRF_TOKEN
               })
           })
           .then(r => r.json())
           .then(data => {

                 alert(data.message);

                 if(data.success){
                     closeModal();

                     // Optimistically update the booked count for the sidebar
                     const currentBooked = parseInt(event.extendedProps.booked || 0);
                     event.setExtendedProp('booked', currentBooked + 1);

                     // Trigger sidebar update immediately
                     if (typeof window.updateSidebar === 'function') {
                         window.updateSidebar(event);
                     }

                     calendar.refetchEvents();
                 }

           });

      };

},

datesSet(info){
    const viewDate = info.view.currentStart; // Get the date for the current view (e.g., the first
day of the
        month)
    const currentLocale = window.I18N.locale; // Assuming window.I18N.locale is set to 'en', 'lv',
or 'ru'
    document.getElementById('calendarTitle').textContent = formatMonthYearCustom(viewDate,
currentLocale);
}

});

calendar.render();

document.getElementById('prevBtn').onclick = () => calendar.prev();
document.getElementById('nextBtn').onclick = () => calendar.next();
document.getElementById('todayBtn').onclick = () => calendar.today();

saveBtn.onclick = () => {

fetch('api/save_class', {
method:'POST',
```

```
headers:{'Content-Type':'application/json'},
body: JSON.stringify({
id: classIdInput.value,
title: titleInput.value,
description: descInput.value,
start_datetime: dateInput.value,
capacity: capInput.value,
color: colorInput.value
,
csrf_token: window.CSRF_TOKEN
})
})
.then(r => r.json())
.then(data => {

if(data.success){
closeModal();
calendar.refetchEvents();
}else{
alert(data.message || window.I18N.saveFailed);
}

});

};

deleteBtn.onclick = () => {

if(!confirm(window.I18N.confirmDelete)) return;

fetch('api/delete_class', {
method:'POST',
headers:{'Content-Type':'application/json'},
body: JSON.stringify({id: classIdInput.value, csrf_token: window.CSRF_TOKEN})
})
.then(r => r.json())
.then(data => {

if(data.success){
closeModal();
calendar.refetchEvents();
}

});

};

});
```

Fails **`export/export_class_participants.php`** – dalībnieku saraksta eksports Excel formātā:

```
<?php
require_once __DIR__ . '/../api/db.php';
require_once __DIR__ . '/../vendor/autoload.php';
require_once __DIR__ . '/export_helpers.php';

use OpenSpout\Common\Entity\Row;
use OpenSpout\Common\Entity\Style\Color;
use OpenSpout\Common\Entity\Style\Style;
use OpenSpout\Writer\XLSX\Writer;
use OpenSpout\Writer\XLSX\Options;

export_require_admin($basePath);
```

```
if (!isset($_GET['class_id']) || !ctype_digit((string) $_GET['class_id'])) {
    http_response_code(422);
    exit('Class ID is required.');
}

$class_id = intval($_GET['class_id']);

// Fetch class title for filename
$stmt = $conn->prepare("SELECT title FROM classes WHERE id = ?");
$stmt->bind_param("i", $class_id);
$stmt->execute();
$class = $stmt->get_result()->fetch_assoc();
$title = $class['title'] ?? 'workout';
$filename = preg_replace('/[^a-z0-9_-]+/i', '_', $title) . "_participants_" . date('Y-m-d') .
".xlsx";

$stmt = $conn->prepare("
    SELECT u.name, u.email, u.phone
    FROM users u
    JOIN reservations r ON u.id = r.user_id
    WHERE r.class_id = ? AND r.status = 'reserved'
");
$stmt->bind_param("i", $class_id);
$stmt->execute();
$result = $stmt->get_result();

$options = new Options();
$options->DEFAULT_ROW_STYLE->setFontSize(9);
$options->DEFAULT_ROW_HEIGHT = 14;
$options->setColumnWidth(7, 1);
$options->setColumnWidth(24, 2);
$options->setColumnWidth(32, 3);
$options->setColumnWidth(18, 4);
$writer = new Writer($options);
$writer->openToBrowser($filename);
$titleStyle = (new Style())->setFontBold()->setFontSize(14)->setFontColor(Color::WHITE)-
>setBackgroundColor(Color::DARK_BLUE);
$metaStyle = (new Style())->setFontSize(10);
$headerStyle = (new Style())->setFontBold()->setFontSize(9)->setFontColor(Color::WHITE)-
>setBackgroundColor(Color::BLUE);
$writer->addRow(Row::fromValues(['MyGoToGym Participant List', $title], $titleStyle));
$writer->addRow(Row::fromValues(['Generated', date('Y-m-d H:i')], $metaStyle));
$writer->addRow(Row::fromValues([]));
$writer->addRow(Row::fromValues([]));
$writer->addRow(Row::fromValues(['No.', 'Name', 'Email address', 'Phone number'], $headerStyle));

$number = 1;
while ($row = $result->fetch_assoc()) {
    $writer->addRow(Row::fromValues([
        $number++,
        export_safe_cell($row['name']),
        export_safe_cell($row['email']),
        export_safe_cell($row['phone'] ?: 'Not provided'),
    ]));
}
$writer->close();
```

Fails **`export/export_class_participants_pdf.php`** – dalībnieku saraksta eksports PDF formātā:

```
<?php
require_once __DIR__ . '/../api/db.php';
require_once __DIR__ . '/../api/csrf.php';
```

```
require_once __DIR__ . '/../vendor/autoload.php';
require_once __DIR__ . '/export_helpers.php';

use Dompdf\Dompdf;

export_require_admin($basePath);

if (!isset($_GET['class_id']) || !ctype_digit((string) $_GET['class_id'])) {
    http_response_code(422);
    exit('Class ID is required.');
}

$classId = (int) $_GET['class_id'];
$classStmt = $conn->prepare('SELECT title, start_datetime FROM classes WHERE id = ?');
$classStmt->bind_param('i', $classId);
$classStmt->execute();
$class = $classStmt->get_result()->fetch_assoc();
if (!$class) {
    http_response_code(404);
    exit('Class not found.');
}

$participants = $conn->prepare("SELECT u.name, u.email, u.phone FROM users u JOIN reservations r ON
u.id =
    r.user_id WHERE r.class_id = ? AND r.status = 'reserved' ORDER BY u.name ASC");
$participants->bind_param('i', $classId);
$participants->execute();
$result = $participants->get_result();

$rows = '';
$number = 1;
while ($participant = $result->fetch_assoc()) {
    $rows .= '<tr><td class="number">' . $number++ . '</td><td>' . export_html($participant['name'])
.
        '</td><td>' . export_html($participant['email']) . '</td><td>' .
export_html($participant['phone'] ?: 'Not
        provided') . '</td></tr>';
}
if ($number === 1) {
    $rows = '<tr><td colspan="4" class="empty">No participants have joined this class
yet.</td></tr>';
}

$logo = '' . base64_encode((string) file_get_contents(__DIR__ .
'/../assets/images/logo2-
    pdf.jpg'));
$html = '<!doctype html><html><head><meta charset="utf-8"><style>
@page{margin:20mm 14mm}body{font-family:DejaVu Sans;color:#1f2937;font-
size:10px}.logo{float:right;width:35mm;height:auto;margin:0 0 4mm 10mm}.brand{color:#2563eb;font-
    weight:bold;letter-spacing:.8px}h1{font-size:22px;margin:7px 0}.meta{color:#64748b;margin:0 0
18px}table{width:100%;border-collapse:collapse}th{background:#1d4ed8;color:#fff;text-
align:left;padding:9px}td{padding:9px;border-bottom:1px solid #dbe3ef}tr:nth-
child(even)td{background:#f8fafc}.number{text-align:center;width:8%}.empty{text-
align:center;padding:25px;color:#64748b}
</style></head><body><img class="logo" src="' . $logo . '"><div
class="brand">MYGOTOGYM</div><h1>Participant
    list</h1><p class="meta"><strong>Class:</strong> ' . export_html($class['title']) .
'<br><strong>Date:</strong>
    ' . date('d.m.Y H:i', strtotime($class['start_datetime'])) . '<br><strong>Generated:</strong>
' . date('d.m.Y
    H:i') . '</p><table><thead><tr><th>No.</th><th>Name</th><th>Email address</th><th>Phone
number</th></tr></thead><tbody>' . $rows . '</tbody></table></body></html>';
```

```
$pdf = new Dompdf();
$pdf->setPaper('A4');
$pdf->loadHtml($html, 'UTF-8');
$pdf->render();
$filename = preg_replace('/[^a-z0-9_-]+/i', '_', $class['title']) . '_participants_' . date('Y-m-d')
. '.pdf';
$pdf->stream($filename, ['Attachment' => true]);
```

Fails **`export/export_excel.php`** – lietotāju un rezervāciju eksports Excel formātā:

```
<?php
require_once __DIR__ . '/../api/db.php';
require_once __DIR__ . '/../vendor/autoload.php';
require_once __DIR__ . '/export_helpers.php';

use OpenSpout\Common\Entity\Row;
use OpenSpout\Common\Entity\Style\Color;
use OpenSpout\Common\Entity\Style\Style;
use OpenSpout\Writer\XLSX\Writer;
use OpenSpout\Writer\XLSX\Options;

if (basename($_SERVER['PHP_SELF'] ?? '') === 'export_excel.php') {
    $query = $_SERVER['QUERY_STRING'] ?? '';
    header("Location: " . $basePath . "/export/export_excel" . ($query ? '?' . $query : ''), true,
301);
    exit;
}

export_require_admin($basePath);

$sql = "
SELECT
  c.id,
  c.title,
  c.description,
  c.start_datetime,
  c.capacity,
  COUNT(r.id) AS booked
FROM classes c
LEFT JOIN reservations r ON c.id = r.class_id
";

$where_clauses = [];
$params = [];
$types = "";

if (isset($_GET['start_date']) && $_GET['start_date'] !== '') {
    $startDate = export_date($_GET['start_date']);
    if (!$startDate) {
        http_response_code(422);
        exit('Invalid start date.');
    }
    $where_clauses[] = "c.start_datetime >= ?";
    $params[] = $startDate . ' 00:00:00';
    $types .= "s";
}
if (isset($_GET['end_date']) && $_GET['end_date'] !== '') {
    $endDate = export_date($_GET['end_date']);
    if (!$endDate) {
        http_response_code(422);
        exit('Invalid end date.');
    }
    $where_clauses[] = "c.start_datetime <= ?";
    $params[] = $endDate . ' 23:59:59';
```

```
    $types .= "s";
}
if (isset($startDate, $endDate) && $startDate > $endDate) {
    http_response_code(422);
    exit('Start date must be before end date.');
}

if (!empty($where_clauses)) {
    $sql .= " WHERE " . implode(" AND ", $where_clauses);
}

$sql .= " GROUP BY c.id ORDER BY c.start_datetime ASC";
$stmt = $conn->prepare($sql);
if (!empty($params)) $stmt->bind_param($types, ...$params);
$stmt->execute();
$result = $stmt->get_result();
$options = new Options();
$options->DEFAULT_ROW_STYLE->setFontSize(9);
$options->DEFAULT_ROW_HEIGHT = 14;
$options->setColumnWidth(8, 1, 5, 6, 7, 8);
$options->setColumnWidth(22, 2);
$options->setColumnWidth(38, 3);
$options->setColumnWidth(18, 4);
$writer = new Writer($options);
$writer->openToBrowser('gym_schedule_report_' . date('Y-m-d') . '.xlsx');

$titleStyle = (new Style())->setFontBold()->setFontSize(14)->setFontColor(Color::WHITE)-
>setBackgroundColor(Color::DARK_BLUE);
$metaStyle = (new Style())->setFontSize(10);
$headerStyle = (new Style())->setFontBold()->setFontSize(9)->setFontColor(Color::WHITE)-
>setBackgroundColor(Color::BLUE);
$writer->addRow(Row::fromValues(['MyGoToGym Schedule Report'], $titleStyle));
$writer->addRow(Row::fromValues(['Generated', date('Y-m-d H:i'), 'Period', trim(($startDate ?? 'All
time') . ' -
    ' . ($endDate ?? 'Today'))], $metaStyle));
$writer->addRow(Row::fromValues([]));
$writer->addRow(Row::fromValues([]));
$writer->addRow(Row::fromValues(['ID', 'Workout title', 'Description', 'Date and time', 'Capacity',
'Booked',
    'Available', 'Attendance'], $headerStyle));

while ($row = $result->fetch_assoc()) {
    $capacity = (int) $row['capacity'];
    $booked = (int) $row['booked'];
    $attendance = $capacity > 0 ? round(($booked / $capacity) * 100, 1) . '%' : '0%';
    $writer->addRow(Row::fromValues([
        (int) $row['id'],
        export_safe_cell($row['title']),
        export_safe_cell($row['description']),
        date('Y-m-d H:i', strtotime($row['start_datetime'])),
        $capacity,
        $booked,
        max(0, $capacity - $booked),
        $attendance,
    ]));
}

$writer->close();
```

Fails **`export/export_helpers.php`** – eksporta palīgfunkcijas:

```
<?php

function export_require_admin(string $basePath): void {
```

```
    if (!isset($_SESSION['user_id'])) {
        header('Location: ' . $basePath . '/login');
        exit;
    }

    if (($_SESSION['role'] ?? '') !== 'admin') {
        http_response_code(403);
        require __DIR__ . '/../403.php';
        exit;
    }
}

function export_date(string $value): ?string {
    $date = DateTime::createFromFormat('!Y-m-d', $value);
    return $date && $date->format('Y-m-d') === $value ? $value : null;
}

function export_safe_cell(?string $value): string {
    $value = trim((string) $value);
    // Spreadsheet programs treat these first characters as formulas. Prefixing
    // them prevents a database value from becoming an executable formula.
    return preg_match('/^[=+\-@]/', $value) ? "'" . $value : $value;
}

function export_html(?string $value): string {
    return htmlspecialchars((string) $value, ENT_QUOTES, 'UTF-8');
}
```

Fails **`export/export_pdf.php`** – datu eksports PDF formātā:

```
<?php
require_once __DIR__ . '/../api/db.php';
require_once __DIR__ . '/../vendor/autoload.php';
require_once __DIR__ . '/export_helpers.php';

use Dompdf\Dompdf;

if (basename($_SERVER['PHP_SELF'] ?? '') === 'export_pdf.php') {
    header('Location: ' . $basePath . '/export/export_pdf' . (!empty($_SERVER['QUERY_STRING']) ? '?'
.
$_SERVER['QUERY_STRING'] : ''), true, 301);
    exit;
}
export_require_admin($basePath);

$start = isset($_GET['start_date']) && $_GET['start_date'] !== '' ? export_date($_GET['start_date'])
: null;
$end = isset($_GET['end_date']) && $_GET['end_date'] !== '' ? export_date($_GET['end_date']) : null;
if ((isset($_GET['start_date']) && $_GET['start_date'] !== '' && !$start) ||
(isset($_GET['end_date']) &&
$_GET['end_date'] !== '' && !$end) || ($start && $end && $start > $end)) {
    http_response_code(422); exit('Invalid date range.');
}
$sql = 'SELECT c.title,c.description,c.start_datetime,c.capacity,COUNT(r.id) AS booked FROM classes
c LEFT JOIN
    reservations r ON c.id=r.class_id';
$where = []; $params = []; $types = '';
if ($start) { $where[] = 'c.start_datetime >= ?'; $params[] = $start . ' 00:00:00'; $types .= 's'; }
if ($end) { $where[] = 'c.start_datetime <= ?'; $params[] = $end . ' 23:59:59'; $types .= 's'; }
if ($where) { $sql .= ' WHERE ' . implode(' AND ', $where); }
$sql .= ' GROUP BY c.id ORDER BY c.start_datetime';
$stmt = $conn->prepare($sql); if ($params) { $stmt->bind_param($types, ...$params); } $stmt-
>execute(); $result =
$stmt->get_result();
```

```
$rows = ''; $count = 0;
while ($row = $result->fetch_assoc()) {
    $count++; $capacity = (int)$row['capacity']; $booked = (int)$row['booked'];
    $rows .= '<tr><td><strong>' . export_html($row['title']) . '</strong></td><td>' .
export_html($row['description']) . '</td><td>' . date('d.m.Y H:i',
strtotime($row['start_datetime'])) . '</td><td
class="center">' . $booked . ' / ' . $capacity . '<br><small>' . ($capacity ? round($booked /
$capacity * 100) :
    0) . '% filled</small></td></tr>';
}
if (!$count) { $rows = '<tr><td colspan="4" class="empty">No workouts found for this
period.</td></tr>'; }
$period = ($start ?: 'All time') . ' - ' . ($end ?: 'Today');
$logo = '' . base64_encode((string) file_get_contents(__DIR__ .
'/../assets/images/logo2-
    pdf.jpg'));
$html = '<!doctype html><html><head><meta charset="utf-8"><style>@page{margin:20mm 14mm}body{font-
family:DejaVu
    Sans;color:#1f2937;font-size:10px}.logo{float:right;width:35mm;height:auto;margin:0 0 4mm
10mm}.brand{color:#2563eb;font-weight:bold;letter-spacing:.8px}h1{font-size:22px;margin:7px
0}.meta{color:#64748b;margin:0 0 18px}table{width:100%;border-
collapse:collapse}th{background:#1d4ed8;color:#fff;text-
align:left;padding:9px}td{padding:9px;border-bottom:1px
    solid #dbe3ef;vertical-align:top}tr:nth-child(even)td{background:#f8fafc}.center{text-
align:center}.empty{text-
    align:center;padding:25px;color:#64748b}small{color:#64748b}</style></head><body><img
class="logo" src="' .
$logo . '"><div class="brand">MYGOTOGYM</div><h1>Workout schedule report</h1><p class="meta">Period:
' .
    export_html($period) . '<br>Generated: ' . date('d.m.Y H:i') .
'</p><table><thead><tr><th>Workout</th><th>Description</th><th>Date and
time</th><th>Attendance</th></tr></thead><tbody>' . $rows . '</tbody></table></body></html>';
$pdf = new Dompdf(); $pdf->setPaper('A4', 'landscape'); $pdf->loadHtml($html, 'UTF-8'); $pdf-
>render(); $pdf-
    >stream('gym_schedule_report_' . date('Y-m-d') . '.pdf', ['Attachment' => true]);
```

Fails **`export/index.php`** – eksporta mapes aizsargfails:

```
<?php

$requestPath = parse_url($_SERVER['REQUEST_URI'] ?? '/', PHP_URL_PATH) ?: '/';
$route = trim(rawurldecode($requestPath), '/');

if (strpos($route, 'export/') === 0) {
    $route = substr($route, strlen('export/'));
} elseif ($route === 'export') {
    http_response_code(404);
    require __DIR__ . '/../404.php';
    exit;
}

$route = trim($route, '/');

if ($route === '' || strpos($route, '..') !== false || strpos($route, "\0") !== false) {
    http_response_code(403);
    require __DIR__ . '/../403.php';
    exit;
}

if (preg_match('/\.php$/i', $route)) {
    $cleanRoute = preg_replace('/\.php$/i', '', $route);
    $query = $_SERVER['QUERY_STRING'] ?? '';
    $basePath = rtrim(str_replace('\\', '/', dirname(dirname($_SERVER['SCRIPT_NAME'] ?? ''))), '/');
```

```
    header('Location: ' . $basePath . '/export/' . $cleanRoute . ($query ? '?' . $query : ''), true,
301);
    exit;
}

$phpFile = __DIR__ . '/' . $route . '.php';
if (!is_file($phpFile)) {
    http_response_code(404);
    require __DIR__ . '/../404.php';
    exit;
}

require $phpFile;
```